Faculty of Artificial Intelligence Engineering

Department of Software Engineering

and Intelligent Information Systems

Public Transportation Platform

Senior2 Project- Completed the requirements for obtaining a bachelor's degree in Informatics Engineering - Software Engineering and Information Systems

Prepared by

Omar Shribati Adnan Alashram

Supervised by

DR. Kadan Joumaa

Eng.Raghad Alhossney

2026

Faculty of Artificial Intelligence Engineering

Department of Software Engineering

and Intelligent Information Systems

منصة للنقل العام

مشروع السنة الثانية - استكمال متطلبات الحصول على درجة البكالوريوس في هندسة المعلوماتية - هندسة البرمجيات ونظم المعلومات

اعداد

محمد عمر شريباتي عدنان الاشرم

اشراف

الدكتور كادان جمعة

المهندسة رغد الحصني

2026

Abstract

The proposed system provides an integrated solution for organizing and managing public transportation services in an efficient and modern way. Traditional transportation systems often suffer from issues such as difficulty in tracking vehicles, poor trip organization, and delays in providing information to users, which can reduce service quality and passenger satisfaction. To address these challenges, the system offers a centralized platform that allows users to easily access transportation services, track trips, view arrival times, and manage bookings in an organized and flexible manner.

The system also improves coordination between passengers, drivers, and transportation management through role-based access and clear workflows that ensure operations are handled efficiently and reliably. In addition, it enables administrators to monitor system performance and manage routes and vehicles, while providing drivers with the necessary tools to update trip statuses and interact with the system directly.

Overall, the system aims to enhance the quality of public transportation services by reducing delays, increasing operational efficiency, facilitating user access to information and services, and improving the overall user experience through a smart and well-structured transportation platform.

الملخص

يتناول هذا المشروع نظامًا ذكيًا ومتطورًا لإدارة خدمات النقل العام وتنظيم عمليات التنقل بين الركاب والسائقين ضمن منصة مركزية متكاملة. تعاني أنظمة النقل التقليدية من مشاكل متعددة مثل صعوبة تتبع المركبات، ضعف تنظيم الرحلات، التأخير في الوصول، وعدم توفر معلومات دقيقة للمستخدمين، مما يؤدي إلى انخفاض كفاءة الخدمة وتراجع مستوى رضا الركاب. يهدف هذا المشروع إلى معالجة هذه المشكلات من خلال تقديم نظام منظم يساعد على تحسين إدارة عمليات النقل وتسهيل الوصول إلى الخدمات بشكل أسرع وأكثر كفاءة.

يوفر النظام بيئة مركزية لإدارة الرحلات والمركبات والمستخدمين، مع دعم إدارة السائقين والركاب وتخصيص الأدوار والصلاحيات بما يضمن تنظيم العمليات بشكل مناسب. كما يساهم النظام في تحسين التنسيق بين مختلف الأطراف من خلال تتبع الرحلات وتحديث حالتها بشكل مستمر، مما يساعد على تقديم تجربة نقل أكثر دقة وفعالية للمستخدمين.

يساهم هذا المشروع في تحسين جودة خدمات النقل العام من خلال تقليل التأخير، رفع كفاءة إدارة الرحلات، تسهيل الوصول إلى المعلومات والخدمات، وتعزيز تجربة المستخدم. ويُعد خطوة نحو تطوير أنظمة النقل التقليدية وتحويلها إلى حلول ذكية وحديثة تعتمد على التنظيم الرقمي وإدارة البيانات لدعم اتخاذ القرارات وتحقيق استجابة أسرع وأكثر كفاءة.

Contents

Abbreviation	Definition
PTP	Public Transportation Platform
API	Application Programming Interface
UI	User Interface
SQL	Structured Query Language
UML	Unified Modeling Language
JWT	JSON Web Token
RTM	Requirements Traceability Matrix
TC	Test Case
Expo	Expo Framework for React Native

Chapter 1 Introduction

1. introduction

In recent years, the rapid development of information and communication technologies has significantly contributed to the emergence of intelligent systems that aim to improve the quality of services, optimize resource management, and enhance user experiences. One of the most important fields that has benefited from these advancements is public transportation, where Intelligent Transportation Systems (ITS) have been introduced to improve transportation efficiency, safety, and accessibility.

Traditional public transportation systems face several challenges, including the lack of real-time vehicle tracking, limited information availability for passengers, inefficient fleet management, and dependence on manual payment and subscription processes. These limitations negatively affect passenger satisfaction and make transportation management more complex.

To address these challenges, the Smart Public Transportation Platform (PTP) has been developed as an integrated digital solution for managing and monitoring public transportation services. The platform connects passengers, drivers, merchants, and administrators within a unified ecosystem that provides real-time information, automated services, and intelligent management capabilities.

The proposed system utilizes modern technologies to provide a complete transportation solution. The backend system is developed using Django REST Framework to manage business logic, data processing, and API services. A mobile application is developed using React Native with Expo to serve passengers, drivers, and merchants. Additionally, an administrative dashboard is developed using React, TypeScript, and Vite to provide management and monitoring capabilities.

The platform includes several advanced features such as real-time vehicle tracking using GPS, instant communication through WebSocket technology, push notification services, QR-based transportation card payment, zone-based subscription management, and an artificial intelligence-based passenger counting system using computer vision techniques with YOLOv8 and ByteTrack.

The main goal of this project is to provide a smart, efficient, and scalable transportation platform that improves public transportation services and supports the concept of smart cities.

2. Problem Statement

The project addresses the current challenges in traditional public transportation systems, where such systems often suffer from difficulties in trip organization, weak vehicle tracking, and delays in providing accurate information to users. These issues can reduce service efficiency and lead to lower passenger satisfaction. In addition, the absence of a unified system for managing transportation operations may cause poor coordination between drivers, administrators, and passengers, as well as difficulties in tracking trip statuses and managing bookings effectively.

These limitations highlight the need for a more organized and efficient solution to improve the management of public transportation services. Therefore, this project proposes a transportation platform that helps organize trips, simplify the management of users and drivers, and improve the monitoring of vehicles and transportation services in a more effective manner. By enhancing trip management processes and providing users with accurate and updated information, the system aims to reduce delays, improve service quality, and enhance the overall user experience in public transportation services.

3. Project Objective

The main objective of this project is to develop an organized and efficient Public Transportation Platform that improves the management of transportation services and facilitates mobility processes through a centralized system that supports the management of trips, vehicles, and users in a clear and flexible manner. The project aims to enhance trip management efficiency, simplify vehicle tracking, and organize booking and coordination processes between passengers, drivers, and administrators, ensuring a more reliable and well-structured transportation service.

The scope of the project focuses on designing and implementing an integrated system that includes user management, driver management, trip and vehicle management, as well as mechanisms for monitoring and organizing transportation operations. The system is designed to support different user roles and flexible workflows that meet the requirements of public transportation management. In addition, an analytical methodology is adopted to define system requirements and organize data effectively, ensuring the clarity, usability, scalability, and future expandability of the proposed platform.

4. Proposed System

The Public Transportation Platform (PTP) is designed as an integrated system that aims to improve the management of transportation services and organize mobility processes between passengers, drivers, and administrators within a unified digital environment. The system seeks to enhance the efficiency of transportation operations by organizing trips, managing vehicles, and facilitating interaction between different users in a clearer and more effective way. In addition, the system addresses many traditional transportation management issues such as poor trip organization, difficulties in vehicle tracking, and delays in providing information to users.

The system provides a set of features that help improve transportation operations, including trip management, vehicle status monitoring, and the organization of booking and coordination processes between passengers and drivers. It also supports continuous updates of trip information, enabling users to access accurate and up-to-date information about available transportation services. Furthermore, the system is designed in a flexible manner that allows future development and expansion according to the needs of transportation institutions or companies.

Moreover, the system provides management tools for passengers, drivers, and administrators through role-based access that ensures clear responsibility distribution and controlled access to system functionalities. By combining trip management, user organization, and transportation operation monitoring within a unified platform, the proposed system contributes to improving service quality, simplifying operational management, and enhancing coordination between all parties involved in public transportation services.

5. Project Scope, Constraints, and Assumptions

5.1 Project Scope

The project includes the development of an integrated Public Transportation Platform that aims to organize and manage transportation services through a unified digital system. The system includes a web platform that allows users to manage trips, monitor vehicles, and organize bookings. It also supports the management of passengers, drivers, and administrators through different access roles and permissions. In addition, the system provides tools for monitoring trip statuses and continuously updating transportation-related information.

5.2 Constraints

The project faces several constraints that may affect the development and implementation process. One of the main constraints is the limited time available for project development, in addition to relying on sample or simulated data instead of real transportation data. Furthermore, some advanced features such as real-time tracking or integration with live map services may be limited due to technical capabilities or available resources during the project development period. Moreover, the system is developed within an academic environment and under a limited budget.

5.3 Assumptions

The project assumes that users have basic knowledge of computer and web application usage, and that they have internet access to use the platform. It is also assumed that drivers and administrators will continuously update trip and vehicle information to ensure the accuracy of the data provided to users. In addition, the system assumes the availability of a suitable operating environment that supports stable and efficient platform performance.

6. Project Contribute

The project introduces a modern and organized approach for managing public transportation services through a unified digital platform that connects passengers, drivers, and administrators within a single system. Unlike traditional transportation management methods that rely on manual coordination and fragmented communication, the proposed platform improves trip organization, booking management, and transportation monitoring in a more efficient and structured way.

The project combines multiple functionalities within one platform, including trip management, vehicle monitoring, booking organization, and role-based user management. This integration helps simplify transportation operations and improves coordination between all parties involved in the transportation process.

In addition, the project provides a flexible and expandable system architecture that can support future enhancements such as real-time tracking, smart route optimization, mobile application integration, or advanced analytics features. The platform also represents a practical case study for applying modern web technologies in the field of transportation management and digital service organization.

7. Research Methodology

This thesis adopts an engineering and descriptive methodology to analyze the challenges of public transportation management and propose an organized digital solution to improve transportation services. The descriptive approach is used to study the existing problems in traditional transportation systems, identify system requirements, and analyze the needs of passengers, drivers, and administrators.

In addition, the engineering methodology is applied in designing and structuring the proposed Public Transportation Platform, including defining system components, organizing workflows, and presenting the technical architecture of the platform. The thesis also focuses on explaining the design decisions, system functionalities, and implementation processes in a clear and systematic manner.

This methodology helps provide a comprehensive understanding of the proposed system and demonstrates how modern digital technologies can be utilized to improve transportation management and enhance the overall quality of transportation services.

8. Development Methodology: Agile Scrum:

The Public Transportation Platform project was developed using the Agile Scrum methodology throughout its development lifecycle. Scrum is an iterative and incremental framework for managing software projects that require continuous feedback, adaptive planning, and frequent delivery of working increments. This methodology was suitable for the project because the system combines several interacting subsystems, including passenger and driver mobile applications, an administrative dashboard, backend APIs, route management, and real-time tracking.

Using Scrum enabled the team to divide the project into clearly defined sprints. Each sprint focused on a specific set of features and concluded with testing and review activities. This approach helped the team validate implemented functionality early, reduce integration risks, and progressively improve the platform based on supervisor feedback and testing results.

8.1 Why Agile Scrum?

The nature of the Public Transportation Platform inherently demanded an Agile approach. The system includes real-time location tracking, route discovery, trip planning, user management, and administrative control. These features require continuous refinement because their correctness depends on integration between backend services, mobile interfaces, database entities, and location-related logic.

A sequential Waterfall model would have delayed integration testing until late in the project, increasing the risk of discovering major incompatibilities between the mobile applications, admin dashboard, and Django backend after most development work had already been completed. Scrum reduced this risk by enforcing incremental delivery, repeated testing, and regular feedback checkpoints.

The three-member team structure also aligned with Scrum principles. Each member had a defined technical responsibility while contributing to documentation and integration tasks. The methodology supported lightweight coordination, fast decision-making, and continuous adjustment of priorities according to development progress and supervisor review feedback.

8.1.1 Scrum Framework Implementation:

The project was executed through five development sprints, beginning with an initial scope determination phase and followed by Sprint 0 through Sprint 4. The following table summarizes the Scrum elements applied in the project:

Scrum Element	Implementation in Project	Frequency/Timing
Product Backlog	**Prioritized list of system features, user stories, and tasks managed through Trello.**	**Continuously refined**
Sprint Planning	**Team planning activity to define sprint goals, select tasks, and assign responsibilities.**	**Start of each sprint**
Sprint Backlog	**Selected tasks for the current sprint including design, backend, frontend, and testing work.**	**Updated during sprint work**
Team Coordination	**Short coordination discussions to review progress, blockers, and integration needs.**	**Weekly or as needed**
Sprint Review	**Review of completed features and supervisor feedback collection.**	**End of each sprint**
Sprint Retrospective	**Reflection on implementation issues, testing results, and improvement actions.**	**End of each sprint**
Increment	**A tested functional increment of the platform delivered at the end of each sprint.**	**Each sprint**
Gantt Chart	**Visual tracking of project schedule, sprint durations, and task dependencies.**	**Updated during planning and review**

Table 1 Scrum Implementation

8.1.2 Sprint Structure:

The project was initially organized into five sprints preceded by a brief scope determination phase. During the first project phase, the team progressed incrementally from analysis and system design to the implementation of account management, buses and routes, real-time tracking, and trip-related functionality. Each sprint focused on delivering a functional part of the system while allowing sufficient time for implementation, integration, and testing.

Scope Determination (March 5, 2026) was a one-day phase in which the team defined the high-level project scope, clarified the main objectives, and established the initial boundaries of the Public Transportation Platform.

Sprint 0: Analysis and Design (March 6 – March 16, 2026) focused on the literature review, project backlog preparation, use case diagram, system specifications, and high-level system architecture. This sprint established the analytical and design foundation for the implementation work that followed.

Sprint 1: Account Management (March 17 – March 27, 2026) focused on user account functionality, authentication, role-based access, passenger and driver profiles, and administrative account operations. Backend implementation was followed by frontend integration and testing.

Sprint 2: Buses and Route Management (March 30 – April 9, 2026) focused on managing the main public transportation entities, including routes, buses, vehicles, stops, and related administrative operations. This sprint established the transportation data required for the subsequent tracking and trip-related functionality.

Sprint 3: Tracking Management (April 10 – April 22, 2026) focused on real-time vehicle tracking. The sprint included vehicle location updates, driver tracking

functionality, passenger-side tracking requirements, and integration testing for live bus monitoring.

Sprint 4: Trip Planning Management (April 23 – May 6, 2026) focused on trip planning, route discovery, arrival-related information, system integration, testing, and final code review. This sprint connected the previously implemented components into a more complete transportation experience for passengers.

Following the completion of the first project phase, development continued during the second graduation project phase. The second phase extended the existing platform by introducing the **transportation payment, transport card, zone, subscription, and merchant components**, together with the required administrative and dashboard functionality.

Phase 2: Payment and Subscription System Development (June 21 – August 8, 2026) focused on extending the existing transportation platform with a zone-based subscription and boarding validation system. The implementation was carried out progressively, beginning with the payment and transport card infrastructure, followed by the development of zones and zone-based subscriptions, merchant functionality, and finally the required administrative and dashboard integration.

The first stage of Phase 2 focused on the **Payment System and Transport Cards**. This stage included the implementation of transport card creation and management, QR-based card identification, payment-related transactions, and the backend services and APIs required to support the new payment workflow. The transport card serves as the passenger's digital transportation identity and provides the basis for subscription management and boarding validation.

The second stage focused on **Zones and Zone-Based Subscriptions**. Administrators can create transportation zones, define their hierarchy and configure

daily, weekly, and monthly subscription prices. Routes are assigned to their corresponding zones, allowing the system to determine whether a passenger's active subscription provides coverage for the route being used. The boarding process validates the passenger's card and checks for an active subscription that covers the zone of the current route.

The third stage focused on the **Merchant Functionality**. A dedicated merchant role was implemented to support the sale and extension of zone-based subscriptions through cash transactions. Merchants can identify a passenger's transport card using its QR token and perform the required subscription operation. The resulting subscription and payment-related transactions are recorded by the backend to provide a traceable history of these operations.

The final stage focused on **Administrative and Dashboard Integration**. The required management functionality was integrated into the administrative dashboard, including zone management, subscription pricing, transport card and passenger subscription information, and access to subscription and transaction records. These components were integrated with the existing passenger, driver, merchant, and backend services to establish a more complete end-to-end transportation workflow.

8.1.3 Project Timeline Plan (Gantt Chart):

The overall project timeline was divided into two main development phases corresponding to the two stages of the graduation project. The first phase spanned from **March 5, 2026 to May 6, 2026**, while the second phase extended from **June 21, 2026 to August 8, 2026**. The Gantt charts present the major activities carried out during each phase and illustrate the progression of the project from its initial analysis and core transportation functionality toward the implementation of the payment and subscription system.

During the **first phase**, the schedule was organized around the scope determination phase and Sprints 0–4. The work progressed from analysis and system design to account management, buses and route management, real-time tracking, and trip planning. This structure allowed the team to develop the core transportation platform incrementally and validate the implemented functionality before moving to the next stage.

During the **second phase**, the development continued from the existing system rather than starting a new project cycle. The work between **June 21 and August 8, 2026** focused on extending the platform with the payment and transport card infrastructure, followed by zones and zone-based subscriptions, merchant functionality, and administrative dashboard integration. The sequence was designed so that each component depended on and extended the functionality implemented in the previous stage.

The Gantt chart demonstrates the overall progression of the project from its initial analysis and core transportation functions to the development of the additional payment and subscription infrastructure. The second phase builds directly on the transportation routes, vehicles, users, and tracking functionality implemented during the first phase. This incremental approach allowed the team to extend the existing system without disrupting its previously implemented components and to gradually integrate the new payment, subscription, merchant, and administrative features into the overall platform.

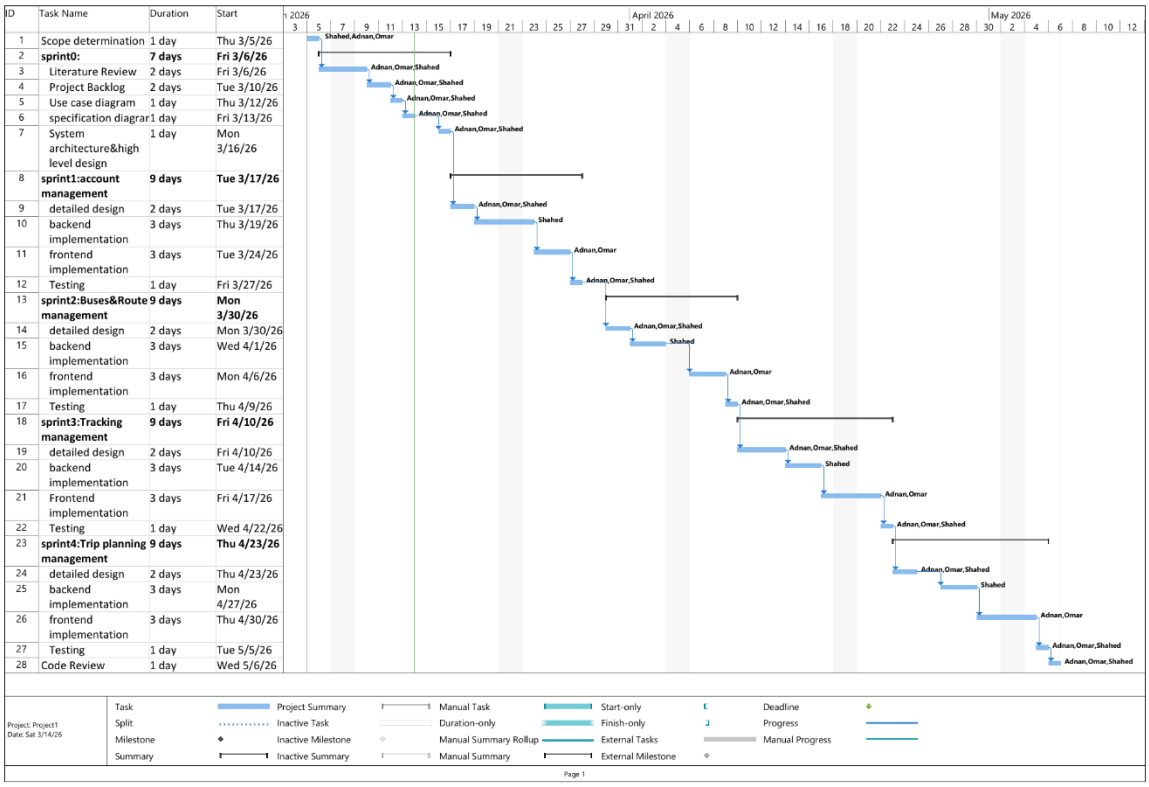


Figure 1: Gantt chart

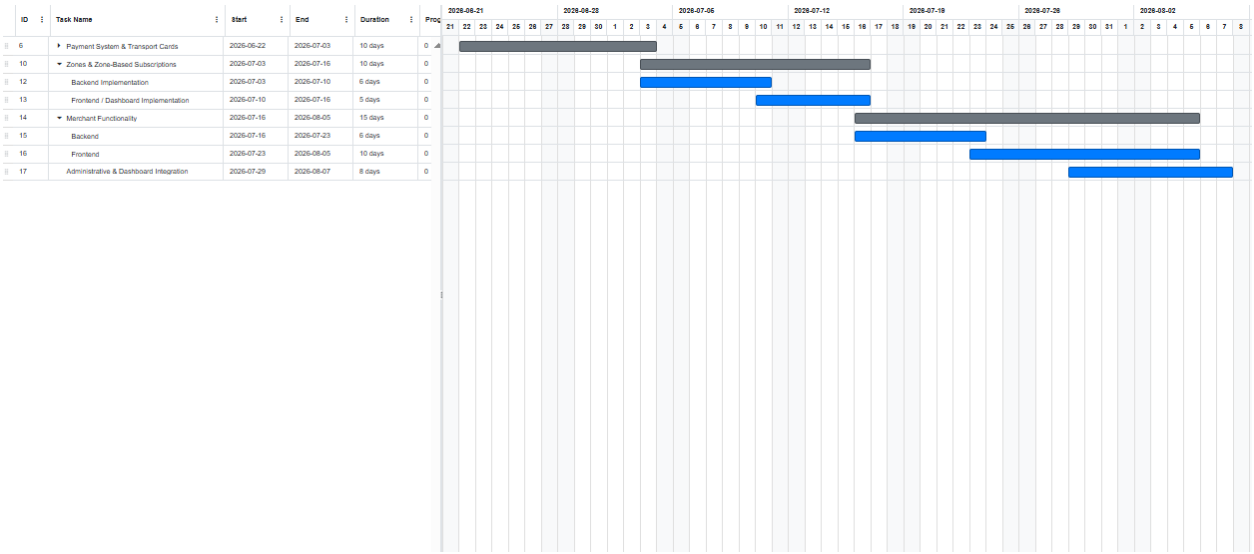


Figure 2: Gantt chart Phase 2

8.2 Risk Management:

Risk management is an essential process for identifying, assessing, and mitigating potential threats to the project. For the Public Transportation Platform, risks were mainly related to real-time tracking accuracy, API integration, database consistency, route planning logic, deployment configuration, and team coordination. The following risk register documents the identified risks and their mitigation strategies.

Table 2 Risk Management Table

Risk ID	Category	Risk Description & Details	Probability	Impact	Status
RK-01	Technical	GPS Accuracy Limitations: inaccurate coordinates may affect bus position display and estimated arrival time.	Medium	High	Partial
RK-02	Technical	Real-Time Tracking Synchronization: delayed or missing driver updates may reduce passenger trust in the system.	Medium	High	Use structured tracking APIs/WebSocket-ready design and test update frequency during integration.

RK-03	Technical	Route and Stop Ordering Errors: incorrectly ordered stops can generate unrealistic route results.	Medium	High	Validate route-stop ordering and allow admin-managed stop creation and route definition.
RK-04	Integration	Frontend/Backend API Mismatch: mobile or dashboard screens may fail if API contracts change.	Medium	Medium	Test endpoints using Postman before frontend integration and document request/response formats.
RK-05	Database	Location Update Volume: frequent location storage may slow database operations.	Low	Medium	Store essential tracking data, index frequently queried fields, and avoid unnecessary historical writes.
RK-06	Security	Unauthorized Access to Admin or Driver Functions: weak role checks may expose sensitive operations.	Medium	High	Apply token-based authentication and role-based restrictions on protected endpoints.
RK-07	Management	Scope Creep: adding unplanned features may delay the final delivery.	Medium	Medium	Prioritize core features through Scrum backlog planning and supervisor review feedback.

RK-08	Deployment	Hosting/Configuration Problems: environment variables, database settings, or server configuration may cause deployment failures.	Medium	Medium	Keep configuration documented, validate setup locally, and maintain the repository as a backup source.
RK-09	Testing	Insufficient End-to-End Testing: integrated workflows may contain hidden defects.	Medium	High	Perform sprint testing, API testing, manual mobile testing, and final code review.

Each identified risk was monitored throughout the project lifecycle. Risks related to tracking, route management, and API integration were given higher priority because they directly affect the core value of the platform. Risk responses were adjusted according to testing results and supervisor review feedback.

8.3 Team Structure and Task Distribution:

The Public Transportation Platform was developed by a team of three students from the software engineering domain. Each member assumed a specific technical role while maintaining cross-functional awareness through regular coordination and shared documentation responsibilities. The project was supervised by Eng. Raghad Alhossny as the project supervisor and work monitoring advisor.

Project title	**Public Transportation Platform**
Project start date	March 5, 2026
Project finish date	May 6, 2026
Project supervisor	DR. Kadan Jumaa

<table>
<tr><td>Project objectives</td><td>Develop a smart public transportation platform that digitizes and enhances public bus transportation by enabling passenger route discovery and live trip tracking, providing real-time vehicle location and occupancy information, supporting zone-based daily, weekly, and monthly transportation subscriptions with QR-code boarding validation, enabling driver trip and location management, providing centralized administrative management of users, drivers, vehicles, routes, stops, zones, subscriptions, complaints, transactions, notifications, and system statistics, and integrating an AI-based passenger counting system using YOLOv8 and ByteTrack to monitor vehicle occupancy in real time.</td></tr>
<tr><td>Approach</td><td>1.Define project scope and objectives.
2. Analyze system requirements and use cases.
3. Design system architecture and database structure.
4. Implement account management.
5. Implement buses and route management.
6. Implement real-time tracking management.
7. Implement trip planning management.
8. Test, review, and document all development phases and outcomes.</td></tr>
<tr><td>Roles and responsibilities</td><td><table><tr><td>Name</td><td>Role</td><td>Responsibility</td></tr><tr><td>Dr.Kadan Jumaa</td><td>Supervisor</td><td>Project supervision and work monitoring</td></tr><tr><td>Eng.Raghad Alhossny</td><td>Supervisor</td><td>Project supervision and work monitoring</td></tr><tr><td>Omar Shribati</td><td>Engineer</td><td>Backend Development - Documentation</td></tr><tr><td>Adnan Alashram</td><td>Engineer</td><td>Frontend Development – Documentation</td></tr></table></td></tr>
<tr><td>Project title</td><td>Public Transportation Platform</td></tr>
<tr><td>Project start date</td><td>March 5, 2026</td></tr>
<tr><td>Project finish date</td><td>May 6, 2026</td></tr>
<tr><td>Project supervisor</td><td>Eng. Raghad Alhossny</td></tr>
<tr><td>Project objectives</td><td>Develop a smart public transportation platform that supports passenger route discovery, real-time bus tracking, arrival time estimation, driver location updates, and administrative management of routes, stops, vehicles, users, trips, complaints, and system statistics.</td></tr>
<tr><td>Approach</td><td>1. Define project scope and objectives.
2. Analyze system requirements and use cases.
3. Design system architecture and database structure.
4. Implement account management.
5. Implement buses and route management.
6. Implement real-time tracking management.
7. Implement trip planning management.
8. Test, review, and document all development phases and outcomes.</td></tr>
<tr><td>Roles and responsibilities</td><td><table><tr><td>Name</td><td>Role</td><td>Responsibility</td></tr><tr><td>Dr. Kadan kumaa</td><td>Supervisor</td><td>Project supervision and work monitoring</td></tr></table></td></tr>
</table>

	Omar Shribati	Engineer	Backend Development - Documentation
	Adnan Alashram	Engineer	Frontend Development - Documentation

8.4 Quality Management in the Project:

Quality management is a critical aspect of the Public Transportation Platform project, ensuring that the delivered system meets the defined requirements and provides reliable, accurate, and usable functionality. The quality management approach covers code quality, testing quality, documentation quality, and user experience quality.

8.4.1 Code Quality Assurance:

Code quality was maintained by organizing backend logic into a clear Django project structure and separating models, serializers, views, and service logic where appropriate. On the frontend side, React Native and React components were developed in a modular manner to improve readability and maintainability. GitHub was used to track changes and preserve source code history.

8.4.2 Testing Quality Assurance:

Testing was performed continuously during and after each sprint. Backend APIs were tested using Postman to validate authentication, route management, account operations, stop management, and admin functions. Manual testing was also performed on the mobile applications and admin dashboard to ensure that frontend screens communicated correctly with backend endpoints.

The testing activities focused on account registration and login, passenger and driver profile operations, admin route and stop management, driver approval workflows, location-related operations, and trip planning behavior. This continuous testing approach helped identify errors early and reduce integration problems.

8.4.3 Documentation Quality Assurance:

Documentation quality was maintained by updating project documentation in parallel with implementation. The team documented sprint progress, system functionality, database entities, API behavior, and project management activities. This ensured consistency between the implemented system and the final report.

8.5 Project Management Tools:

Effective project management required tools for planning, tracking, communication, testing, and collaboration. The selected tools supported the Agile Scrum workflow while matching the actual practices used by the team.

Tool	Category	Purpose	Usage Frequency
MS Project	Project Planning	Gantt chart creation, task scheduling, dependency management, and resource allocation.	Sprint planning and timeline review
Trello	Task Management	Sprint backlog organization, task tracking, to-do lists, and progress visibility.	Daily or weekly
GitHub	Version Control	Source code repository, branch management, commit history, collaboration, and backup of project versions.	Daily

Postman	API Testing	Testing REST API endpoints, validating request bodies, authentication headers, and response formats.	During backend and integration testing
WhatsApp Group	Communication	Quick team communication, urgent coordination, informal updates, and meeting reminders.	Daily

8.5.1 Monitoring and Continuous Evaluation Mechanisms:

Continuous monitoring and evaluation were essential to ensure that the project remained aligned with its objectives and schedule. The team used supervisor reviews, task tracking, GitHub commits, and sprint testing to evaluate progress and identify problems early.

8.5.2 Weekly Team Meetings:

The team conducted regular coordination meetings to review completed tasks, discuss implementation challenges, and plan upcoming work. These meetings focused on sprint progress, technical blockers, integration needs, and documentation updates.

8.5.3 Supervisor Reviews:

Supervisor reviews were conducted with Eng. Raghad Alhossny at key points during the project. These reviews provided external evaluation of progress, technical approach, system behavior, and report quality. Feedback from these reviews was incorporated into sprint work and documentation updates.

8.6 Software Configuration Management (SCM):

Software Configuration Management (SCM) is the discipline of managing software changes in a controlled and traceable manner. For the Public Transportation Platform, SCM practices were implemented using Git and GitHub. The repository was used to store the backend, dashboard, mobile application, project configuration, and documentation-related code artifacts.

8.6.1 Git Repository Structure:

The project source code was managed using a public GitHub repository containing the main project folders and files. The repository includes the Django backend project, the React admin dashboard, the React Native mobile application, configuration files, and project documentation.

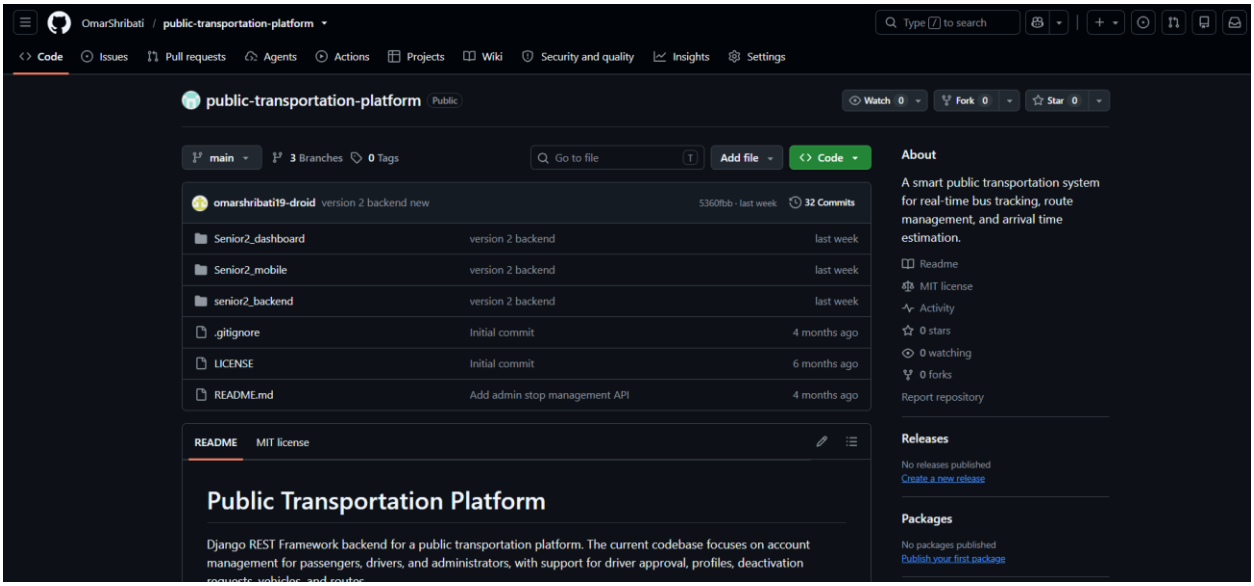


Figure 2:Git Repo Structure

8.6.2 Branching and Merging Strategy:

The project adopted a lightweight branching strategy suitable for a small academic team. The repository currently includes three main branches: main, development, and testing. This structure supports separation between stable code, active development, and validation work.

Branch Name	Purpose & Description	Usage
main	Stable project branch containing the latest integrated version of the system.	Final integrated work and stable commits
development	Development branch used to collect ongoing feature work before it becomes stable.	Backend, dashboard, and mobile feature integration
testing	Testing branch used to validate features and detect issues before final integration.	Testing and verification of integrated work

Team Members and Responsibilities:

Team Member	Role	Responsibilities
Omar Shribati	Backend Developer - Documentation	Backend API development, database integration, authentication logic, route and tracking backend support, documentation preparation.

Adnan Alashram	Frontend Developer - Documentation	Frontend implementation, mobile UI components, dashboard/frontend support, testing assistance, and documentation support.
Team Member	Role	Responsibilities

8.6.3 Collaborative Workflow - Shared GitHub Integration:

The team used GitHub as the central collaboration platform. The repository currently shows no formal pull requests, so the workflow is documented as a lightweight shared-branch integration process rather than a strict pull-request workflow. In this process, developers implemented features locally, tested them, pushed changes to GitHub, synchronized with the shared branches, and coordinated integration through team communication.

Step	Workflow Activity	Description
Step 1	**Task Assignment**	**Tasks were assigned according to sprint priorities and team member responsibilities.**
Step 2	**Local Development**	**Each member implemented assigned functionality locally using the appropriate project component.**
Step 3	**Testing**	**Backend endpoints were tested using Postman and frontend behavior was manually verified.**
Step 4	**Push to GitHub**	**Validated changes were pushed to the repository and synchronized with shared branches.**
Step 5	**Integration**	**Integrated code was checked through testing and supervisor review feedback.**
Step 6	**Code Review**	**The final development stage included a code review task to verify project stability before submission.**

8.7 Development Workflow:

The development workflow followed an end-to-end lifecycle for each feature: task identification, detailed design, backend implementation, frontend implementation, testing, integration, and documentation. This workflow was repeated across the four implementation sprints to ensure consistency and traceability.

Backend functionality was implemented and validated first when API behavior was required. After API validation, frontend screens were connected to backend endpoints. This sequence reduced integration conflicts and allowed the team to test each feature from both the server-side and user-interface perspectives.

Summary:

The SCM practices adopted by the Public Transportation Platform team provided a practical framework for coordinating development across three team members. GitHub preserved source code history, the main-development-testing branch structure supported lightweight collaboration, and continuous testing helped maintain stability.

Chapter 2 Theoretical study and analysis

1. Previous Studies

A. Artificial Intelligence for Improving Public Transport (2023)

This study focused on the use of artificial intelligence techniques to improve public transportation services and explored how AI can be applied in transportation management, performance monitoring, and user experience enhancement. The study also discussed several AI applications such as fault prediction, data analysis, and traffic management.

What the study provided:

- A comprehensive analysis of AI applications in public transportation.
- Improved transportation management through intelligent data analysis.
- Various approaches for enhancing service quality.

Limitations:

- Focused more on theoretical research than practical implementation.
- Did not provide an integrated platform for managing users, trips, and bookings.
- Required a practical system that could be directly applied.

B. Smart Transit with GPS Based Real-Time Bus Tracking (2025)

This study introduced a GPS-based real-time bus tracking system using cloud and web technologies to provide live bus location information and improve passenger experience.

What the study provided:

- Real-time bus tracking using GPS technology.
- A web interface for displaying live bus locations.
- Improved estimation of arrival times.

Limitations:

- Did not support integrated booking or user management.
- Focused mainly on tracking functionality.
- Lacked a complete role-based management system for administrators and drivers.

C. Passenger Flow Prediction and Management Method (2023)

This study proposed a model based on neural networks and deep learning techniques to predict passenger flow in public transportation systems and improve passenger management.

What the study provided:

- Improved passenger flow prediction accuracy.
- Application of deep learning techniques in transportation analysis.
- Support for trip planning and congestion management.

Limitations:

- Relied heavily on AI techniques and large datasets.
- Did not provide a complete platform for trip and user management.
- Required extensive transportation data to operate effectively.

D. A Simulation-Based Optimization Approach for Designing Transit Networks (2023)

This study focused on designing and optimizing public transportation networks using simulation techniques to improve transportation efficiency and reduce waiting times.

What the study provided:

- Optimization of public transportation network design.
- Reduction of waiting times and improved operational efficiency.
- Use of simulation models for transportation analysis.

Limitations:

- Focused mainly on transportation planning aspects.
- Did not provide a real operational platform for passengers and drivers.
- Did not support booking or account management features.

E. Sustainability Analysis Framework for On-Demand Public Transit Systems (2023)

This study proposed a framework for analyzing the sustainability of on-demand public transportation systems in terms of efficiency, environmental impact, and service quality.

What the study provided:

- Evaluation of transportation system sustainability and efficiency.
- Analysis of environmental and operational impacts.
- Support for improving transportation-related decision making.

Limitations:

- Focused more on theoretical analysis than practical implementation.

- Did not provide a practical platform for transportation management.
- Lacked direct support for daily transportation operations and user interaction.

Although many studies have addressed smart public transportation systems and transportation management, most of them focused on a specific aspect such as tracking, prediction, or data analysis. In contrast, our project aims to provide an integrated platform that combines trip management, user management, driver management, and booking functionalities within a unified and user-friendly system.

2. Existing Systems

The purpose of this Literature Review is to analyze and evaluate existing systems and methodologies related to the development of public transportation platforms and transportation management solutions. This study aims to provide a comprehensive overview of the current landscape in public transportation services, focusing on how modern web technologies, transportation management systems, tracking solutions, and digital platforms are utilized to improve operational efficiency and enhance user experience.

The analysis includes comparisons between several transportation-related systems and their key features, such as trip management, vehicle tracking, booking organization, user management, and coordination between passengers, drivers, and administrators. The review also highlights the limitations of existing solutions and identifies the aspects that can be improved through the proposed Public Transportation Platform.

- Transit . subway & bus time

https://www.transitapp.com/

Transit is a public transportation and mobility app that helps you plan your city travel using buses, trains, the metro, bicycles, and other journeys without needing a car.

- Citymapper

https://citymapper.com/

is a global urban mobility application that provides real-time public transport tracking, route planning, and arrival time estimation across multiple cities worldwide.

It supports multi-modal transportation including buses, metro, trains, and walking with live data integration

- Moovit

https://moovit.com/

Moovit is a public transportation application that provides multimodal trip planning and real-time transit information for urban mobility.

It enables users to search routes, view live bus arrivals, receive service alerts, and navigate step-by-step using an interactive map interface.

Table 3 Comparison of Reference study

Feature	Transit	Citymapper	Moovit	PPT(our system)
Route planning	✗	✓	✓	✓*
Real-Time Tracking	✓	✓	✓	✓*
Nearby buses	✓	✓	✓	✓*
ETA Prediction	✓	✓	✓	✓
Multi-modal Transport	✓	✓	✓	✗
Route Management	✗	✗	✗	✓*
Bus Management	✗	✗	✗	✓*
Driver Management	✗	✗	✗	✓*
Geographic Scalability	✓	✓	✓	✗

Features marked with the symbol (*) indicate that they will be fully completed during this semester.

3. Research Gap

Based on the reviewed studies and existing transportation systems, several important limitations and unresolved issues can be identified. Although many solutions exist for public transportation management, most of them focus on individual functionalities such as route planning, real-time tracking, passenger flow prediction, or data analysis, rather than providing a complete integrated system.

One of the main gaps is the lack of a unified platform that combines all essential transportation operations in a single system. Existing solutions often separate functionalities such as trip management, vehicle tracking, booking, and user management, which leads to fragmentation and inefficient coordination between system components.

Another limitation is the insufficient support for full operational management, where most systems are designed mainly for passengers and do not provide strong administrative and driver management features. This includes limited control over trip scheduling, driver assignment, and system-wide management of transportation operations.

Additionally, many existing systems focus heavily on technical aspects such as prediction or tracking, but do not provide a complete real-world operational platform that can be used by all stakeholders (passengers, drivers, and administrators) in a unified environment.

Therefore, the main gap identified is the absence of an integrated public transportation platform that combines trip management, vehicle management, user and driver management, and booking functionalities within a single, easy-to-use system. This gap represents the main motivation behind the development of the proposed Public Transportation Platform.

4. Theoretical Framework

The proposed Smart Public Transportation Platform is based on several fundamental concepts in software engineering and modern application development. These concepts provide the technical foundation that enables the system to operate in a structured, efficient, and scalable way. They include client–server architecture, RESTful APIs, database management, authentication and authorization, and software design principles. They can be summarized as follows:

4.1 Client–Server Architecture

The system follows a client–server architecture, where client applications communicate with the backend server through defined interfaces. The mobile application and the admin dashboard send requests to the backend, which processes the requests, applies the required business logic, and returns the appropriate data. This separation improves system organization and allows the frontend and backend components to be developed and maintained independently.

4.2 REST API Architecture

The platform uses RESTful APIs to enable communication between the client applications and the backend. Data exchange is performed using standard HTTP methods such as GET, POST, PATCH, and DELETE. The REST API provides structured endpoints for authentication, route and trip management, passenger services, payment and subscription operations, notifications, and other system functionalities.

4.3 Database Management System

A relational database management system is used to store and organize the platform's data. The system uses MySQL to manage information related to users, drivers, vehicles, routes, stops, trips, vehicle locations, transport cards, zone subscriptions, payment transactions, and notifications. This ensures structured data storage, consistency between related entities, and efficient data retrieval.

4.4 Authentication & Authorization

The system implements authentication and authorization mechanisms to control access to its different functionalities. Role-based access control is used to provide appropriate permissions for passengers, drivers, merchants, and administrators. In addition, bus scanner devices use a dedicated authentication mechanism based on a device secret key. These mechanisms ensure that each user or device can access only the resources and operations authorized for its role.

4.5 Software Design Principles

The platform follows modern software design principles such as scalability, maintainability, modularity, and separation of concerns. These principles are reflected in the separation of the backend services, mobile application, admin dashboard, AI passenger-counting service, and scanner simulator. This structure allows individual components to be developed, updated, and maintained independently while reducing the impact of changes on other parts of the system.

5. Feasibility Study

5.1 Technical Feasibility

The proposed Smart Public Transportation Platform is technically feasible as it is developed using widely adopted technologies such as Django REST Framework, React Native, React, TypeScript, MySQL, and Django Channels. The platform also integrates additional technologies for specific requirements, including WebSocket communication for real-time features and YOLOv8 with ByteTrack for AI-based passenger counting. These technologies are well-supported and provide extensive documentation and libraries that facilitate the development process.

The system can be developed and tested using commonly available development hardware and software tools. Although the AI passenger-counting component requires additional computational resources compared with the main web and mobile components, it can operate as a separate service, allowing the rest of the platform to function independently. The development technologies and frameworks used in the project are generally available without requiring proprietary software licenses.

Therefore, from a technical perspective, the proposed platform is feasible to implement using the available technologies, development tools, and resources.

5.2 Financial Feasibility

The project is considered financially feasible because most of the technologies and development tools used are open-source or available without licensing fees. The main development environment can therefore be established with relatively low costs. However, deploying the complete system in a real-world environment may involve additional operational costs, such as:

• Cloud hosting services for the backend and database.

• Domain name registration.

• Hosting or infrastructure for the admin dashboard and other system components.

• Additional infrastructure or hardware for the bus-mounted camera and scanner devices.

• Optional costs related to third-party services used for mapping, geocoding, or notification delivery, depending on the final deployment configuration.

Overall, the development of the proposed system is financially feasible for an academic project, while a real-world deployment would require additional infrastructure and operational costs depending on the scale of the transportation network.

5.3 Stakeholders Identification and Needs Analysis

The system has several stakeholders who benefit from the project and interact with it directly or indirectly. Each stakeholder has specific needs and expectations from the system.

1. Passenger (End User):

The passenger is one of the main stakeholders in the system. Passengers need an easy-to-use mobile application that allows them to view available bus routes, track buses in real time, and find nearby transportation lines. They can also manage their transport card, view active zone subscriptions, and access their boarding and subscription history. They expect accurate location and occupancy information, fast performance, and a simple user interface that improves their transportation experience and reduces waiting time.

2. Driver:

Drivers use the system to manage and track their assigned trips. They need a reliable application that allows them to start and stop trips, view their assigned route, and send live location updates. The system also provides passenger occupancy information generated by the AI passenger-counting service and can detect route deviations. Drivers expect the system to operate smoothly and reliably during their trips.

3. System Administrator (Admin):

The administrator is responsible for managing the entire system. The admin needs tools to manage routes, vehicles, drivers, users, transport cards, scanner devices, zones, merchants, and system notifications. The administrator also needs access to transaction records and monitoring features to ensure accurate data and effective control over system operations.

4. Merchant :

Merchants provide services related to transport cards and subscriptions. They need an interface that allows them to look up passenger cards using QR tokens, sell or extend zone subscriptions, perform card balance top-ups, and view their transaction history. Merchants expect a simple and reliable system that allows transactions to be processed quickly and accurately.

5. Future Developer:

Future developers are considered indirect stakeholders. They need a well-structured and maintainable system that is easy to update and improve in the future. Clear code organization, modular architecture, proper documentation, and separation of system components are important for future development and maintenance.

5.4 High-Level Requirements:

Passenger Requirements:

- **User management:** The system shall allow passengers to register, log in, and manage their personal profiles, including requesting account deactivation when needed.
- **Trip planning and route discovery:** The system shall enable passengers to search for routes and trips and to view detailed route information, including stops, path, and pricing.
- **Live bus tracking:** The system shall provide real-time tracking of buses and active trips, allowing passengers to monitor vehicle location on a map while a trip is in progress.
- **Transport card and subscription management**: The system shall allow passengers to own a transport card and to view their card balance, active and past zone subscriptions, and subscription purchase history.

- **Boarding validation**: The system shall validate a passenger's transport card when boarding a bus by checking the card status and the validity of the passenger's zone subscription against the route being boarded.
- **Favorites management:** The system shall allow passengers to save and manage favorite routes for quick access.
- **Complaint submission:** The system shall allow passengers to submit complaints, optionally including a supporting image.
- **Notifications:** The system shall notify passengers of relevant events, such as boarding confirmation, through push notifications and in-app delivery.

Driver Requirements:

- **Account and profile management:** The system shall allow drivers to register by submitting the required identification documents and to manage their profile information, subject to administrator approval.
- **Trip operation:** The system shall allow drivers to start and stop trips on their assigned vehicle and route, and to view their current trip status.
- **Live location reporting:** The system shall allow drivers' vehicles to report live GPS location while a trip is active, supporting real-time tracking and route-deviation detection.
- **Notifications:** The system shall notify drivers of relevant events, such as changes to their approval status or route-deviation alerts.

Admin Management:

- **Driver approval:** The system shall allow administrators to review, approve, or reject driver registration requests.
- **Route, stop, and vehicle management**: The system shall allow administrators to create and manage routes, stops, and vehicles, including assigning vehicles to routes and generating route paths.
- **Zone and pricing management**: The system shall allow administrators to define transportation zones and configure subscription prices for each zone.
- **Transport card and device management**: The system shall allow administrators to issue transport cards, manage card status, and register bus scanner devices.
- **Monitoring and reporting**: The system shall provide administrators with platform-wide statistics, transaction records, and live vehicle tracking.
- **Complaint handling**: The system shall allow administrators to view complaints submitted by passengers.
- **Notification management**: The system shall allow administrators to send notifications to individual users or broadcast notifications to all passengers.

Merchant Requirements:

- **Card lookup**: The system shall allow merchants to look up a passenger's transport card and view its current zone subscriptions.
- **Subscription sales**: The system shall allow merchants to sell or extend a zone subscription for a passenger's transport card in exchange for cash payment.
- **Transaction history**: The system shall allow merchants to view their own subscription sale transaction history.

- **Zone reference data:** The system shall allow merchants to view available zones and their subscription prices in order to complete a sale.

System / Platform Requirements

- **Real-time communication:** The system shall support real-time, bidirectional updates for vehicle tracking, trip tracking, and notifications.
- **Automated passenger counting:** The system shall support automatic counting of passengers boarding and alighting a vehicle using an onboard camera-based detection service.
- **Secure, role-based access**: The system shall enforce distinct authentication and access rules for passengers, drivers, merchants, administrators, and bus scanner devices.
- **Auditable transactions**: The system shall maintain a record of subscription purchases and boarding attempts, including both successful and rejected attempts, for audit purposes.

5.5 System Features

5.5.1 Functional Requirements

5.5.1 User Authentication and Account Management

REQ-1.1:

The system shall allow passengers and drivers to create new accounts by providing the required personal information.

REQ-1.2:

The system shall require administrator approval before a newly registered driver account becomes active.

REQ-1.3:

The system shall allow users to log in using valid email and password credentials.

REQ-1.4:

The system shall issue an authentication token after successful user login.

REQ-1.5:

The system shall allow authenticated users to log out from the application.

REQ-1.6:

The system shall allow passengers and drivers to view their profile information.

REQ-1.7:

The system shall allow passengers and drivers to update their profile information.

REQ-1.8:

The system shall allow passengers and drivers to submit account deactivation requests.

REQ-1.9:

- The system shall provide appropriate authentication mechanisms for passengers, merchants, administrators, drivers, and scanner devices according to their roles.

5.5.2 Passenger Features

REQ-2.1:

The system shall allow passengers to search for available routes and trips.

REQ-2.2:

The system shall allow passengers to view route details, including stops, route path, and pricing information.

REQ-2.3:

The system shall allow passengers to track an active trip in real time on a map.

REQ-2.4:

The system shall allow passengers to add routes to their favorite routes.

REQ-2.5:

The system shall allow passengers to view their favorite routes.

REQ-2.6:

The system shall allow passengers to remove routes from their favorite routes.

REQ-2.7:

The system shall allow passengers to submit complaints.

REQ-2.8:

The system shall allow passengers to attach an optional image when submitting a complaint.

REQ-2.9:

The system shall allow passengers to view their transport card information.

REQ-2.10:

The system shall allow passengers to view their card balance and active zone subscriptions.

REQ-2.11:

The system shall allow passengers to view their subscription purchase history.

REQ-2.12:

The system shall allow passengers to view their boarding and payment transaction history.

REQ-2.13:

The system shall allow passengers to receive system notifications, including successful boarding notifications.

5.5.3 Driver Features:

REQ-3.1:

The system shall allow drivers to register by submitting the required identification and driving license documents.

REQ-3.2:

The system shall allow drivers to view their profile information.

REQ-3.3:

The system shall allow drivers to update their profile information.

REQ-3.4:

The system shall allow drivers to start an assigned trip using an assigned vehicle and route.

REQ-3.5:

The system shall allow drivers to stop an active trip.

REQ-3.6:

The system shall allow drivers to retrieve the status of their current trip.

REQ-3.7:

The system shall allow the system to receive live GPS location updates from drivers during active trips.

REQ-3.8:

The system shall automatically detect route deviations based on the driver's reported GPS location.

REQ-3.9:

The system shall generate an alert when a vehicle exceeds the configured route-deviation distance threshold.

REQ-3.10:

The system shall notify drivers about relevant account approval status changes and route-deviation events.

5.5.4 Admin Features:

REQ-4.1:

The system shall allow administrators to manage passenger, driver, and merchant accounts.

REQ-4.2:

The system shall allow administrators to create, update, activate, and deactivate user accounts where applicable.

REQ-4.3:

The system shall allow administrators to review driver applications.

REQ-4.4:

The system shall allow administrators to approve or reject driver applications.

REQ-4.5:

The system shall allow administrators to manage routes, stops, and vehicles.

REQ-4.6:

The system shall allow administrators to assign vehicles to routes.

REQ-4.7:

The system shall allow administrators to generate route paths.

REQ-4.8:

The system shall allow administrators to manage transportation zones and their hierarchy levels.

REQ-4.9:

The system shall allow administrators to configure daily, weekly, and monthly subscription prices for transportation zones.

REQ-4.10:

The system shall allow administrators to issue transport cards to passengers.

REQ-4.11:

The system shall allow administrators to activate or block transport cards.

REQ-4.12:

The system shall allow administrators to register and manage bus scanner devices.

REQ-4.13:

The system shall allow administrators to view platform-wide statistics and reports.

REQ-4.14:

The system shall allow administrators to view all boarding and payment transactions.

REQ-4.15:

The system shall allow administrators to view all subscription sale transactions.

REQ-4.16:

The system shall allow administrators to manage merchants and view merchant transaction histories.

REQ-4.17:

The system shall allow administrators to monitor the live location of any active vehicle.

REQ-4.18:

The system shall allow administrators to review passenger complaints.

REQ-4.19:

The system shall allow administrators to send notifications to specific users.

REQ-4.20:

The system shall allow administrators to broadcast notifications to all passengers.

5.5.5 Merchant Features

REQ-5.1:

The system shall allow merchants to look up a passenger's transport card using its QR token.

REQ-5.2:

The system shall allow merchants to view the active zone subscriptions associated with a transport card.

REQ-5.3:

The system shall allow merchants to sell zone subscriptions on behalf of passengers.

REQ-5.4:

The system shall allow merchants to extend existing zone subscriptions.

REQ-5.5:

The system shall support daily, weekly, and monthly zone subscription durations.

REQ-5.6:

The system shall allow merchants to view available transportation zones and their subscription prices.

REQ-5.7:

The system shall allow merchants to view their own subscription sale transaction history.

5.5.6 Public Transportation and Route Management:

REQ-6.1:

The system shall allow administrators to define transportation routes.

REQ-6.2:

The system shall allow each route to be associated with a specific transportation zone.

REQ-6.3:

The system shall allow administrators to define route start and end coordinates.

REQ-6.4:

The system shall allow administrators to generate and store route paths.

REQ-6.5:

The system shall allow administrators to define transportation stops.

REQ-6.6:

The system shall allow administrators to assign stops to routes in a specific order.

REQ-6.7:

The system shall allow administrators to register vehicles.

REQ-6.8:

The system shall allow administrators to assign vehicles to routes.

REQ-6.9:

The system shall allow administrators to manage the active or inactive status of routes.

5.5.7 Real-Time Tracking:

REQ-7.1:

The system shall broadcast vehicle location updates in real time during active trips.

REQ-7.2:

The system shall allow passengers to track active trips in real time.

REQ-7.3:

The system shall allow administrators to monitor vehicles in real time.

REQ-7.4:

The system shall automatically detect deviations between the vehicle's reported location and its assigned route.

REQ-7.5:

The system shall generate route-deviation alerts when the configured distance threshold is exceeded.

REQ-7.6:

The system shall include the current passenger count, vehicle capacity, occupancy percentage, and full-status information in live tracking updates when available.

5.5.8 Passenger Counting / AI Features:

REQ-8.1:

The system shall provide a dedicated camera-based service for passenger counting.

REQ-8.2:

The AI service shall detect and track people captured by the bus camera.

REQ-8.3:

The AI service shall detect people crossing a virtual counting line.

REQ-8.4:

The system shall classify detected line crossings as passenger boarding or alighting events.

REQ-8.5:

The system shall maintain a running passenger count for each vehicle.

REQ-8.6:

The AI service shall periodically send the current passenger count to the backend.

REQ-8.7:

The passenger-count update shall be authenticated using the driver's authentication credentials.

REQ-8.8:

The system shall update the vehicle occupancy information based on the reported passenger count.

REQ-8.9:

The system shall make the updated passenger count available to connected applications through the real-time tracking mechanism.

5.5.9 Notification System:

REQ-9.1:

The system shall allow passengers and drivers to receive notifications through push notifications.

REQ-9.2:

The system shall provide real-time in-app notification delivery through a WebSocket channel.

REQ-9.3:

The system shall store generated notifications in the database.

REQ-9.4:

The system shall notify drivers about account approval or rejection status.

REQ-9.5:

The system shall notify passengers about successful boarding transactions.

REQ-9.6:

The system shall allow administrators to send notifications to individual users.

REQ-9.7:

The system shall allow administrators to broadcast notifications to all passengers.

REQ-9.8:

The system shall maintain the read and unread status of notifications.

REQ-9.9:

The system shall allow users to mark individual notifications as read.

REQ-9.10:

The system shall allow users to mark all notifications as read.

REQ-9.11:

The system shall allow users to retrieve their unread notification count.

5.5.10 Transport Card and Payment System:

REQ-10.1:

The system shall allow administrators to issue a unique transport card for each passenger.

REQ-10.2:

The system shall assign a unique card number and QR token to each transport card.

REQ-10.3:

The system shall support active, inactive, and blocked transport card statuses.

REQ-10.4:

The system shall allow the bus scanner to validate a transport card using its QR token.

REQ-10.5:

The system shall validate the card status before granting boarding access.

REQ-10.6:

The system shall validate whether the passenger has an active subscription covering the zone of the boarded route.

REQ-10.7:

The system shall grant boarding access when the transport card satisfies the required validation conditions.

REQ-10.8:

The system shall reject boarding when the transport card does not satisfy the required validation conditions.

REQ-10.9:

The system shall record every boarding attempt, including successful and rejected attempts.

REQ-10.10:

The system shall record the outcome status and relevant information for each payment transaction.

REQ-10.11:

The system shall provide a cash balance top-up capability for transport cards through authorized merchants.

5.5.11 Subscription and Zone Management:

REQ-11.1:

The system shall allow administrators to define transportation zones.

REQ-11.2:

The system shall allow administrators to assign a hierarchy level to each transportation zone.

REQ-11.3:

The system shall allow administrators to configure daily, weekly, and monthly subscription prices for each zone.

REQ-11.4:

The system shall allow administrators to assign each route to a specific transportation zone.

REQ-11.5:

The system shall allow merchants to purchase zone subscriptions on behalf of passengers.

REQ-11.6:

The system shall allow merchants to extend existing zone subscriptions.

REQ-11.7:

The system shall support daily, weekly, and monthly subscription types.

REQ-11.8:

The system shall allow a passenger to hold active subscriptions for multiple zones simultaneously.

REQ-11.9:

The system shall automatically select the most appropriate active subscription that covers the zone of the boarded route.

REQ-11.10:

The system shall record each subscription purchase as a subscription transaction.

5.5.12 Complaint and Reporting Features

REQ-12.1:

The system shall allow passengers to submit complaints.

REQ-12.2:

The system shall allow passengers to attach an optional image to a complaint.

REQ-12.3:

The system shall allow administrators to review submitted complaints.

REQ-12.4:

The system shall provide platform-wide statistical reports for administrators.

5.6 Functional Requirements

Account Management

FR-01: The system shall allow passengers to register by providing the required personal information.

FR-02: The system shall allow drivers to register by providing the required personal information and identification/license documents.

FR-03: The system shall subject newly registered driver accounts to administrator approval before the account becomes usable.

FR-04: The system shall allow users to log in using valid email and password credentials.

FR-05: The system shall allow users to log out, invalidating their current authentication token.

FR-06: The system shall allow passengers and drivers to view their profile information.

FR-07: The system shall allow passengers and drivers to update their profile information.

FR-08: The system shall allow passengers and drivers to request deactivation of their account.

Trip Planning and Route Information

FR-09: The system shall allow passengers to search for available trips and routes.

FR-10: The system shall display detailed route information, including associated stops, route path, and price.

FR-11: The system shall allow passengers to save and manage favorite routes.

Real-Time Tracking

FR-12: The system shall provide passengers with real-time tracking of an active trip.

FR-13: The system shall provide administrators with real-time tracking of any vehicle.

FR-14: The system shall receive and process live GPS location updates submitted by drivers during an active trip.

FR-15: The system shall detect when a driver's reported location deviates from the assigned route beyond a defined distance threshold and generate a corresponding alert.

Driver Trip Operations

FR-16: The system shall allow drivers to start a trip on their assigned vehicle and route.

FR-17: The system shall allow drivers to stop an active trip.

FR-18: The system shall allow drivers to retrieve the status of their current trip.

Administrator — Account and Driver Management

FR-19: The system shall allow administrators to view, create, and update passenger, driver, and merchant accounts.

FR-20: The system shall allow administrators to activate or deactivate passenger, driver, and merchant accounts.

FR-21: The system shall allow administrators to review pending driver applications.

FR-22: The system shall allow administrators to approve or reject driver applications.

Administrator — Route, Stop, and Vehicle Management

FR-23: The system shall allow administrators to create and update routes, including assigning each route to a transportation zone.

FR-24: The system shall allow administrators to create and update stops.

FR-25: The system shall allow administrators to assign stops to routes in a defined order.

FR-26: The system shall allow administrators to register and manage vehicles.

FR-27: The system shall allow administrators to assign vehicles to routes.

Administrator — Reporting and Complaint Handling

FR-28: The system shall provide administrators with platform-wide statistics.

FR-29: The system shall allow administrators to view complaints submitted by passengers.

Transport Card and Zone Subscription System

FR-30: The system shall allow administrators to issue a transport card for a passenger.

FR-31: The system shall allow administrators to activate or block a transport card.

FR-32: The system shall allow administrators to define transportation zones, including a hierarchy level and daily, weekly, and monthly subscription prices.

FR-33: The system shall allow merchants to look up a passenger's transport card using its associated QR token.

FR-34: The system shall allow merchants to view a looked-up card's current zone subscriptions.

FR-35: The system shall allow merchants and administrators to view the list of available zones and their subscription prices.

FR-36: The system shall allow merchants to sell a zone subscription (daily, weekly, or monthly) for a passenger's transport card in exchange for cash payment.

FR-37: The system shall extend an existing, unexpired zone subscription when a merchant sells a subscription for the same zone and card.

FR-38: The system shall allow a transport card to hold concurrent active subscriptions for more than one zone.

FR-39: The system shall record every zone subscription sale as a transaction associated with the card, the merchant, and the zone.

FR-40: The system shall allow passengers to view their own active and past zone subscriptions.

FR-41: The system shall allow passengers to view their own subscription purchase history.

FR-42: The system shall allow administrators to view all subscription sale transactions.

FR-43: The system shall allow administrators to view a passenger's transport card together with its associated zone subscriptions.

FR-44: The system shall allow merchants to add cash balance to a passenger's transport card.

FR-45: The system shall allow merchants to view their own top-up transaction history.

Boarding Validation

FR-46: The system shall allow administrators to register a bus scanner device and associate it with a vehicle.

FR-47: The system shall authenticate bus scanner devices using a device-specific secret key.

FR-48: The system shall validate a scanned transport card's status when a passenger boards a vehicle.

FR-49: The system shall determine the zone of the route associated with the vehicle's active trip.

FR-50: The system shall determine whether the scanned card holds an active, unexpired zone subscription that covers the route's zone.

FR-51: The system shall select the subscription providing the broadest applicable zone coverage when a card holds multiple active subscriptions.

FR-52: The system shall grant boarding access when a valid, covering subscription is found, and deny access otherwise.

FR-53: The system shall record every boarding attempt, whether granted or denied, together with the specific outcome.

FR-54: The system shall send a notification to the passenger upon a successful boarding validation.

Passenger Counting

FR-55: The system shall receive passenger count updates from an authenticated driver session.

FR-56: The system shall associate a received passenger count with the vehicle's currently active trip.

FR-57: The system shall make updated passenger count information available to connected clients in real time.

Notifications

FR-58: The system shall allow administrators to send a notification to a specific user.

FR-59: The system shall allow administrators to broadcast a notification to all passengers.

FR-60: The system shall deliver notifications to passengers and drivers through push notifications and a real-time in-app channel.

FR-61: The system shall track the read/unread status of notifications.

Complaints

FR-62: The system shall allow passengers to submit a complaint, optionally including a supporting image.

- Reporting & Dashboard

REQ-6.1:

The system shall provide administrators with a dashboard displaying overall system statistics.

REQ-6.2:

The system shall display statistics about active vehicles in the system.

REQ-6.3:

The system shall display statistics about route usage and transportation demand.

REQ-6.4:

The system shall display passenger activity statistics.

REQ-6.5:

The system shall allow administrators to monitor system performance through visual reports and charts.

5.7 Non-Functional Requirements

Security

NFR-01*: The system shall require authentication before granting access to protected functionality.*

NFR-02: *The system shall store user passwords using secure hashing rather than plain text.*

NFR-03: *The system shall enforce role-based access control, distinguishing between passenger, driver, merchant, and administrator capabilities.*

NFR-04: *The system shall authenticate bus scanner devices independently from user accounts, using a dedicated device credential.*

NFR-05: The system shall restrict administrative account privileges to a single, uniquely designated administrator account.

Reliability and Data Integrity

NFR-06*: The system shall maintain an auditable record of subscription sales and boarding attempts, including rejected attempts, to support traceability of transactions.*

NFR-07*: The system shall preserve referential consistency between related entities, such as vehicles, routes, drivers, and trips.*

NFR-08: *The system shall ensure that concurrent operations on the same transport card, such as simultaneous subscription purchases, do not result in inconsistent card state.*

Real-Time Responsiveness

NFR-09: *The system shall propagate vehicle location updates to tracking clients with minimal delay through a real-time communication channel.*

NFR-10: *The system shall deliver notifications to connected clients through a real-time channel in addition to push notifications.*

Availability

NFR-11: *The system shall remain accessible to passengers, drivers, merchants, and administrators as a continuously running backend service.*

Usability

NFR-12: *The system shall provide role-specific interfaces tailored to the needs of passengers, drivers, merchants, and administrators.*

NFR-13: *The system shall present transport card, subscription, and transaction information in a clear and consistent format across passenger, merchant, and administrator views.*

Maintainability

NFR-14: *The system shall separate business logic from request-handling logic to support maintainability and future extension.*

NFR-15*: The system shall organize backend functionality by domain (accounts, routes, tracking, payments, notifications) to support ease of maintenance.*

Compatibility

NFR-16: *The system shall expose its backend functionality through a REST API and WebSocket endpoints consumable by both the mobile application and the administrative dashboard.*

Scalability

NFR-17: *The system shall support a growing number of zones, routes, vehicles, and transport cards without requiring changes to the underlying data model.*

Req-ID	Req Title	Category	Actor	Priority
REQ-1.1	The system shall allow passengers and drivers to create new accounts by entering the required personal information.	Account Management	Passenger / Driver	High
REQ-1.2	The system shall validate user input data during the registration process to ensure correctness and completeness.	Account Management	Passenger / Driver	High
REQ-1.3	The system shall allow users to log in using valid email and password credentials.	Account Management	Passenger / Driver	High
REQ-1.4	The system shall allow users to log out from the application.	Account Management	Passenger / Driver	Medium
REQ-1.5	The system shall allow passengers and drivers to view their profile information.	Account Management	Passenger / Driver	High
REQ-1.6	The system shall allow passengers and drivers to update their editable profile information.	Account Management	Passenger / Driver	Medium
REQ-1.7	The system shall allow passengers and drivers to request account deactivation.	Account Management	Passenger / Driver	Medium
REQ-1.8	The system shall require administrator approval before a newly registered driver account becomes active.	Account Management	Driver / Admin	High
REQ-1.9	The system shall provide administrators with authentication mechanisms for accessing administrative functions.	Account Management	Admin	High
REQ-1.10	The system shall provide merchants with authentication mechanisms for accessing merchant functions.	Account Management	Merchant	High

2. Trip Planning and Route Search

Req-ID	Req Title	Category	Actor	Priority
REQ-2.1	The system shall allow the driver to sign up or sign in after the account has been verified by the Admin.	Trip Planning	Passenger	High
REQ-2.2	The system shall display available routes based on the selected search criteria.	Trip Planning	Passenger	Medium
REQ-2.3	The system shall allow passengers to view detailed information about a selected route.	Route Information	Passenger	High
REQ-2.4	The system shall display the stops associated with each route in their defined order.	Route Information	Passenger	High
REQ-2.5	The system shall display the route path on a map.	Route Information	Passenger	High
REQ-2.6	The system shall display the pricing information associated with the route.	Route Information	Passenger	High
REQ-2.7	The system shall allow passengers to add routes to their favorites.	Personalization	Passenger	Medium
REQ-2.8	The system shall allow passengers to view their favorite routes.	Personalization	Passenger	Medium
REQ-2.9	The system shall allow passengers to remove routes from their favorites.	Personalization	Passenger	Medium

3. Real-Time Tracking:

Req-ID	Req Title	Category	Actor	Priority
REQ-3.1	The system shall allow passengers to view the live location of buses operating on an active trip.	Real-Time Tracking	Passenger	High
REQ-3.2	The system shall update the bus location in real time during an active trip.	Real-Time Tracking	Driver	High
REQ-3.3	The system shall allow administrators to monitor the live location of vehicles.	Real-Time Tracking	Admin	High
REQ-3.4	The system shall provide real-time trip tracking information to passengers while the trip is active.	The system shall provide real-time trip tracking information to passengers while the trip is active.	Passenger	High
REQ-3.5	The system shall detect route deviations based on the driver's reported location.	Real-Time Tracking	System	High
REQ-3.6	The system shall notify the driver when the vehicle exceeds the configured route-deviation distance threshold.	Real-Time Tracking	Driver	Medium

4. Driver Management and Trips

Req-ID	Req Title	Category	Actor	Priority

DR-1.1	The system shall allow drivers to register by submitting the required identification and license documents.	Driver Management	Driver	High
DR-1.2	The system shall allow administrators to review driver registration applications.	Driver Management	Admin	High
DR-1.3	The system shall allow administrators to approve or reject driver registration applications.	Driver Management	Admin	High
DR-1.4	The system shall allow drivers to view their assigned trip information.	Trip Management	Driver	High
DR-1.5	The system shall allow drivers to start an assigned trip.	Trip Management	Driver	High
DR-1.6	The system shall allow drivers to stop an active trip.	Trip Management	Driver	High
DR-1.7	The system shall allow drivers to retrieve the current status of their active trip.	Trip Management	Driver	High
DR-1.8	The system shall receive the driver's live GPS location during an active trip.	Real-Time Tracking	Driver	High
DR-1.9	The system shall detect route deviations based on the driver's reported GPS location.	Real-Time Tracking	System	High
DR-1.10	The system shall notify drivers when a route deviation is detected.	Notifications	Driver	High

5. Admin Management

Req-ID	Req Title	Category	Actor	Priority
AD-1.1	The system shall allow the administrator to manage passenger accounts.	User Management	Admin	High
AD-1.2	The system shall allow the administrator to manage driver accounts.	User Management	Admin	High
AD-1.3	The system shall allow the administrator to manage merchant accounts.	User Management	Admin	High
AD-1.4	The system shall allow the administrator to activate or deactivate user accounts.	User Management	Admin	High
AD-1.5	The system shall allow the administrator to update user information.	User Management	Admin	Medium
AD-1.6	The system shall allow the administrator to review driver approval requests.	User Management	Admin	High

6. Vehicle and Route Management:

Req-ID	Req Title	Category	Actor	Priority
AD-1.1	The system shall allow the administrator to manage passenger accounts.	User Management	Admin	High
AD-1.2	The system shall allow the administrator to manage driver accounts.	User Management	Admin	High
AD-1.3	The system shall allow the administrator to manage merchant accounts.	User Management	Admin	High

AD-1.4	The system shall allow the administrator to activate or deactivate user accounts.	User Management	Admin	High
AD-1.5	The system shall allow the administrator to update user information.	User Management	Admin	Medium
AD-1.6	The system shall allow the administrator to review driver approval requests.	User Management	Admin	High

7. Zone Management

Req-ID	Req Title	Category	Actor	Priority
AD-3.1	The system shall allow the administrator to create transportation zones.	Zone Management	Admin	High
AD-3.2	The system shall allow the administrator to define the hierarchy level of each transportation zone.	Zone Management	Admin	High
AD-3.3	The system shall allow the administrator to define daily subscription prices for each zone.	Zone Management	Admin	High
AD-3.4	The system shall allow the administrator to define weekly subscription prices for each zone.	Zone Management	Admin	High
AD-3.5	The system shall allow the administrator to define monthly subscription prices for each zone.	Zone Management	Admin	High
AD-3.6	The system shall allow the administrator to assign a transportation zone to a route.	Zone Management	Admin	High

8. Transport Card :

Req-ID	Req Title	Category	Actor	Priority
CARD-1.1	The system shall allow the administrator to issue a unique transport card for a passenger.	Transport Card	Admin	High
CARD-1.2	The system shall assign a unique card number and QR-encoded token to each transport card.	Transport Card	System	High
CARD-1.3	The system shall allow passengers to view their transport card information and balance.	Transport Card	Passenger	High
CARD-1.4	The system shall allow administrators to activate or block transport cards.	Transport Card	Admin	High
CARD-1.5	The system shall validate the transport card status before processing a boarding transaction.	Transport Card	System	High
CARD-1.6	The system shall record every boarding attempt as a payment transaction with its corresponding outcome status.	Transport Card	System	High

9. Subscription Management

Req-ID	Req Title	Category	Actor	Priority
SUB-1.1	The system shall allow merchants to view the transportation zones available for subscription.	Subscription Management	Merchant	High
SUB-1.2	The system shall allow merchants to look up a passenger's transport card using its QR token or card information.	Subscription Management	Merchant	High
SUB-1.3	The system shall allow merchants to view the active zone subscriptions associated with a passenger's transport card.	Subscription Management	Merchant	High
SUB-1.4	The system shall allow merchants to sell a zone subscription for a selected duration.	Subscription Management	Merchant	High
SUB-1.5	The system shall support daily, weekly, and monthly zone subscriptions.	Subscription Management	Merchant	High
SUB-1.6	The system shall allow a passenger to hold concurrent subscriptions for multiple transportation zones.	Subscription Management	Passenger	High

10. Payment and Boarding

Req-ID	Req Title	Category	Actor	Priority

PAY-1.1	The system shall allow a scanner device to scan and validate a passenger's transport card during boarding.	Payment	Scanner Device	High
PAY-1.2	The system shall authenticate the scanner device before processing a boarding transaction.	Payment	Scanner Device	High
PAY-1.3	The system shall verify the transport card status before approving boarding.	Payment	System	High
PAY-1.4	The system shall verify whether the passenger has valid subscription coverage for the route zone.	Payment	System	High
PAY-1.5	The system shall approve boarding when the passenger's card and subscription satisfy the required conditions.	Payment	System	High
PAY-1.6	The system shall reject boarding when the transport card or subscription does not satisfy the required conditions.	Payment	System	High
PAY-1.7	The system shall record successful and rejected boarding attempts.	Payment	System	High
PAY-1.8	The system shall allow administrators to view boarding payment transactions.	Payment	Admin	High
PAY-1.9	The system shall allow passengers to view their boarding transaction history.	Payment	Passenger	Medium

5.5.3.1 Basic UML Diagrams

Figure 3: High Level usecase

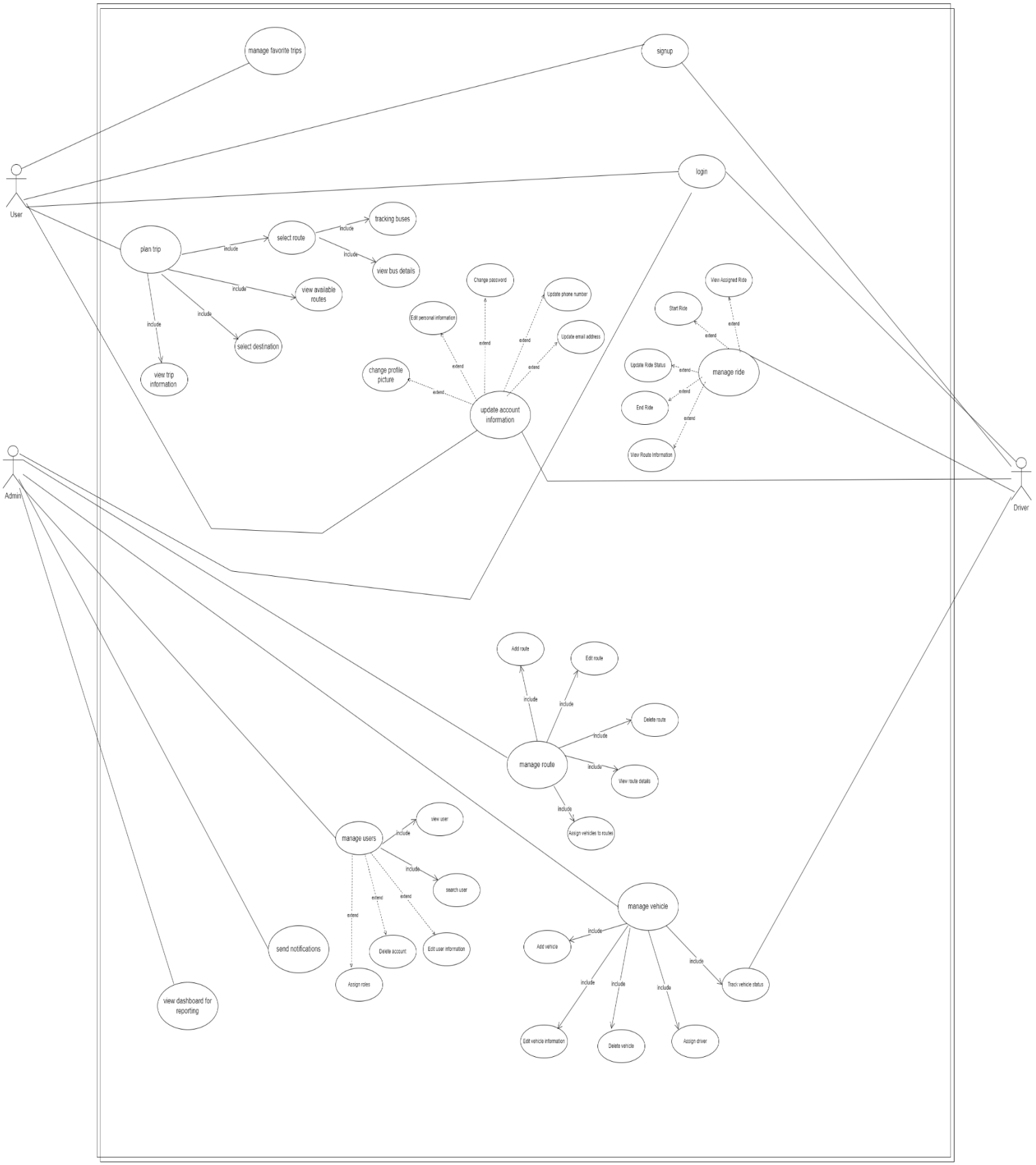
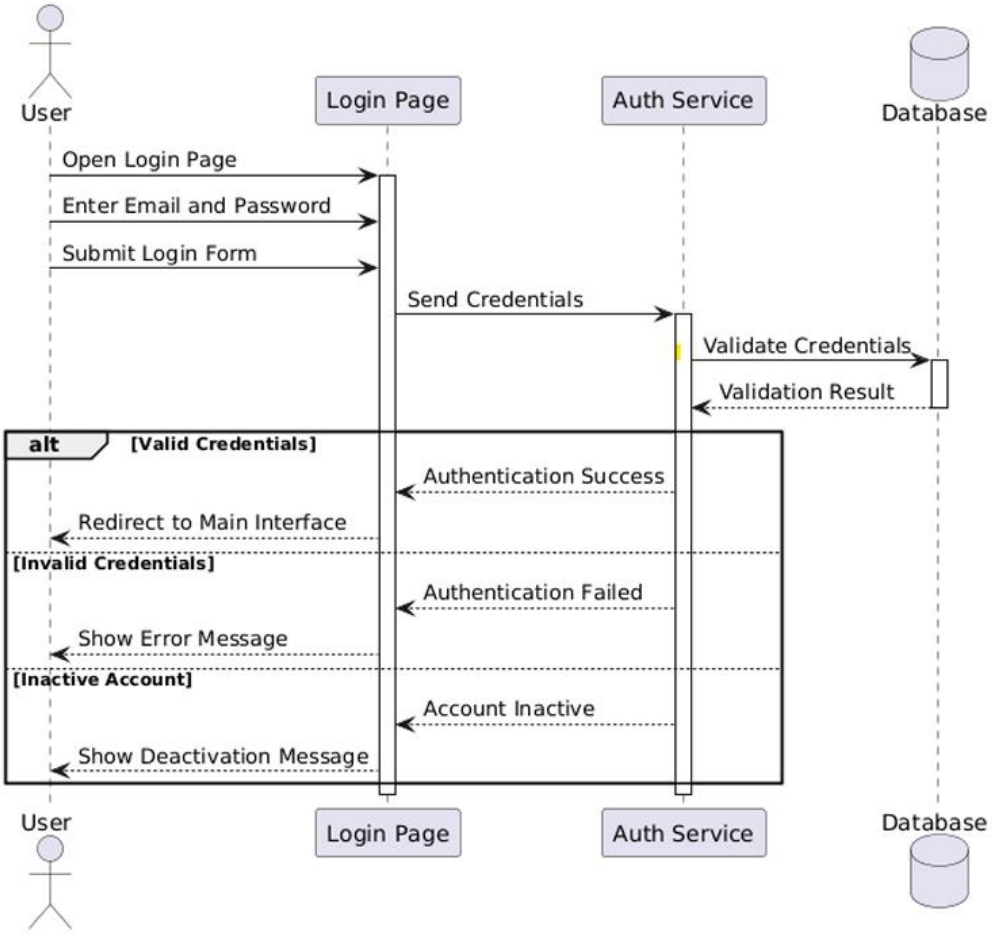


figure 4 : Low Level usecase

5.5.3.2 System features (use case specifications- Sequence Diagrams):

- Sign in

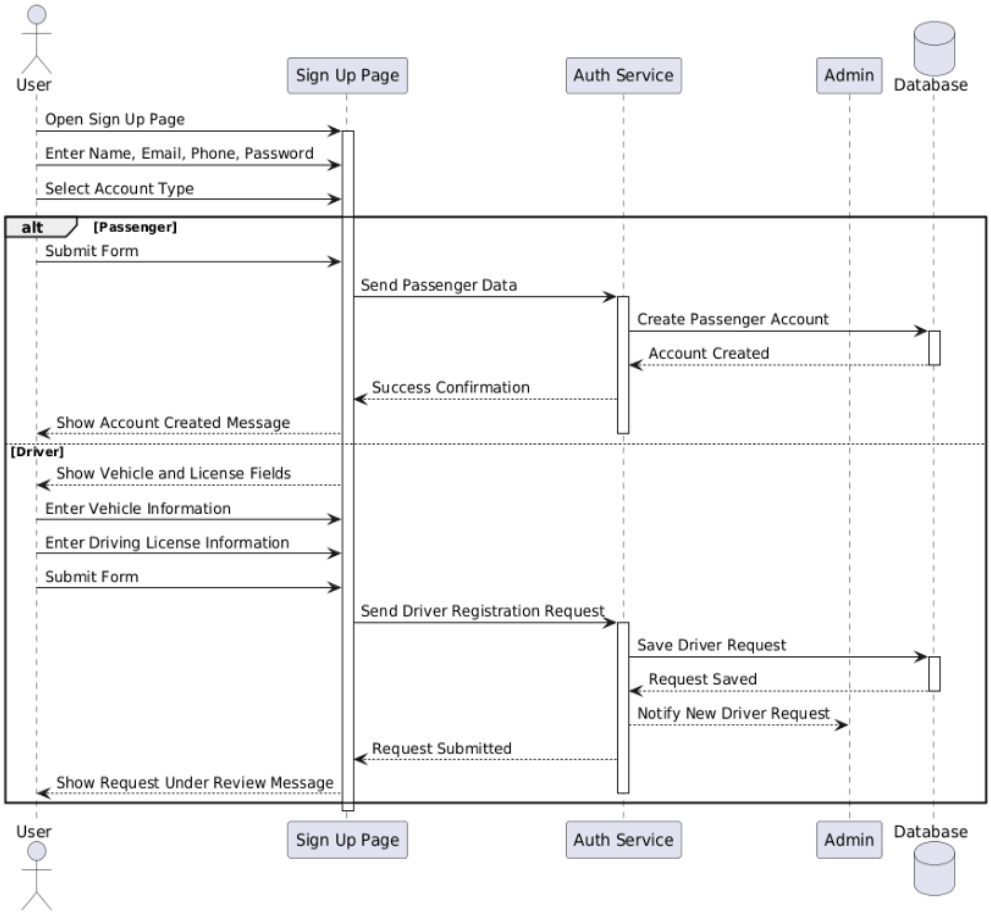
Use case title	**Sign in**
Participating users	User (passenger/Driver/Admin)
The flow of events	The user opens the system main interface. The user clicks on the "Sign in" button. The system displays the sign in form. The user enters email and password. The user submits the login form. The system validates the entered credentials. The system grants access and redirects the user to the main interface.
Alternative flow	Exception 1: In step "6": if the email or password is incorrect, the system displays an error message and asks the user to try again. Exception 2: In step "6": if the account is inactive, the system denies access and displays a message indicating that the account is deactivated.
Entry condition	The user is on the system main interface and has an existing account.
Exit condition	The user is successfully authenticated and logged in.


User
Login Page
Auth Service
Database
Open Login Page
Enter Email and Password
Submit Login Form
Send Credentials
Validate Credentials
Validation Result
alt
[Valid Credentials]
Authentication Success
Redirect to Main Interface
[Invalid Credentials]
Authentication Failed
Show Error Message
[Inactive Account]
Account Inactive
Show Deactivation Message
User
Login Page
Auth Service
Database

- Sign Up

Use case title	**Sign in**
Participating users	User (passenger/Driver/Admin)
The flow of events	The user accesses the system main interface. The user clicks on the "Sign Up" button. The system opens the sign-up form. The user enters the required basic information: name, email, phone number, and password. The user selects the account type using one of the checkboxes: Passenger or Driver. If the user selects Driver, the system displays additional fields for vehicle information and driving license information. If the user selects Driver, the user enters the required driver-specific information. The user submits the sign-up form. If the selected account type is Passenger, the system creates the account directly and stores the user information in the database. If the selected account type is Driver, the system stores the request and sends it to the admin for review. If the selected account type is Driver, the system displays a message stating that the request will be processed.
Alternative flow	In step "4", if any required basic field is missing, the system displays a

	validation message and asks the user to complete the missing fields. Exception 2: In step "4", if the entered email already exists, the system displays an error message and asks the user to use another email. Exception 3: In step "6", if the user selects Driver and does not complete the additional required driver fields, the system displays a validation message and prevents form submission. Exception 4: In step "8", if the submitted data is invalid, the system rejects the request and asks the user to correct the entered information.
Entry condition	The user is on the main interface of the system.
Exit condition	A passenger account is created directly, or a driver registration request is sent to the admin for processing

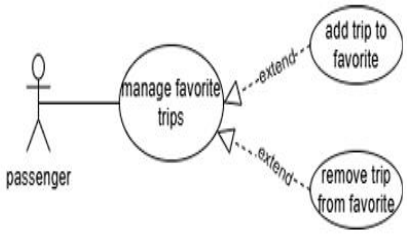

User
Sign Up Page
Auth Service
Admin
Database
Open Sign Up Page
Enter Name, Email, Phone, Password
Select Account Type
alt
[Passenger]
Submit Form
Send Passenger Data
Create Passenger Account
Account Created
Success Confirmation
Show Account Created Message
[Driver]
Show Vehicle and License Fields
Enter Vehicle Information
Enter Driving License Information
Submit Form
Send Driver Registration Request
Save Driver Request
Request Saved
Notify New Driver Request
Request Submitted
Show Request Under Review Message
User
Sign Up Page
Auth Service
Admin
Database

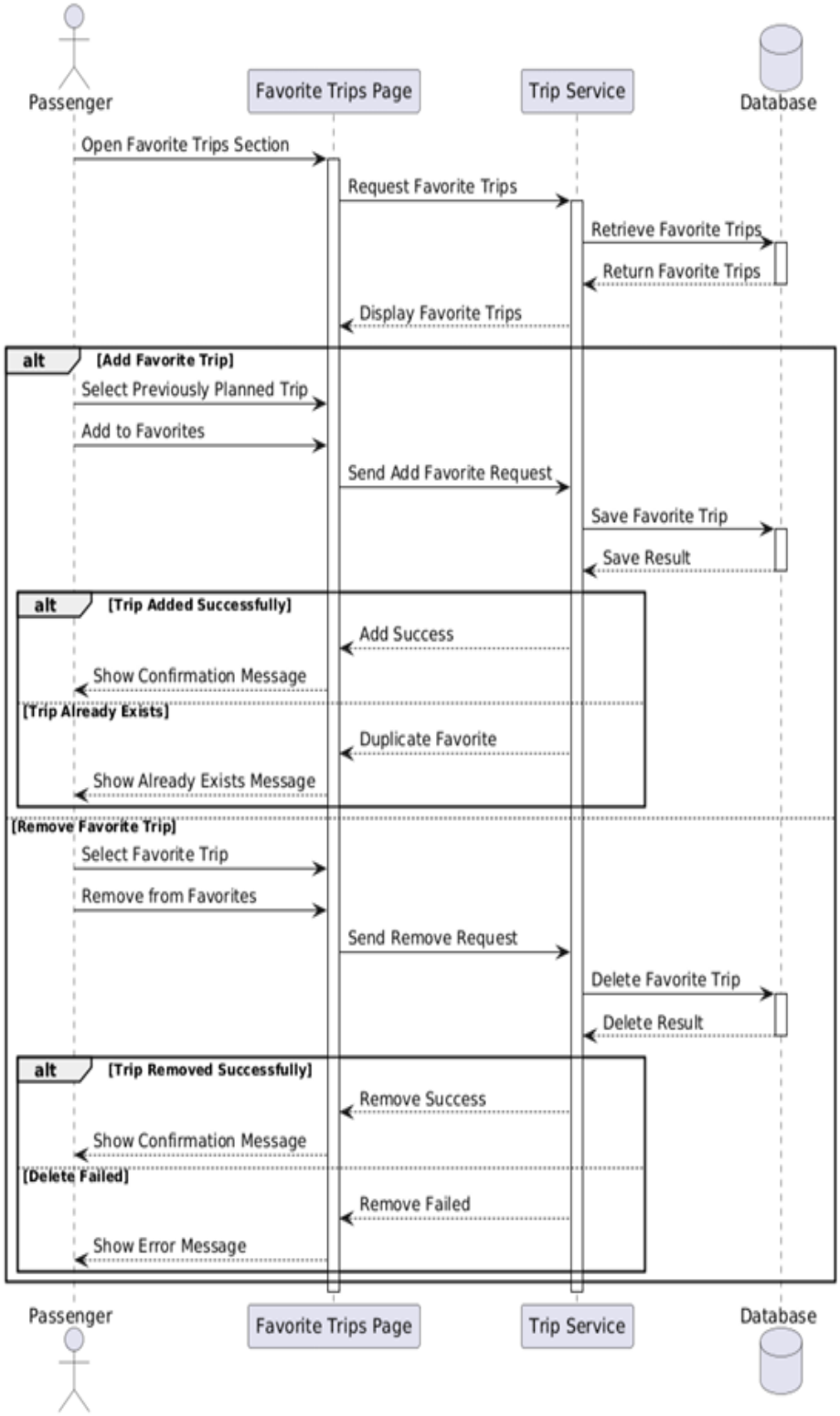
- Manage Favorite Trips

Use case title	**Manage Favorite Trips**
Participating users	Passenger
The flow of events	The passenger logs in to the system. The passenger accesses the main application interface. The passenger opens the trips section. The system displays the current list of favorite trips. If the passenger wants to add a favorite trip, the passenger selects a previously planned trip. The passenger chooses to add the selected trip to favorites. The system stores the selected trip in the favorite trips list. If the passenger wants to remove a favorite trip, the passenger selects a trip from the favorite trips list. The passenger chooses to remove the selected trip from favorites. The system deletes the selected trip from the favorite trips list. The system displays a confirmation message.
Alternative flow	Exception 1:

	In step "5": if the passenger has not previously planned or selected any trip, the system does not allow adding a favorite trip and displays a message indicating that no trip is available to add. Exception 2: In step "6": if the selected trip already exists in the favorite trips list, the system displays a message indicating that the trip is already saved as a favorite and prevents duplication. Exception 3: In step "9": if no trip is selected from the favorite trips list, the system displays a message asking the passenger to select a trip before removing it. Exception 4: In step "10", if the delete operation fails, the system displays an error message and keeps the trip in the favorite trips list.
Entry condition	The passenger is logged in and has previously planned at least one trip

Exit condition	The favorite trips list is updated successfully by adding or removing the selected trip.

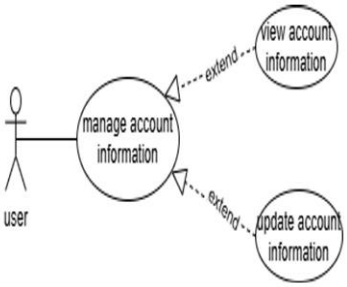

passenger
manage favorite trips
add trip to favorite
remove trip from favorite
extend
extend

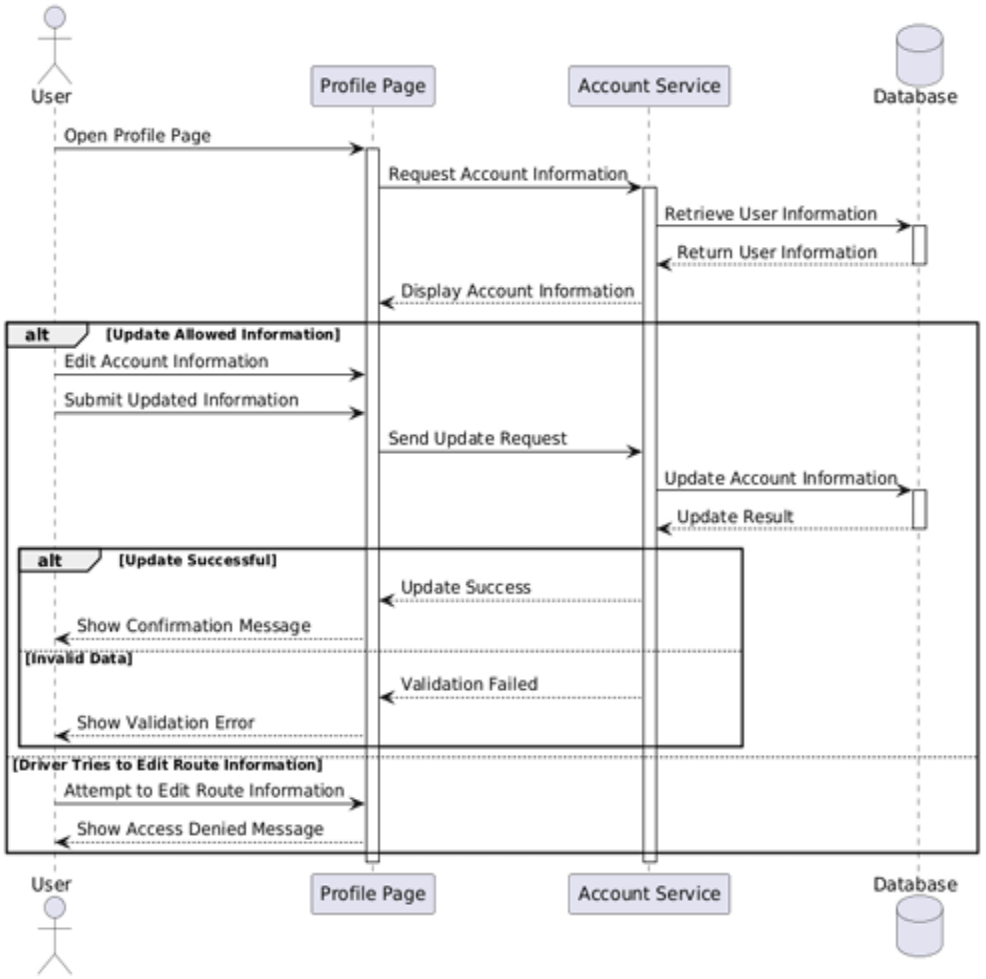

Passenger
Favorite Trips Page
Trip Service
Database
Open Favorite Trips Section
Request Favorite Trips
Retrieve Favorite Trips
Return Favorite Trips
Display Favorite Trips
alt
[Add Favorite Trip]
Select Previously Planned Trip
Add to Favorites
Send Add Favorite Request
Save Favorite Trip
Save Result
alt
[Trip Added Successfully]
Add Success
Show Confirmation Message
[Trip Already Exists]
Duplicate Favorite
Show Already Exists Message
[Remove Favorite Trip]
Select Favorite Trip
Remove from Favorites
Send Remove Request
Delete Favorite Trip
Delete Result
alt
[Trip Removed Successfully]
Remove Success
Show Confirmation Message
[Delete Failed]
Remove Failed
Show Error Message
Passenger
Favorite Trips Page
Trip Service
Database

- Manage Account Information

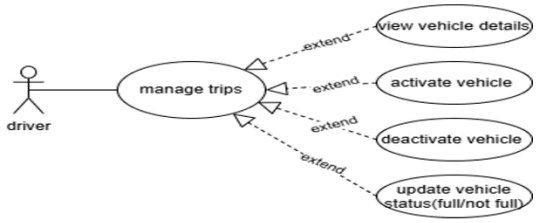
Use case title	**Manage Favorite Trips**
Participating users	Passenger, Driver
The flow of events	1. The passenger or driver logs in to the system. The user accesses the main application interface. The user opens the profile page. The system displays the current account information. The user reviews the displayed information. If the user wants to update the information, the user edits the allowed fields. The user submits the updated information. The system validates the entered data. The system updates the account information in the database. The system displays a confirmation message.
Alternative flow	Exception 1: In step "8", if any required field is empty, the system displays a validation message and asks the user to complete the missing fields.

	Exception 2: In step "8", if the entered email or phone number is already used by another account, the system displays an error message and rejects the update. Exception 3: In step "6", if the driver attempts to edit route information, the system prevents the action and displays a message that route information cannot be modified by the driver. Exception 4: In step "9", if the update operation fails, the system displays an error message and keeps the previous account information unchanged.
Entry condition	The passenger or driver has already logged in and accessed the main application interface.
Exit condition	The user account information is displayed and, if modified successfully, the updated information is saved to the system.


user
manage account information
extend
view account information
extend
update account information

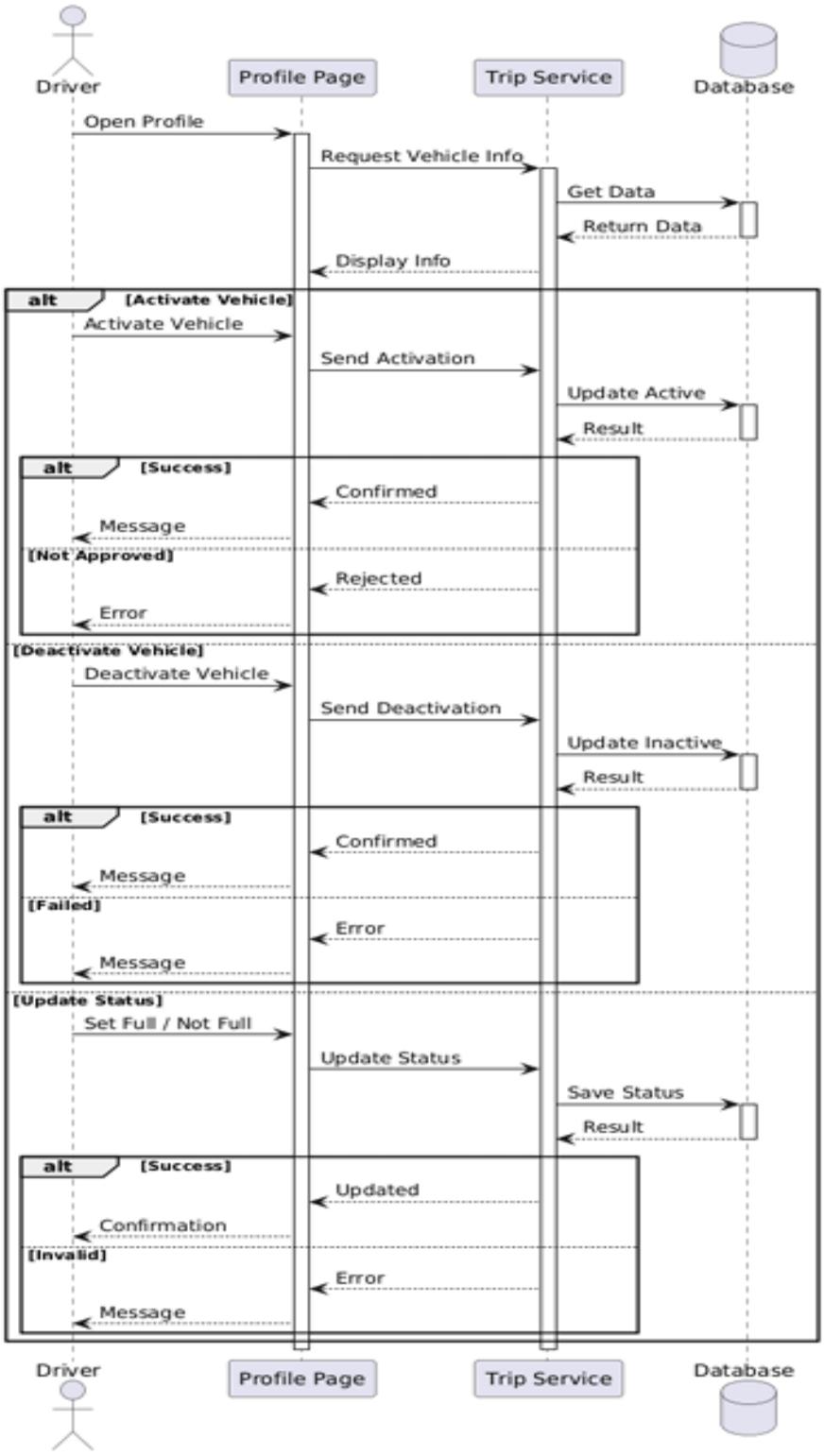

User
Profile Page
Account Service
Database
Open Profile Page
Request Account Information
Retrieve User Information
Return User Information
Display Account Information
alt
[Update Allowed Information]
Edit Account Information
Submit Updated Information
Send Update Request
Update Account Information
Update Result
alt
[Update Successful]
Update Success
Show Confirmation Message
[Invalid Data]
Validation Failed
Show Validation Error
[Driver Tries to Edit Route Information]
Attempt to Edit Route Information
Show Access Denied Message
User
Profile Page
Account Service
Database

- Manage Trip



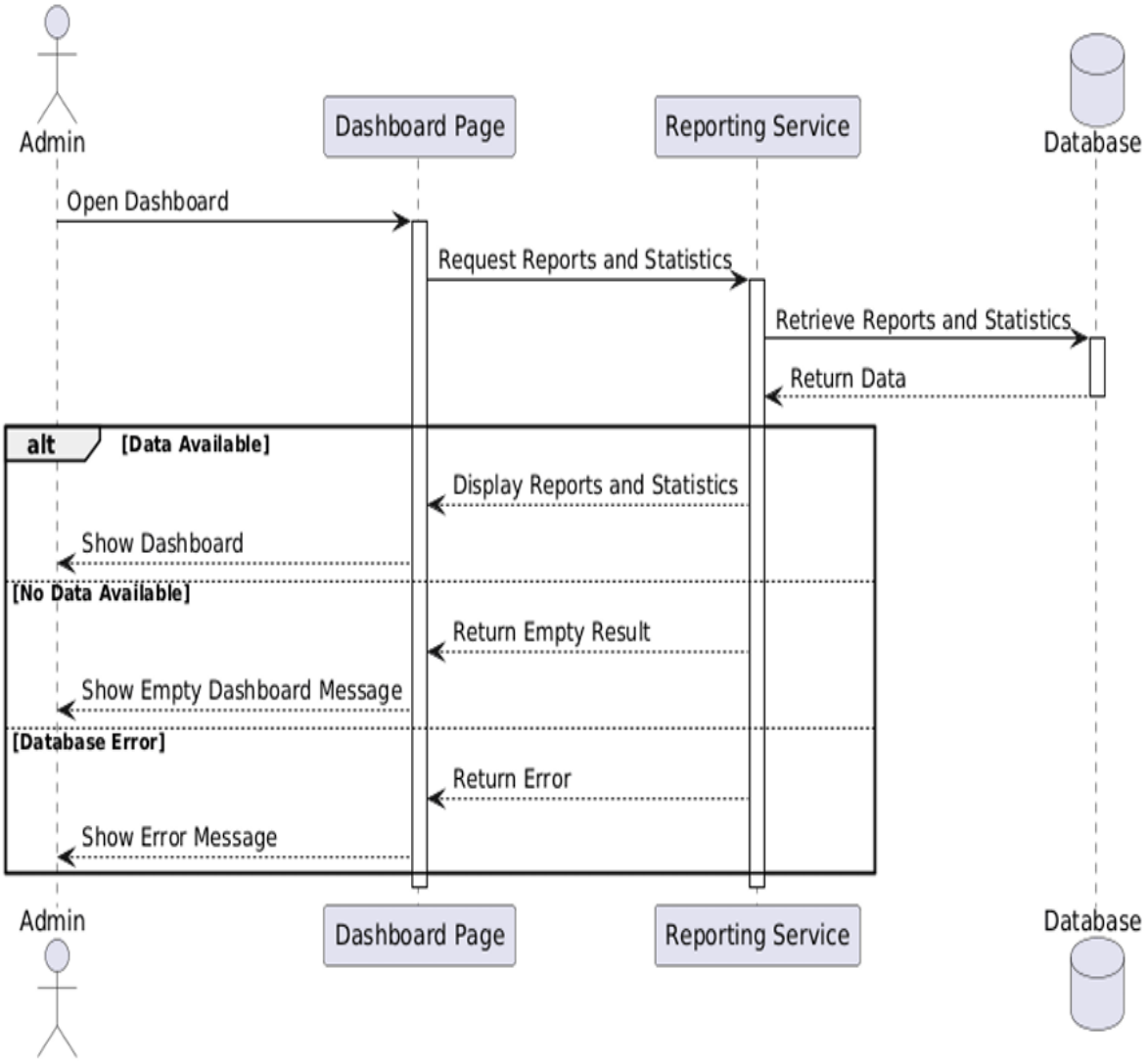
Use case title	**Manage Trip**
Participating users	Driver
The flow of events	The driver logs in to the system. The driver accesses the main interface. The driver opens the profile page. The system displays vehicle information. The driver chooses to activate the vehicle. The system activates the vehicle and enables tracking. The driver updates vehicle status (full / not full). The driver may deactivate the vehicle when stopping work. The system updates vehicle status in the database.
Alternative flow	Exception 1: In step "6", if activation fails, the system displays an error message. Exception 2: In step "8", if deactivation fails, the system displays an error message.

Entry condition	Driver account is approved and logged in.
Exit condition	Vehicle status is updated (activated, deactivated, or occupancy updated).


Driver
Profile Page
Trip Service
Database
Open Profile
Request Vehicle Info
Get Data
Return Data
Display Info
alt [Activate Vehicle]
Activate Vehicle
Send Activation
Update Active
Result
alt [Success]
Confirmed
Message
[Not Approved]
Rejected
Error
[Deactivate Vehicle]
Deactivate Vehicle
Send Deactivation
Update Inactive
Result
alt [Success]
Confirmed
Message
[Failed]
Error
Message
[Update Status]
Set Full / Not Full
Update Status
Save Status
Result
alt [Success]
Updated
Confirmation
[Invalid]
Error
Message
Driver
Profile Page
Trip Service
Database

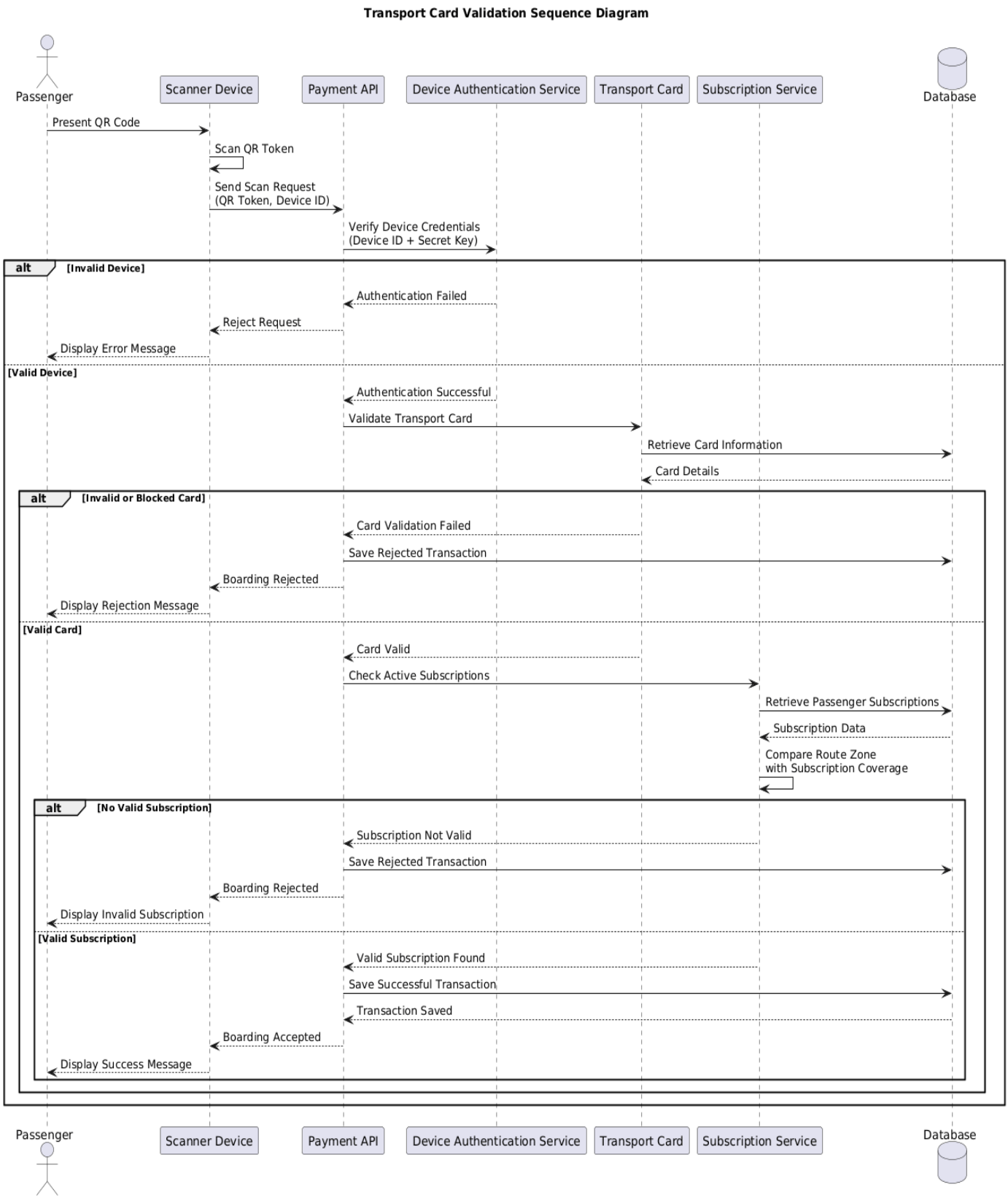
- View Dashboard for Reporting

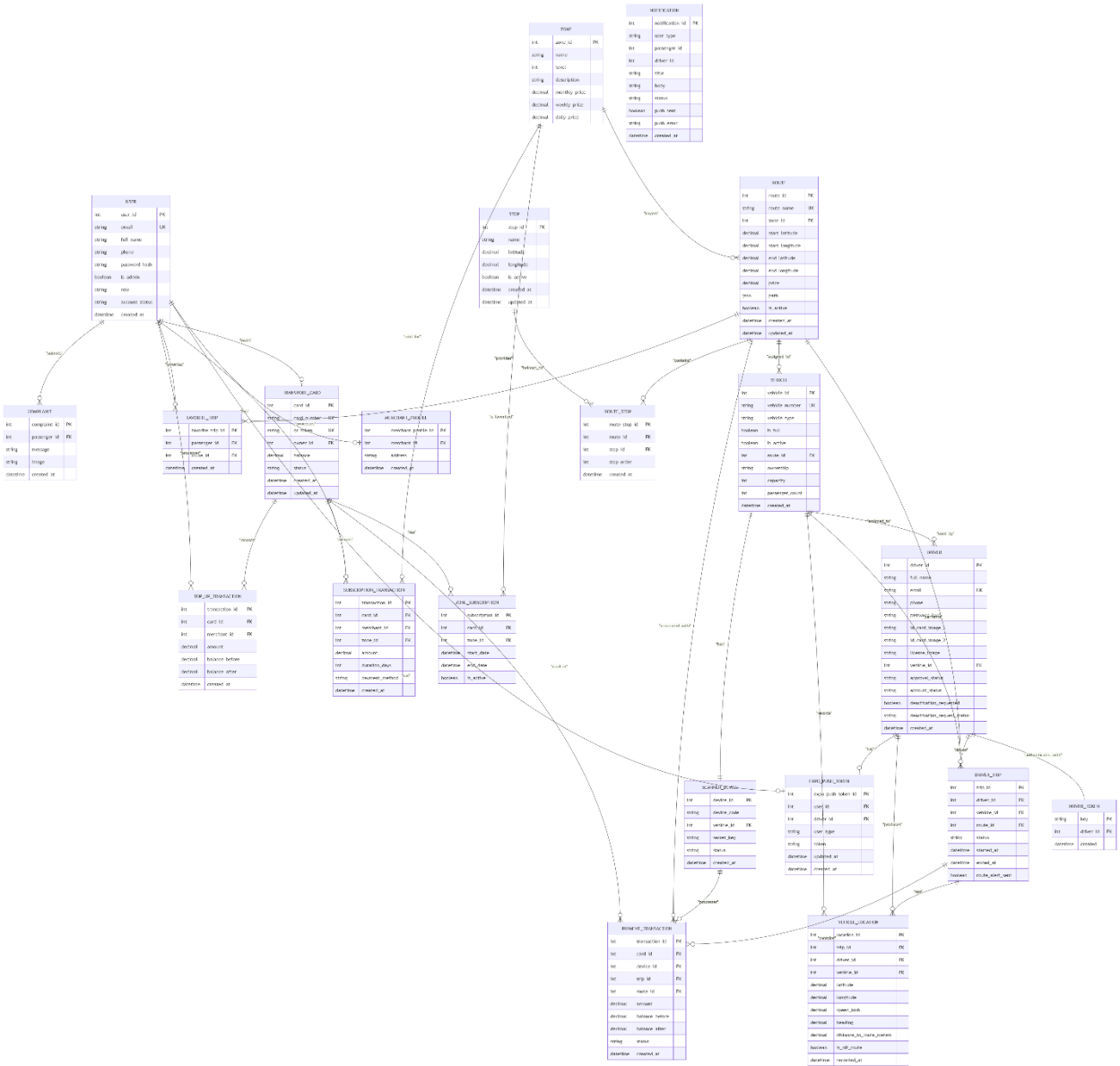
Use case title	**View Dashboard for Reporting**
Participating users	Admin
The flow of events	The admin logs in to the system. The admin accesses the main interface. The system retrieves reports and statistics. The system displays reports and statistics on the dashboard
Alternative flow	Exception 1: In step "3", if no data is available, the system displays an empty dashboard message. Exception 2: In step "3", if a database error occurs, the system displays an error message.
Entry condition	Admin is logged in.
Exit condition	Dashboard with reports and statistics is displayed.


Admin
Dashboard Page
Reporting Service
Database
Open Dashboard
Request Reports and Statistics
Retrieve Reports and Statistics
Return Data
alt
[Data Available]
Display Reports and Statistics
Show Dashboard
[No Data Available]
Return Empty Result
Show Empty Dashboard Message
[Database Error]
Return Error
Show Error Message
Admin
Dashboard Page
Reporting Service
Database

Use case title	Validate Transport Card During Boarding
Participating users	Passenger, Scanner Device
The flow of events	. The passenger presents the transport card QR code to the scanner device. 2. The scanner device scans the QR token and sends a boarding validation request to the system. 3. The system verifies the scanner device credentials using the device ID and secret key. 4. The system validates the transport card information and retrieves the card status. 5. The system checks the passenger's active zone subscriptions. 6. The system compares the route zone with the available subscriptions. 7. If a valid subscription exists, the system records a successful boarding transaction. 8. The system sends a boarding approval response to the scanner device. 9. The scanner device displays a successful boarding message to the passenger.
Alternative flow	Exception 1: In step "3", if the scanner device authentication fails, the system rejects the request and displays an invalid device message. Exception 2: In step "4", if the transport card is invalid or blocked, the system rejects the boarding request and records a failed transaction. Exception 3:

	In step "6", if no valid subscription covers the route zone, the system rejects the boarding request and displays an invalid subscription message. Exception 4: In step "7", if a database error occurs while recording the transaction, the system displays a transaction processing error message.
Entry condition	The passenger has a valid transport card, and the scanner device is registered and authenticated.
Exit condition	The boarding request is either accepted and recorded successfully, or rejected with a corresponding error message.


Transport Card Validation Sequence Diagram
Passenger
Scanner Device
Payment API
Device Authentication Service
Transport Card
Subscription Service
Database
Present QR Code
Scan QR Token
Send Scan Request
(QR Token, Device ID)
Verify Device Credentials
(Device ID + Secret Key)
alt [Invalid Device]
Authentication Failed
Reject Request
Display Error Message
[Valid Device]
Authentication Successful
Validate Transport Card
Retrieve Card Information
Card Details
alt [Invalid or Blocked Card]
Card Validation Failed
Save Rejected Transaction
Boarding Rejected
Display Rejection Message
[Valid Card]
Card Valid
Check Active Subscriptions
Retrieve Passenger Subscriptions
Subscription Data
Compare Route Zone
with Subscription Coverage
alt [No Valid Subscription]
Subscription Not Valid
Save Rejected Transaction
Boarding Rejected
Display Invalid Subscription
[Valid Subscription]
Valid Subscription Found
Save Successful Transaction
Transaction Saved
Boarding Accepted
Display Success Message



5.5.3.4 ERD Diagram

5.5.2 Modeling according to the methodology used

5.5.2.1 User Stories

- User (Passenger)

ID	Actor	User Story
US-1	Passenger	As a passenger, I want to create an account so that I can access the public transportation platform.
US-2	Passenger	As a passenger, I want to log into the system so that I can use transportation services securely.
US-3	Passenger	As a passenger, I want to view available routes so that I can choose a suitable transportation line.
US-4	Passenger	As a passenger, I want to view route details including stops, paths, and prices so that I can plan my trip effectively.
US-5	Passenger	As a passenger, I want to track buses in real time so that I can know the current bus location and estimated arrival time.
US-6	Passenger	As a passenger, I want to view active trips so that I can follow my selected transportation route.
US-7	Passenger	As a passenger, I want to add routes to favorites so that I can access frequently used routes quickly.
US-8	Passenger	As a passenger, I want to remove routes from favorites so that I can manage my saved routes.
US-9	Passenger	As a passenger, I want to submit complaints so that transportation issues can be reported to the administrator.
US-10	Passenger	As a passenger, I want to attach images to complaints so that I can provide additional information about reported issues.
US-11	Passenger	As a passenger, I want to view my transport card information so that I can check my card status and details.
US-12	Passenger	As a passenger, I want to view my active zone subscriptions so that I know which transportation zones I can access.
US-13	Passenger	As a passenger, I want to view my payment and boarding history so that I can track my previous transactions.
US-14	Passenger	As a passenger, I want to receive notifications so that I can be informed about important transportation updates.
US-15	Passenger	As a passenger, I want to update my profile information so that my account data remains accurate.
US-16	Driver	As a driver, I want to create an account so that I can apply to use the transportation platform.
US-17	Driver	As a driver, I want my account to be approved by the administrator so that I can access driver functionalities.
US-18	Driver	As a driver, I want to log into the system so that I can manage my assigned trips securely.

US-19	Driver	As a driver, I want to view my assigned vehicle and route so that I can manage my transportation tasks correctly.
US-20	Driver	As a driver, I want to start a trip so that passengers and administrators can track the active trip.
US-21	Driver	As a driver, I want to stop a trip so that the system updates the trip status correctly.
US-22	Driver	As a driver, I want to send my live location so that the system can provide real-time bus tracking.
US-23	Driver	As a driver, I want to receive route deviation notifications so that I can correct my path when needed.
US-24	Driver	As a driver, I want to view and update my profile information so that my account remains updated.
US-25	Admin	As an administrator, I want to log into the system so that I can manage platform operations securely.
US-26	Admin	As an administrator, I want to manage passenger accounts so that user information remains organized.
US-27	Admin	As an administrator, I want to manage driver accounts so that only approved drivers can operate.
US-28	Admin	As an administrator, I want to approve or reject driver applications so that driver access is controlled.
US-29	Admin	As an administrator, I want to manage vehicles so that buses can be registered and maintained.
US-30	Admin	As an administrator, I want to manage routes and stops so that transportation information remains accurate.
US-31	Admin	As an administrator, I want to assign vehicles to routes so that transportation operations can be organized.
US-32	Admin	As an administrator, I want to manage transportation zones so that subscription coverage can be configured.
US-33	Admin	As an administrator, I want to define subscription prices so that passengers can use available plans.
US-34	Admin	As an administrator, I want to issue and manage transport cards so that passengers can access payment services.
US-35	Admin	As an administrator, I want to register scanner devices so that bus payment validation can be performed securely.
US-36	Admin	As an administrator, I want to view payment transactions so that financial activities can be monitored.
US-37	Admin	As an administrator, I want to review passenger complaints so that reported issues can be managed.
US-38	Admin	As an administrator, I want to send notifications so that passengers and drivers receive important updates.
US-39	Admin	As an administrator, I want to view dashboards and reports so that I can monitor system performance and activities.

US-40	Merchant	As a merchant, I want to log into the system so that I can provide transport card services.
US-41	Merchant	As a merchant, I want to search for passenger transport cards so that I can manage subscription services.
US-42	Merchant	As a merchant, I want to view available zones and prices so that I can sell suitable subscriptions.
US-43	Merchant	As a merchant, I want to add or extend passenger subscriptions so that passengers can use transportation services.
US-44	Merchant	As a merchant, I want to view my subscription sales history so that I can track performed transactions.

4. Initial Test Cases

The following preliminary test cases provide an initial overview of the testing activities planned for the Public Transportation Platform. These test cases are designed to verify the main functional requirements of the system and ensure that its core features operate as expected. Detailed test case descriptions, execution steps, and results will be presented in the Testing Chapter.

Table 4Test Cases for Authentication and Account Management

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-AUTH-01	Passenger registers a new account with valid data	No existing account uses the test email address	1. Send POST /api/auth/register with role=passenger and required fields 2. Submit valid full name, email, phone, and password	email: passenger1@test.com, password: Test@1234	Account is created with role 'passenger' and account_status 'active'; a success response is returned	Account created successfully; response confirms passenger role	**Passed**
TC-AUTH-02	Passenger registration fails with a duplicate email	An account already exists with the given email	1. Send POST /api/auth/register using an email already registered 2. Submit otherwise valid data	email: passenger1@test.com (existing)	Request is rejected with a validation error indicating the email is already in use	400 Bad Request returned with an email-uniqueness validation error	**Passed**
TC-AUTH-03	Driver registers with required identification documents	No existing driver account uses the test email	1. Send POST driver registration request with personal data 2. Attach ID card images (front/back) and license image	email: driver1@test.com, id_card_image_1, id_card_image_2, license_image	Driver account is created with approval_status 'pending' and is not yet usable for trips	Driver account created with approval_status = pending, as expected	**Passed**

TC-AUTH-04	Unapproved driver cannot start a trip	Driver account exists with approval_status = pending	1. Log in as the pending driver 2. Attempt to start a trip via POST /api/driver/trip/start	driver approval_status: pending	Request is rejected; trip cannot be started until the driver is approved	Request denied with an authorization/approval error	**Passed**
TC-AUTH-05	User logs in with valid credentials	An active passenger/merchant/admin account exists	1. Send POST /api/auth/login with correct email and password	email: passenger1@test.com, password: Test@1234	A valid authentication token is returned along with the user's role	200 OK with a DRF auth token and role='passenger' returned	**Passed**
TC-AUTH-06	Login fails with incorrect password	An active account exists	1. Send POST /api/auth/login with a valid email and an incorrect password	email: passenger1@test.com, password: WrongPass1	Request is rejected with an invalid-credentials error; no token is issued	400/401 response returned; no token issued	**Passed**
TC-AUTH-07	Login optionally registers an Expo push token	An active account exists; mobile app has a device push token	1. Send POST /api/auth/login including an expo_push_token field	expo_push_token: ExponentPushToken[xxxx]	Login succeeds and the push token is stored/updated for the account	Login succeeds; ExpoPushToken record created or updated for the user	**Passed**
TC-AUTH-08	Authenticated user logs out	User is logged in with a valid token	1. Send POST /api/auth/logout with the Authorization header set	Authorization: Token <valid_token>	The token is invalidated; subsequent requests using it are rejected	Token deleted; later requests with the same token return 401	**Passed**
TC-AUTH-09	Passenger updates own profile information	Passenger is logged in	1. Send PATCH to the passenger profile endpoint with updated full_name/phone	full_name: Updated Name, phone: 0999999999	Profile fields are updated and reflected on subsequent GET requests	Profile updated successfully; GET reflects new values	**Passed**
TC-AUTH-10	Passenger submits an account deactivation request	Passenger is logged in with an active account	1. Send POST to the passenger deactivation-request endpoint	N/A	The deactivation request is recorded for administrator review	Request accepted; account flagged for deactivation review	**Passed**
TC-AUTH-11	Unauthenticated request to a protected endpoint is rejected	No Authorization header is provided	1. Send GET /api/payment/my-card/ without an Authorization header	N/A	Request is rejected with a 401 Unauthorized response	401 Unauthorized returned as expected	**Passed**

\

Table 5 Test Cases for Passenger Features

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-PAX-01	Passenger searches for available trips/routes	At least one active route exists in the system	1. Log in as passenger 2. Send GET to the trip search endpoint with search parameters	destination coordinates or route keyword	A list of matching, active routes is returned	List of matching routes returned with route name, stops, and price	**Passed**
TC-PAX-02	Passenger views a route's detail	A route with assigned stops exists	1. Send GET to the route detail endpoint for a valid route_id	route_id: 3	Route details are returned, including ordered stops and path	Route detail returned including stop sequence and generated path	**Passed**
TC-PAX-03	Passenger tracks an active trip in real time	A trip with status='active' exists on the passenger's route of interest	1. Open a WebSocket connection to ws/passenger/trips/<trip_id>/tracking/ with a valid token	trip_id of an active trip	Connection is accepted and live vehicle location updates are streamed	Connection accepted; location updates received as the vehicle moves	**Passed**
TC-PAX-04	Passenger cannot track a non-active trip	A trip exists with status='completed'	1. Attempt to open the trip-tracking WebSocket for the completed trip_id	trip_id of a completed trip	Connection is rejected or closed since the trip	Connection rejected/ closed as the trip is not	**Passed**

					is not active	currently active	
TC-PAX-05	Passenger adds a route to favorites	Passenger is logged in; a valid route exists	1. Send POST to the favorite-trips endpoint with the route_id	route_id: 3	A Favorite Trip record is created for the passenger	Favorite route saved and returned in the passenger's favorites list	**Passed**
TC-PAX-06	Passenger removes a route from favorites	A favorite route already exists for the passenger	1. Send DELETE to the favorite-trip-detail endpoint for the favorite_trip_id	favorite_trip_id: 5	The favorite entry is removed and no longer appears in the list	Favorite removed successfully	**Passed**
TC-PAX-07	Passenger views own transport card information	Passenger owns a TransportCard	1. Send GET /api/payment/my-card/	N/A	Card number, QR token, balance, and status are returned	Card details returned matching the passenger's issued card	**Passed**
TC-PAX-08	Passenger views own active and past zone subscriptions	Passenger's card has at least one ZoneSubscription record	1. Send GET /api/payment/my-card/subscriptions/	N/A	All subscriptions are returned with zone, dates, is_active, and is_expired	Subscriptions list returned, including expired entries marked is_expired=true	**Passed**

TC-PAX-09	Passenger views own subscription purchase history	At least one SubscriptionTransaction exists for the passenger's card	1. Send GET /api/payment/my-card/subscription-transactions/	N/A	All subscription purchase records for the card are returned	Transaction history returned with zone, amount, and duration	**Passed**
TC-PAX-10	Passenger views own boarding/payment transaction history	At least one PaymentTransaction exists for the passenger's card	1. Send GET /api/payment/my-card/transactions/	N/A	Boarding attempts (successful and rejected) are returned in chronological order	Transaction history returned with status and timestamps for each scan	**Passed**
TC-PAX-11	Passenger submits a complaint with a supporting image	Passenger is logged in	1. Send POST to the complaints endpoint with a message and an image file	message: 'Bus arrived 20 minutes late', image: photo.jpg	The complaint is stored and associated with the passenger	Complaint created successfully with the image attached	**Passed**
TC-PAX-12	Passenger without a transport card receives an appropriate response	Passenger account exists without an issued TransportCard	1. Send GET /api/payment/my-card/	N/A	A 404 response indicating no transport card is found	404 Not Found returned with a descriptive message	**Passed**

Table 6Test Cases for Driver Features

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-ADM-01	Admin logs in and accesses the admin dashboard	The single administrator account exists (is_admin=True)	1. Log in with admin credentials 2. Access an admin-only endpoint	admin email/password	Admin authenticates successfully and admin-only endpoints are accessible	Login successful; admin endpoints accessible	**Passed**
TC-ADM-02	Non-admin user is denied access to an admin-only endpoint	A passenger account is logged in	1. Send a request to an admin-only endpoint (e.g. GET /api/payment/zones/ POST)	N/A	Request is rejected with a 403 Forbidden response	403 Forbidden returned as expected	**Passed**
TC-ADM-03	System enforces a single administrator account	An admin account already exists	1. Attempt to create a second account with is_admin=True	is_admin: true	Creation is rejected due to the single-admin business rule	Validation error raised; second admin account not created	**Passed**
TC-ADM-04	Admin lists all pending driver applications	At least one driver account has approval_status='pending'	1. Send GET to the driver-requests endpoint	N/A	All pending driver applications are listed	Pending drivers listed correctly	**Passed**
TC-AD	Admin approves a	A driver exists with	1. Send POST to approve	driver_id	approval_status changes	Driver approved;	**Passed**

M-05	driver application	approval_status= 'pending'	the driver's application		to 'approved'; driver receives a notification	notification sent to the driver	
TC-ADM-06	Admin rejects a driver application	A driver exists with approval_status= 'pending'	1. Send POST to reject the driver's application	driver_id	approval_status changes to 'rejected'; driver receives a notification	Driver application rejected; notification sent	**Passed**
TC-ADM-07	Admin creates a new route with an assigned zone	At least one Zone and a set of Stops exist	1. Send POST to the routes endpoint with route_name, zone, and stop_ids	route_name: 'R-Downtown', zone_id: 1, stop_ids: [1,2,3]	Route is created, path is generated via OSRM, and stops are linked in order	Route created with generated path and correctly ordered stops	**Passed**
TC-ADM-08	Admin creates a route without assigning a zone	Zone field is optional at the API level	1. Send POST to the routes endpoint omitting the zone field	route_name: 'R-Test', zone: omitted	Route is created with zone=null; boarding validation will later fail for this route until a zone is assigned	Route created with zone=null as expected	**Passed**
TC-ADM-09	Admin registers a new vehicle	Admin is logged in	1. Send POST to the vehicles endpoint with vehicle details	vehicle_number: 'BUS-101', capacity: 40	Vehicle record is created and available for route/driv	Vehicle created successfully	**Passed**

					er assignment		
TC-ADM-10	Admin assigns a vehicle to a route	A vehicle and a route exist	1. Send PATCH to the vehicle detail endpoint setting route_id	vehicle_id, route_id	Vehicle.route is updated to the specified route	Vehicle successfully linked to the route	**Passed**

Table 7 Test Cases for Merchant Features

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-MER-01	Merchant looks up a passenger card by QR token	A valid TransportCard exists	1. Send GET /api/payment/cards/lookup/?token=<qr_token>	qr_token of an existing card	Card details are returned, including current zone subscriptions	Card details and subscriptions array returned correctly	**Passed**
TC-MER-02	Merchant card lookup fails for a non-existent token	N/A	1. Send GET /api/payment/cards/lookup/?token=INVALID-TOKEN	qr_token: invalid	A 404 response is returned indicating the card was not found	404 Not Found returned as expected	**Passed**
TC-MER	Merchant views the list of	At least one Zone is configured	1. Log in as merchant 2. Send GET /api/payment/zones/	N/A	Merchant receives the zone list with daily/weekly	200 OK returned with	**Passed**

-03	zones and prices				/monthly prices	zone list, matching the admin-configured prices	
TC-MER-04	Non-admin merchant is blocked from creating a zone	Merchant is logged in	1. Send POST /api/payment/zones/ with valid zone data	name, level, prices	Request is rejected with a 403 Forbidden response (admin-only)	403 Forbidden returned as expected	**Passed**
TC-MER-05	Merchant sells a new zone subscription to a passenger	Passenger's card exists and is not blocked	1. Send POST /api/payment/zone-subscription/ with qr_token, zone_id, subscription_type	subscription_type: daily	A ZoneSubscription is created/extended and a SubscriptionTransaction is recorded	201 Created; subscription and transaction returned in the response	**Passed**
TC-MER-06	Merchant extends an existing unexpired subscription	Passenger's card already has an active subscription for the same zone	1. Send another zone-subscription purchase for the same card and zone	same zone_id as the existing active subscription	The existing subscription's end_date is extended rather than a duplicate being created	Subscription end_date extended by the purchased duration	**Passed**

TC-MER-07	Merchant cannot sell a subscription to a blocked card	Passenger's card has status='blocked'	1. Attempt to purchase a subscription for the blocked card	qr_token of a blocked card	Request is rejected with an explanatory error	400 Bad Request returned: card is blocked and cannot purchase a subscription	**Passed**
TC-MER-08	Merchant views own subscription sale transaction history	Merchant has completed at least one subscription sale	1. Send GET to the merchant's own transaction history endpoint	N/A	All sales made by this merchant are listed	Transaction history returned correctly, scoped to the logged-in merchant	**Passed**
Test Case ID	**Test Scenario**	**Preconditions**	**Test Steps**	**Test Data**	**Expected Result**	**Actual Result**	**Status**

Table 8 Test Cases for Route, Stop, and Vehicle Management

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status

TC-RTE-01	Admin creates a new stop	Admin is logged in	1. Send POST to the stops endpoint with name/coordinates	name: 'Central Station', latitude, longitude	Stop is created and available for assignment to routes	Stop created successfully	**Passed**
TC-RTE-02	Admin assigns stops to a route in a defined order	Route and stops exist	1. Send POST/PATCH with an ordered list of stop_ids	stop_ids: [4, 1, 7]	RouteStop records are created preserving the given stop_order	Stops linked to the route in the specified order	**Passed**
TC-RTE-03	Admin updates a route's active status	A route exists with is_active=True	1. Send PATCH to the route detail endpoint setting is_active=False	is_active: false	Route is marked inactive and excluded from passenger route search	Route updated to inactive; no longer returned in trip search	**Passed**
TC-RTE-04	Route creation fails with a duplicate route name	A route with the given name already exists	1. Send POST to create a route reusing an existing route_name	route_name: existing name	Request is rejected due to the unique constraint on route_name	400 Bad Request returned with a uniqueness validation error	**Passed**
TC-RTE-05	Admin lists all vehicles	At least one vehicle exists	1. Send GET to the vehicles endpoint	N/A	All registered vehicles are	Vehicle list returned correctly	**Passed**

					returned		
TC-RTE-06	Admin retrieves a specific vehicle's detail	A vehicle exists	1. Send GET to the vehicle detail endpoint with a valid vehicle_id	vehicle_id: 2	Vehicle details are returned, including assigned route and driver if any	Vehicle detail returned as expected	**Passed**
TC-RTE-07	Admin tracks a vehicle's live location	The vehicle has an active trip reporting locations	1. Open a WebSocket connection to ws/admin/vehicles/<vehicle_id>/tracking/ as an admin	vehicle_id of an active vehicle	Connection is accepted and live location updates are streamed to the admin	Location updates received in real time	**Passed**
TC-RTE-08	Non-admin user is denied access to vehicle tracking WebSocket	A passenger token is used	1. Attempt to open ws/admin/vehicles/<vehicle_id>/tracking/ using a passenger token	N/A	Connection is rejected since the endpoint is admin-only	Connection rejected/ closed as expected	**Passed**

Table 9 Test Cases for Real-Time GPS Tracking (WebSocket)

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-WS-01	WebSocket connection is authenticated via a valid token query parameter	A valid DRF token exists	1. Connect to a tracking WebSocket endpoint with ?token=<valid_token>	token: valid DRF token	Connection is accepted	Connection accepted successfully	**Passed**
TC-WS-02	WebSocket connection is rejected with an invalid token	N/A	1. Connect to a tracking WebSocket endpoint with ?token=invalid	token: invalid	Connection is rejected	Connection closed/rejected as expected	**Passed**
TC-WS-03	Vehicle location update is broadcast to all subscribed clients	A passenger and an admin are both connected to tracking channels for the same vehicle/trip	1. Driver submits a location update 2. Observe both connected clients	N/A	Both clients receive the updated location in near real time	Both clients received the broadcast update	**Passed**
TC-WS-04	Notification WebSocket delivers an initial unread count on connect	The user has unread notifications	1. Connect to ws/notifications/ with a valid token	N/A	An initial payload with the current unread notification count is sent	Unread count delivered immediately upon connection	**Passed**
TC-WS-05	Driver authenticates on the notificatio	A valid DriverToken exists	1. Connect to ws/notifications/ with	N/A	Connection resolves the	Connection accepted and correctly	**Passed**

	n WebSocket using a DriverToken		?token=<driver_token>		token as a DriverToken and joins the driver's notification group	scoped to the driver	
TC-WS-06	Off-route status is reflected in the tracking broadcast	A trip's latest location is flagged is_off_route=True	1. Connect to the trip-tracking WebSocket while the vehicle is off-route	N/A	The broadcast payload includes the off-route indicator	Off-route flag present in the broadcast payload	**Passed**

Table 10 Test Cases for Transport Card and Payment System

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-CARD-01	Admin issues a transport card to a passenger	Passenger does not yet own a card	1. Send POST /api/payment/cards/ with the passenger's account reference	owner: passenger user_id	A TransportCard is created with a unique card_number and qr_token, status='inactive'	Card created successfully with a unique QR token	**Passed**
TC-CARD-02	Admin cannot issue a secon	Passenger already owns a	1. Attempt to create another card for the same passenger	owner: same passenger	Request is rejected due to	400 Bad Request returned due to the	**Passed**

	d card to the same passenger	Transport Card			the one-to-one card/owner constraint	uniqueness constraint	
TC-CARD-03	Admin blocks a transport card	A card exists with status='active'	1. Send PATCH to the card status endpoint with status='blocked'	status: blocked	Card status is updated to 'blocked'	Card status updated successfully	**Passed**
TC-CARD-04	Blocked card is rejected at scan time	Card status='blocked'; an active trip and registered scanner device exist	1. Send POST /api/payment/scan/ with the blocked card's qr_token	qr_token of blocked card	Boarding is denied with a card_inactive-type status; the attempt is logged	403 Forbidden returned; PaymentTransaction recorded with status card_inactive	**Passed**
TC-CARD-05	Admin views a specific card's detail	A card exists	1. Send GET to the card detail endpoint with card_id	card_id: 6	Card details are returned	Card detail returned correctly	**Passed**
TC-CARD-06	Admin views a passenger's card together with its zone subscriptions	Passenger owns a card with at least one subscription	1. Send GET /api/payment/passengers/<passenger_id>/card/	passenger_id	Response includes passenger info, card info, and a subscriptions array	Response returned matching the documented structure, including subscriptions	**Passed**

TC-CARD-07	Merchant tops up a card's cash balance	A card exists and is not blocked	1. Send POST to the top-up endpoint with qr_token and amount	amount: 5000	card.balance increases by the given amount; a TopUpTransaction is recorded	Balance updated; TopUpTransaction created with correct before/after balances	**Passed**
TC-CARD-08	Top-up fails for a blocked card	Card status='blocked'	1. Attempt to top up the blocked card	qr_token of blocked card	Request is rejected with an explanatory error	400 Bad Request returned: card is blocked and cannot be recharged	**Passed**
TC-CARD-09	Top-up or subscription purchase activates an inactive card	Card status='inactive'	1. Perform a top-up or subscription purchase against the inactive card	N/A	Card status automatically transitions to 'active'	Card status updated to active as part of the same transaction	**Passed**
TC-CARD-10	Admin views all payment (boarding) transactions platform-wide	At least one PaymentTransaction exists	1. Send GET /api/payment/transactions/	N/A	All boarding attempts (success and rejected) are listed	Transaction list returned correctly across all cards/devices	**Passed**

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-ZONE-01	Admin creates a new zone with subscription prices	Admin is logged in	1. Send POST /api/payment/zones/ with name, level, and daily/weekly/monthly prices	name: suburbs, level: 2, daily_price: 150, weekly_price: 700, monthly_price: 2000	Zone is created and available for route assignment and subscription sales	Zone created successfully with the given prices	**Passed**
TC-ZONE-02	Admin updates a zone's prices	A zone already exists	1. Send PATCH /api/payment/zones/<zone_id>/ with a new monthly_price	monthly_price: 2200	The zone's price is updated and reflected in subsequent subscription sales	Zone price updated successfully	**Passed**
TC-ZONE-03	Subscription purchase fails when the zone has no price configured	A zone exists with daily_price= 0 or null	1. Attempt to sell a daily subscription for that zone	subscription_type: daily on an unpriced zone	Request is rejected indicating the zone has no configured price for that duration	400 Bad Request returned with the expected message	**Passed**
TC-ZONE-04	Passenger card holds concurrent subscriptions for two differe	A card already holds an active subscription for zone A	1. Purchase a subscription for zone B on the same card	zone A (existing) and zone B (new)	Both subscriptions coexist as separate active ZoneSubscription records	Card ends up with two independent active subscriptions	**Passed**

	nt zones						
TC-ZONE-05	Boarding is granted when the subscription's zone level covers the route's zone	Card holds an active subscription for a zone with level ≥ the route's zone level	1. Scan the card on a route belonging to a lower- or equal-level zone	subscription zone level 3, route zone level 2	Boarding is granted (200 success)	200 OK returned; PaymentTransaction recorded with status success	**Passed**
TC-ZONE-06	Boarding is denied when the subscription's zone does not cover the route's zone	Card holds an active subscription only for a lower-level zone than the route	1. Scan the card on a route belonging to a higher-level zone	subscription zone level 1, route zone level 3	Boarding is denied with a zone_not_allowed-type status	403 Forbidden returned; rejection logged as zone_not_allowed	**Passed**
TC-ZONE-07	Boarding is denied when the card has no active subscription	Card has never purchased a subscription	1. Scan the card on any zoned route	N/A	Boarding is denied with a no_subscription-type status	402 Payment Required returned; rejection logged as no_subscription	**Passed**
TC-ZONE-08	Boarding is denied when the	Card's only subscription has an end_date in the past	1. Scan the card on the subscribed zone's route after expiry	expired subscription	Boarding is denied with a subscriptio	402 Payment Required returned; rejection	**Passed**

	card's subscription has expired				n_expired-type status	logged as subscription_expired	
TC-ZONE-09	System selects the best-covering subscription when multiple are active	Card holds active subscriptions for both a low-level and a high-level zone, with different expiry dates	1. Scan the card on a route requiring the high-level zone	two concurrent active subscriptions	The subscription with sufficient zone level is used for the decision, regardless of which expires sooner	Boarding granted using the correctly covering subscription	**Passed**
TC-ZONE-10	Repurchasing the same zone extends rather than duplicates the subscription	Card holds an unexpired active subscription for a given zone	1. Purchase another subscription of the same zone and type	same zone_id, subscription_type: weekly	end_date of the existing subscription is extended by the new duration; no duplicate row is created	Single ZoneSubscription row observed with extended end_date	**Passed**
TC-ZONE-11	Merchant and admin both retrieve the same zone list via one shared	Zones are configured	1. Call GET /api/payment/zones/ as admin 2. Call the same endpoint as merchant	N/A	Both roles receive an identical zone list; a passenger receives 403	Admin and merchant both receive 200 with the same data; passenger receives 403	**Passed**

	endpoint						
TC-ZONE-12	Admin views all subscription sale transactions platform-wide	At least one Subscription Transaction exists	1. Send GET /api/payment/subscription-transactions/	N/A	All subscription sales across all merchants are listed	Transaction list returned correctly	**Passed**

Table 11 test Cases for Scanner Device Payment Validation

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-SCAN-01	Admin registers a scanner device for a vehicle	A vehicle exists without an assigned scanner device	1. Send POST to the devices endpoint with device_code and vehicle_id	device_code: DEV-101, vehicle_id	A ScannerDevice is created with a generated secret key	Device registered successfully; secret key generated	**Passed**
TC-SCAN-02	Scanner authenticates using a valid device secret key	A registered Scanner Device exists	1. Send POST /api/payment/scan/ with header X-Device-Secret set to the correct secret	X-Device-Secret: <valid secret>	Device authentication succeeds and the request is processed	Authentication succeeded; scan processed	**Passed**

TC-SCAN-03	Scan request fails with an invalid or missing device secret	N/A	1. Send POST /api/payment/scan/ with an incorrect or absent X-Device-Secret header	X-Device-Secret: wrong-secret	Request is rejected with a 401/403 authentication error	401 Unauthorized returned as expected	**Passed**
TC-SCAN-04	Scan of an unknown QR token is rejected	The submitted qr_token does not match any card	1. Send a scan request with a non-existent qr_token	qr_token: UNKNOWN-TOKEN	Request is rejected with a card-not-found error	400 Bad Request returned: invalid QR code - card not found	**Passed**
TC-SCAN-05	Scan is rejected when the vehicle has no active trip	The scanning device's vehicle has no DriverTrip with status='active'	1. Scan a valid, active card on a vehicle with no active trip	N/A	Request is rejected indicating no active trip was found	400 Bad Request returned: no active trip found for this bus	**Passed**
TC-SCAN-06	Scan is rejected when the trip's route has no assigned zone	Active trip exists on a route with zone=null	1. Scan a valid card on this trip	N/A	Request is rejected indicating the route has no zone assigned	400 Bad Request returned: this route has no zone assigned	**Passed**
TC-SCAN-07	Successful scan sends a notification to the passenger	Card holds a subscription that covers the boarded	1. Perform a successful scan	N/A	A boarding-confirmation notification is generated for the passenger	Notification created and delivered via push/WebSocket	**Passed**

		route's zone					
TC-SCAN-08	Every scan attempt, successful or rejected, is persisted for audit purposes	N/A	1. Perform a mix of successful and rejected scans 2. Query PaymentTransaction records for the card	N/A	Each attempt (including rejections) appears as its own PaymentTransaction row with the correct status	All attempts recorded, including rejections where trip/route could not be resolved	**Passed**
TC-SCAN-09	Scanner device cannot authenticate as a regular user account	N/A	1. Attempt to access a scanner-only endpoint using a passenger's user token instead of a device secret	Authorization: Token <passenger_token>	Request is rejected since the endpoint only accepts ScannerDeviceAuthentication	401/403 returned as the endpoint does not accept user tokens	**Passed**
TC-SCAN-10	Admin lists all registered scanner devices	At least one Scanner Device is registered	1. Send GET to the devices endpoint	N/A	All registered devices are returned with their associated vehicle and status	Device list returned correctly	**Passed**

Table 12 test Cases for Notification System

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-NOTIF-01	Admin sends a notification to a specific user	Admin is logged in; target user exists	1. Send POST to the send-notification endpoint with a	title: 'Service update', body: 'Route 5 will be	A Notification record is created and delivered	Notification created and delivered; recipient receives it in real time	**Passed**

			user_id, title, and body	delayed today'	via push and WebSocket to that user		
TC-NOTIF-02	Admin broadcasts a notification to all passengers	Multiple passenger accounts exist	1. Send POST to the broadcast-notification endpoint	title: 'Platform maintenance notice'	A notification is created and delivered to every passenger	All passengers received the broadcast notification	**Passed**
TC-NOTIF-03	Driver receives a notification when their application is approved/rejected	Driver has a pending application	1. Admin approves or rejects the driver 2. Observe the driver's notification channel	N/A	Driver receives a notification reflecting the new approval status	Notification received by the driver as expected	**Passed**
TC-NOTIF-04	Notification delivery failure is recorded without blocking the request	The recipient has no valid Expo push token registered	1. Trigger a notification for a user without a push token	N/A	The Notification row is still created; push_sent is false and push_error is populated, but the triggering request still succeeds	Notification stored with push_sent=false; originating action completed successfully	**Passed**
TC-NOTIF-05	Passenger's unread notification count is accurate	Passenger has a mix of read and unread notifications	1. Connect to ws/notifications/ 2. Observe the initial unread-	N/A	The count matches the number of notifications with	Unread count matched the database state	**Passed**

			count payload		status='unread'		
TC-NOTIF-06	Notification is marked as read	An unread notification exists for the user	1. Send the mark-as-read request for the notification	notification_id	Notification status changes to 'read'	Status updated to read successfully	**Passed**

Table 13 test Cases for AI Passenger Counting Service

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-AI-01	AI service detects a person in the camera frame	The AI service is running with a valid camera source configured	1. Present a person to the configured camera feed	COCO class 0 (person)	The YOLOv8-based detector identifies the person with a confidence above the configured threshold	Person detected and bounding box drawn with confidence above threshold	**Passed**
TC-AI-02	AI service assigns a persistent tracking ID to a detected person	A person is detected in consecutive frames	1. Observe the same person across multiple frames	N/A	The same tracking ID is retained across frames while the person remains visible	Tracking ID remained stable across frames	**Passed**

TC-AI-03	Passenger boarding is counted when crossing the virtual line inward	A tracked person crosses the configured counting line in the 'enter' direction	1. Simulate a person walking across the line into the bus	N/A	An enter event is registered and total_in is incremented	total_in incremented by 1 upon crossing	**Passed**
TC-AI-04	Passenger alighting is counted when crossing the virtual line outward	A tracked person crosses the counting line in the 'exit' direction	1. Simulate a person walking across the line out of the bus	N/A	An exit event is registered and total_out is incremented	total_out incremented by 1 upon crossing	**Passed**
TC-AI-05	Current passenger count is transmitted to the backend, authenticated as the driver	The AI service is configured with a valid driver authentication token	1. Trigger a count change 2. Observe the outgoing POST to /api/driver/passenger-count/update	N/A	The request is accepted and associated with the driver's currently active trip/vehicle	Request accepted; passenger count updated on the corresponding vehicle	**Passed**
TC-AI-06	Passenger count update is rejected when the driver has no active trip	The authenticated driver has no trip with status='active'	1. Send a passenger-count update while the driver has no active trip	N/A	Request is rejected or ignored since no active vehicle/trip context can be resolved	Request rejected with an appropriate error indicating no active trip	**Passed**

Table 14 test Cases for AI Passenger Counting Service

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-COMP-01	Passenger submits a complaint without an image	Passenger is logged in	1. Send POST to the complaints endpoint with only a message field	message: 'Driver skipped my stop'	Complaint is created successfully without a required image	Complaint created with image left null	**Passed**
TC-COMP-02	Complaint submission fails with an empty message	Passenger is logged in	1. Send POST to the complaints endpoint with an empty message field	message: ''	Request is rejected with a validation error	400 Bad Request returned indicating the message is required	**Passed**
TC-COMP-03	Admin views all complaints across all passengers	Multiple passengers have submitted complaints	1. Send GET to the admin complaints endpoint	N/A	All complaints are listed with the submitting passenger identified	All complaints returned correctly with passenger references	**Passed**
TC-COMP-04	A passenger cannot view another passenger's	Two passengers exist, each with their own complaints	1. Log in as passenger A 2. Attempt to access a	N/A	Passenger-facing endpoints do not expose complaints belongin	Access restricted to the requesting passenger's own submissions/admin-only listing	**Passed**

	complaints		complaints listing scoped to all users		g to other passengers		
TC-COMP-05	Complaint submission with an unsupported file type is rejected	Passenger is logged in	1. Submit a complaint with a non-image file attached	image: document.pdf	Request is rejected due to invalid file type for the image field	400 Bad Request returned indicating an invalid image format	**Passed**

Table 15 test Cases for AI Passenger Counting Service

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-DASH-01	Admin dashboard displays active vehicle count	Multiple vehicles exist with varying is_active values	1. Send GET to the statistics endpoint	N/A	Returned statistics include an accurate count of currently active vehicles	Active vehicle count matched the database state	**Passed**
TC-DASH-02	Admin dashboard reflects up-to-date transaction totals	New subscription/top-up transactions were recently created	1. Perform a new transaction 2. Re-query the statistics endpoint	N/A	Statistics reflect the newly created transaction	Statistics updated to include the new transaction	**Passed**
TC-DASH-03	Non-admin access to the statistics	A merchant or passenger token is used	1. Send GET to the statistics endpoint using a	N/A	Request is rejected with a 403 Forbidden response	403 Forbidden returned as expected	**Passed**

	endpoint is denied		non-admin token				
TC-DASH-04	Statistics endpoint responds correctly when no operational data exists yet	A freshly seeded environment with no trips/transactions	1. Send GET to the statistics endpoint on an empty dataset	N/A	Endpoint returns zero-valued statistics rather than an error	Zero/empty statistics returned without error	**Passed**

Test Execution Summary

Total test cases: 120

Passed: 120

Failed: 0

Not Tested: 0

Requirements Traceability Matrix

req-id	Use cases	analysis	System design	Detailed design	coding	Test cases
REQ-1.1	US-1	Sign Up				TC-AUTH-01
REQ-1.3	US-2	Sign in				TC-AUTH-02
REQ-2.1	US-3	Plan Trip				TC-02
REQ-2.2, REQ-2.3, REQ-2.4	US-4	Plan Trip				TC-03, TC-08, TC-16
REQ-2.6, REQ-3.1	US-5	Plan Trip / Route Information				TC-04, TC-31
REQ-4.1, REQ-4.5	US-6	Bus Tracking				TC-05
REQ-4.3	US-7	Bus Tracking				TC-07, TC-15
REQ-3.2.1, REQ-3.2.2	US-8	Bus Information				TC-06
REQ-2.7	US-9	Manage Favorite Trips				TC-09
REQ-2.7	US-10	Manage Favorite Trips				TC-10, TC-11
REQ-2.7	US-11	Manage Favorite Trips				TC-12, TC-14

REQ-1.5, REQ-1.6, REQ-1.7	US-12	Manage Account Information				TC-17, TC-18, TC-19, TC-20, TC-21, TC-22, TC-23
DR-1	US-13	Sign Up				TC-AUTH-01
DR-1	US-14	Sign in				TC-AUTH-02
DR-7	US-15	Manage Trip				TC-53
DR-5	US-16	Manage Trip				TC-54
DR-8	US-17	Manage Trip				TC-48, TC-51
DR-3	US-18	Manage Trip				TC-49
DR-4	US-19	Manage Trip				TC-50
DR-8	US-20	Manage Trip				TC-51
DR-7	US-21	Manage Trip				TC-52
DR-2	US-22	Manage Account Information				TC-17, TC-18, TC-19, TC-20, TC-21, TC-22, TC-23
AD-1.1	US-23	Sign in				TC-AUTH-02
AD-2.1	US-24	Manage Users				TC-32, TC-33, TC-34, TC-35, TC-36, TC-37, TC-38, TC-39
AD-2.1, AD-2.1.2	US-25	Manage Users				TC-32, TC-39
AD-2.1, AD-2.2.4	US-26	Manage Users / Manage Drivers				TC-36, TC-37
AD-2.1.3, AD-2.1.4	US-27	Manage Users				TC-38
AD-4.1, AD-4.2, AD-4.3, AD-4.4	US-28	Manage Route				TC-24, TC-25, TC-26, TC-27, TC-28, TC-29, TC-30, TC-31
AD-4.1	US-29	Manage Route				TC-25, TC-26, TC-27
AD-4.3	US-30	Manage Route				TC-28, TC-29
AD-4.4	US-31	Manage Route				TC-30
AD-3.1, AD-3.2, AD-3.3, AD-3.4	US-32	Manage Vehicle				TC-40, TC-41, TC-42, TC-43, TC-44, TC-45, TC-46, TC-47
AD-3.1	US-33	Manage Vehicle				TC-41, TC-42, TC-43
AD-3.3	US-34	Manage Vehicle				TC-44, TC-45
AD-3.4	US-35	Manage Vehicle				TC-46
AD-6.1, AD-6.2	US-36	Sending Notification				TC-57, TC-58, TC-59, TC-61
AD-6.1	US-37	Sending Notification				TC-60
AD-7.1, AD-7.5	US-38	View Dashboard for Reporting				TC-62, TC-66
AD-7.2, AD-7.3, AD-7.4	US-39	View Dashboard for Reporting				TC-63, TC-64, TC-65

Chapter 3 Design and Architecture

3.1 High-Level Design

This chapter presents the architectural design of the Smart Public Transportation Platform (PTP). The architecture describes the main system components, their responsibilities, communication mechanisms, and the interaction between different actors and services.

The proposed system follows a client-server architecture combined with a layered backend architecture. The platform consists of multiple client applications, a centralized backend server, a relational database, and independent external services.

The client side includes a mobile application developed using React Native with Expo, which provides different interfaces for passengers, drivers, and merchants. In addition, an administration dashboard is used by administrators to manage system resources such as users, vehicles, routes, subscriptions, and notifications.

The backend is implemented using Django REST Framework (DRF) and Django Channels. It provides RESTful APIs for standard operations and WebSocket communication for real-time features such as live bus tracking, passenger occupancy updates, and instant notifications.

The backend follows a layered architecture:

Presentation Layer: Responsible for handling HTTP requests, API endpoints, authentication, and returning responses through Django REST Framework views.

Business Logic Layer: Contains application services responsible for implementing the core system rules, including trip management, payment validation, notification handling, route deviation detection, and passenger counting processing.

Data Access Layer: Responsible for interaction with the MySQL database through Django ORM models.

Communication Layer: Handles real-time communication using WebSocket consumers and asynchronous messaging through Django Channels.

The system also includes two standalone services:

AI Passenger Counting Service

A separate Python process responsible for analyzing video streams from bus cameras. It uses YOLOv8 and ByteTrack algorithms to detect and track passengers, calculate occupancy, and send updated passenger counts to the backend.

Bus Scanner Service

A QR-based scanning system that simulates the physical payment terminal installed inside buses. It communicates with the backend using secure device authentication to validate passenger transport cards and subscriptions.

The overall architecture enables scalability, separation of responsibilities, and easier maintenance by isolating independent system components

3.2 System Block Diagram

The system block diagram provides a high-level overview of the major components of the Smart Public Transportation Platform and illustrates the communication flow between them.

The architecture consists of four main layers:

1. Client Layer

This layer contains all user-facing applications:

Passenger Mobile Application

Driver Mobile Application

Merchant Mobile Application

Administrator Dashboard

Clients communicate with the backend through REST APIs for standard operations such as authentication, route management, subscription management, and profile operations.

For real-time features, clients establish WebSocket connections to receive live vehicle positions, passenger occupancy information, and notifications.

2. Backend Application Layer

The backend represents the core of the platform and is implemented using Django REST Framework.

It contains several business modules:

Authentication and User Management

User registration and login

Role-based access control

Token authentication

Trip and Vehicle Management

Driver trips

Vehicle assignment

GPS location processing

Route Management

Route creation

Stop management

Route path generation

Real-Time Tracking Service

Receives driver GPS updates

Detects route deviations

Broadcasts vehicle positions using WebSockets

Payment and Subscription Management

Transport card management

Zone-based subscription validation

Scanner device authentication

Payment transaction logging

Notification Service

Push notifications using Expo Push Notifications

Real-time notification delivery through WebSockets

3. External Services Layer

The system integrates with external services:

AI Passenger Counting Service

Receives camera frames

Performs passenger detection and tracking

Sends occupancy updates to the backend

Mapping Services

OSRM service for route path generation

Nominatim service for geocoding operations

4. Data Storage Layer

All persistent system data is stored in a MySQL database.

The database contains entities representing:

Users and authentication data

Drivers and vehicles

Routes and stops

Trips and GPS history

Transport cards and subscriptions

Payment transactions

Notifications

Passenger occupancy information

The backend accesses the database using Django ORM, providing abstraction between application logic and database operations.

Communication Flow:

Users interact with their applications through the client layer.

Requests are sent to the Django backend through REST APIs.

Backend views validate requests and delegate operations to service classes.

Services execute business rules and interact with database models.

Real-time events are delivered through WebSocket connections.

External services such as AI passenger counting communicate with backend APIs to update system information.

3.3 Architectural Design Pattern

The architectural design pattern defines the general organization and communication structure of the system. For the Smart Public Transportation Platform (PTP), the selected architectural pattern is Client-Server Architecture supported by a Layered Backend Design.

This pattern is suitable because the platform consists of multiple client applications, including passenger, driver, merchant mobile interfaces, and an administrator dashboard. All clients communicate with a centralized backend responsible for authentication, business logic processing, real-time communication, and database management.

The selected architecture separates the system responsibilities into different layers, improving maintainability and allowing each component to be developed and modified independently.

• Selected Pattern: Client-Server Architecture with Layered Backend Design

The clients communicate with the backend server through REST APIs for standard operations and WebSocket channels for real-time features such as vehicle tracking, passenger occupancy updates, and notifications.

The backend follows a layered structure consisting of:

Presentation Layer: Responsible for handling API requests, authentication, input validation, and returning responses through Django REST Framework views.

Business Logic Layer: Contains service modules responsible for implementing system rules such as trip management, payment validation, route deviation detection, notifications, and passenger counting processing.

Data Access Layer: Provides communication with the MySQL database through Django ORM models.

• Justification

The platform requires a centralized architecture capable of supporting different user roles, including passengers, drivers, merchants, and administrators. The layered design provides a clear separation between user interaction, business processing, and data storage.

Furthermore, the architecture supports real-time transportation features through WebSocket communication and allows integration with external services such as the AI passenger counting system and QR-based bus scanner system.

• Benefits

The selected architecture provides several advantages:

Improves separation of concerns between system components.

Simplifies maintenance and future expansion.

Supports real-time communication for tracking and notifications.

Centralizes authentication and security management.

Allows independent development of backend services and external components.

Provides clear mapping between application modules and database entities.

3.4 System Architecture Design

The system architecture is organized into four main areas: the Client area, the Server area, the Presentation Layer, the Business Logic Layer, and the Data Access Layer. The Client area contains the user interface through which users interact with the system. The Presentation Layer receives requests through the API gateway and controller. The Business Logic Layer contains the functional modules responsible for managing accounts, notifications, trips, live locations, vehicles, routes, and reports. The Data Access Layer provides controlled access to the MySQL database.

The Account Management module handles passenger, driver, and admin accounts. The Notification Services module manages Expo push notification tokens and notification delivery. The Trip Management module handles driver trip start/stop operations and passenger trip tracking. The Location Service receives and processes driver GPS updates. The Vehicle Management module manages vehicle status, fullness, and route assignment. The Route Management module manages routes, stops, and route-stop ordering. The Reporting Service provides administrative statistics and reports.

3.4 Database Design

3.4.1 Entity-Relationship Diagram (ERD Diagram)

The Entity-Relationship Diagram (ERD) represents the database structure of the Smart Public Transportation Platform. It illustrates the main entities, their attributes, and relationships implemented in the MySQL database.

The ERD includes the main entities responsible for managing user accounts, transportation operations, payment services, and real-time features.

The main database entities include:

User and Authentication Entities:

Represent passengers, drivers, merchants, and administrators with their authentication information and roles.

Transportation Entities:

Include vehicles, routes, stops, route-stop relationships, driver trips, and vehicle locations used for managing public transportation operations and tracking.

Payment Entities:

Include transport cards, zone subscriptions, subscription transactions, top-up transactions, payment transactions, and scanner devices responsible for managing the electronic payment system.

Notification Entities:

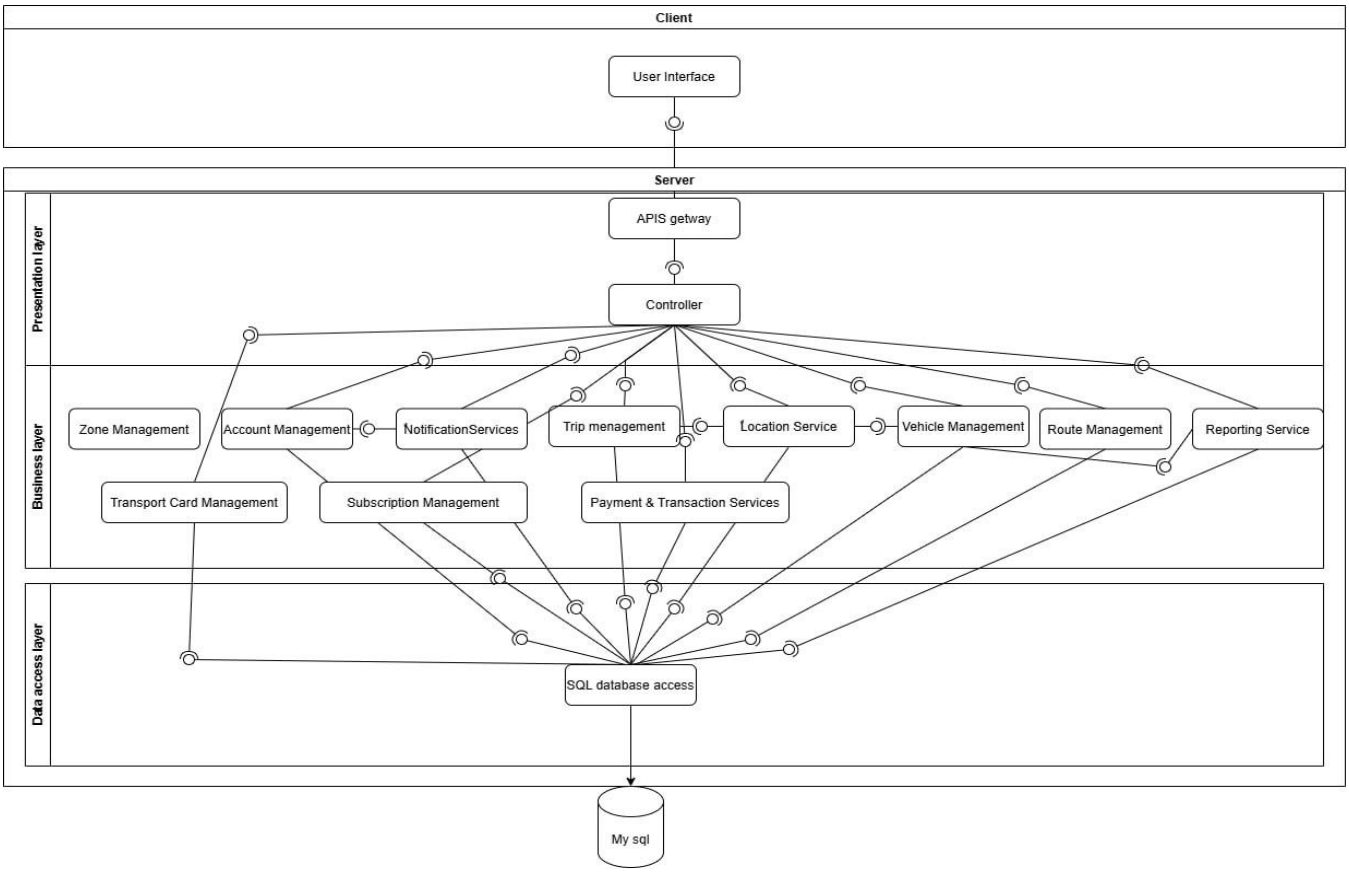
Store user notifications and Expo push tokens used for delivering real-time alerts.

Passenger Service Entities:

Include complaints and favorite trips that allow passengers to interact with the platform.

AI Passenger Counting Data:

Passenger occupancy information is stored within the vehicle entity and updated through communication with the AI counting service.



3.4.2 Relational Schema

The relational schema defines the structure of the MySQL database used in the Smart Public Transportation Platform. It describes the database tables, attributes, data types, constraints, and the purpose of each field.

The schema was designed to support all main platform functionalities including user management, vehicle and route operations, real-time tracking, AI passenger counting, transport card payment, subscription management, and notification services.

Table Name	Column Name	Data Type	Constraints	Description

User	user_id	INT	PRIMARY KEY	Unique identifier for each user
User	email	VARCHAR(150)	NOT NULL, UNIQUE	User email used for authentication
User	password	VARCHAR(255)	NOT NULL	Hashed user password
User	full_name	VARCHAR(150)	NOT NULL	User full name
User	phone	VARCHAR(30)	NOT NULL	User phone number
User	role	VARCHAR(30)	DEFAULT passenger	Defines user role (passenger, merchant, admin)
User	is_admin	BOOLEAN	DEFAULT FALSE	Indicates administrator account
User	account_status	VARCHAR(30)	DEFAULT active	Current account status
Driver	Column Name	Data Type	Constraints	Description
Driver	driver_id	INT	PRIMARY KEY	Driver identifier
Driver	full_name	VARCHAR(150)	NOT NULL	Driver name
Driver	email	VARCHAR(150)	UNIQUE, NOT NULL	Driver email
Driver	password	VARCHAR(255)	NOT NULL	Hashed driver password

Driver	phone	VARCHAR(30)	NOT NULL	Driver phone number
Driver	vehicle_id	INT	FOREIGN KEY	Assigned vehicle
Driver	approval_status	VARCHAR(30)	DEFAULT pending	Driver approval state
Driver	account_status	VARCHAR(30)	DEFAULT active	Driver account status
Vehicle	vehicle_id	INT	PRIMARY KEY	Vehicle identifier
Vehicle	vehicle_number	VARCHAR(50)	UNIQUE, NOT NULL	Vehicle number
Vehicle	vehicle_type	VARCHAR(50)	NOT NULL	Vehicle category
Vehicle	capacity	INT	DEFAULT 50	Maximum passenger capacity
Vehicle	passenger_count	INT	DEFAULT 0	Current passenger count from AI service
Vehicle	is_full	BOOLEAN	DEFAULT FALSE	Indicates whether vehicle reached capacity
Vehicle	route_id	INT	FOREIGN KEY	Assigned route

Vehicle	is_active	BOOLEAN	DEFAULT TRUE	Vehicle availability status
Route	route_id	INT	PRIMARY KEY	Route identifier
Route	route_name	VARCHAR(150)	UNIQUE, NOT NULL	Route name
Route	zone_id	INT	FOREIGN KEY	Transportation zone
Route	start_latitude	DECIMAL	NOT NULL	Starting latitude
Route	start_longitude	DECIMAL	NOT NULL	Starting longitude
Route	end_latitude	DECIMAL	NOT NULL	Ending latitude
Route	end_longitude	DECIMAL	NOT NULL	Ending longitude
Route	path	JSON	-	Generated route path
Route	price	DECIMAL	-	Route price
Stop	stop_id	INT	PRIMARY KEY	Stop identifier
Stop	name	VARCHAR(150)	NOT NULL	Stop name
Stop	latitude	DECIMAL	NOT NULL	Stop latitude

Stop	longitude	DECIMAL	NOT NULL	Stop longitude
Stop	latitude	DECIMAL(10,7)	NOT NULL	Stop latitude
Stop	longitude	DECIMAL(10,7)	NOT NULL	Stop longitude
DriverTrip	trip_id	INT	PRIMARY KEY	Trip identifier
DriverTrip	driver_id	INT	FOREIGN KEY	Assigned driver
DriverTrip	vehicle_id	INT	FOREIGN KEY	Used vehicle
DriverTrip	route_id	INT	FOREIGN KEY	Trip route
DriverTrip	status	VARCHAR(30)	NOT NULL	Trip status
DriverTrip	started_at	DATETIME	-	Trip start time
DriverTrip	ended_at	DATETIME	-	Trip end time
VehicleLocation	location_id	INT	PRIMARY KEY	Location record identifier
VehicleLocation	trip_id	INT	FOREIGN KEY	Related trip
VehicleLocation	latitude	DECIMAL	NOT NULL	Current latitude
VehicleLocation	longitude	DECIMAL	NOT NULL	Current longitude

VehicleLocation	speed_kmh	DECIMAL	-	Vehicle speed
VehicleLocation	is_off_route	BOOLEAN	DEFAULT FALSE	Route deviation indicator
Zone	zone_id	INT	PRIMARY KEY	Zone identifier
Zone	name	VARCHAR(50)	UNIQUE	Zone name
Zone	level	INT	NOT NULL	Zone hierarchy level
Zone	daily_price	DECIMAL	-	Daily subscription price
Zone	weekly_price	DECIMAL	-	Weekly subscription price
Zone	monthly_price	DECIMAL	-	Monthly subscription price
TransportCard	card_id	INT	PRIMARY KEY	Card identifier
TransportCard	card_number	VARCHAR(100)	UNIQUE	Physical card number
TransportCard	qr_token	VARCHAR(255)	UNIQUE	QR value encoded on card
TransportCard	owner_id	INT	FOREIGN KEY	Card owner
TransportCard	balance	DECIMAL	DEFAULT 0	Card balance

ZoneSubscription	subscription_id	INT	PRIMARY KEY	Subscription identifier
ZoneSubscription	card_id	INT	FOREIGN KEY	Related card
ZoneSubscription	zone_id	INT	FOREIGN KEY	Covered zone
ZoneSubscription	start_date	DATE	NOT NULL	Subscription start
ZoneSubscription	end_date	DATE	NOT NULL	Subscription expiration
ExpoPushToken	driver_id	INT	FOREIGN KEY (Driver.driver_id)	Driver owner
ExpoPushToken	user_type	VARCHAR(30)	NOT NULL	Token owner type
ExpoPushToken	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Token creation time

3.4.3 Constraints and Keys

Database constraints are used to maintain data integrity, enforce business rules, and ensure consistency across the Smart Public Transportation Platform. The database design uses primary keys, foreign keys, unique constraints, default values, and validation rules to guarantee reliable data management and prevent invalid relationships between entities.

Primary keys are used to uniquely identify each record within the database tables:

- User.id → PRIMARY KEY

- Vehicle.id → PRIMARY KEY
- Route.id → PRIMARY KEY
- Zone.id → PRIMARY KEY
- Stop.id → PRIMARY KEY
- RouteStop.id → PRIMARY KEY
- DriverTrip.id → PRIMARY KEY
- VehicleLocation.id → PRIMARY KEY
- FavoriteTrip.id → PRIMARY KEY
- Complaint.id → PRIMARY KEY
- Notification.id → PRIMARY KEY
- ExpoPushToken.id → PRIMARY KEY
- TransportCard.id → PRIMARY KEY
- ZoneSubscription.id → PRIMARY KEY
- SubscriptionTransaction.id → PRIMARY KEY
- TopUpTransaction.id → PRIMARY KEY
- PaymentTransaction.id → PRIMARY KEY
- ScannerDevice.id → PRIMARY KEYForeign Keys

Foreign keys establish relationships between tables and maintain referential integrity:

Foreign Key	References	Description
Driver.vehicle_id	Vehicle.vehicle_id	Links driver to assigned vehicle
Vehicle.route_id	Route.route_id	Links vehicle to assigned route

RouteStop.route_id	Route.route_id	Links route-stop record to route
RouteStop.stop_id	Stop.stop_id	Links route-stop record to stop
DriverTrip.driver_id	Driver.driver_id	Links trip to driver
DriverTrip.vehicle_id	Vehicle.vehicle_id	Links trip to vehicle
DriverTrip.route_id	Route.route_id	Links trip to route
FavoriteTrip.passenger_id	User.user_id	Links favorite route to passenger
FavoriteTrip.route_id	Route.route_id	Links favorite record to route
Complaint.passenger_id	User.user_id	Links complaint to passenger
Notification.passenger_id	User.user_id	Links notification to passenger
Notification.driver_id	Driver.driver_id	Links notification to driver
ExpoPushToken.user_id	User.user_id	Links token to passenger/user
ExpoPushToken.driver_id	Driver.driver_id	Links token to driver

Unique Constraints

Unique constraints prevent duplicated values for attributes that must remain unique:

- User.email → UNIQUE
- Vehicle.vehicle_number → UNIQUE
- TransportCard.card_number → UNIQUE
- TransportCard.qr_token → UNIQUE
- ScannerDevice.device_code → UNIQUE
- ExpoPushToken.token → UNIQUE

Business Constraints

The following constraints enforce system business rules:

- Each passenger can have only one TransportCard.
- Each ScannerDevice belongs to only one vehicle.
- A route must belong to one transportation zone.
- A zone subscription must reference an existing transport card and valid zone.
- A payment transaction must contain the validation result status.
- Vehicle occupancy cannot exceed the defined vehicle capacity.
- The vehicle fullness status (is_full) is automatically calculated based on:

passenger_count >= capacity

- Only active subscriptions can be used during passenger boarding validation.
- Blocked or inactive transport cards cannot access transportation services.
- Subscription expiration dates must be greater than the subscription start date.
- Driver trips must have a valid assigned driver, vehicle, and route.

Indexing

Indexes are applied to frequently searched fields to improve database performance:

- User.email
- TransportCard.qr_token
- TransportCard.card_number
- VehicleLocation.vehicle_id
- DriverTrip.status
- Notification.user_id
- PaymentTransaction.created_at

These indexes improve query performance for authentication, card scanning, live tracking, notification retrieval, and transaction history.

3.5 External API Design

This section documents the most important HTTP APIs exposed by the Public Transportation Platform. The backend exposes RESTful endpoints through Django REST Framework. These endpoints support authentication, account management, passenger trip search, live trip tracking, driver trip lifecycle management, live GPS location updates, vehicle status updates, and administrative operations.

3.5.1 API Gateway Overview

The API gateway layer represents the unified access point through which client applications communicate with the backend. The mobile application and admin dashboard send HTTP requests to the backend API endpoints. Authentication is performed using Token Authentication, where protected requests include the Authorization header in the form: Token <token>.

•Routes requests to the appropriate controller and business service module.

•Protects restricted endpoints using Token Authentication and role-based checks.

•Provides a unified REST API surface for passenger, driver, and admin operations.

•Supports real-time tracking workflows through WebSocket communication in addition to HTTP APIs.

Base URL: /api/ (or the deployed domain URL in production).

Endpoint	Method	Service / App	Description	Request Parameters	Response	Status Codes
/api/auth/register	POST	Auth	Register a passenger or driver account.	email, full_name, phone, password, account_type. Driver only: has_vehicle, ID images, license image, optional vehicle_type, vehicle_number, route_id.	Account data. Passenger receives token; driver receives approval status.	201, 400, 500
/api/auth/login	POST	Auth	Authenticate passenger, driver, or admin.	email, password	User data and authentication token.	200, 400, 500
/api/auth/logout	POST	Auth	Log out by deleting the active token.	Authorization : Token <token>	detail message.	200, 401, 500
/api/accounts/passenger/profile	GET / PATCH	Accounts	View or update the passenger profile.	GET: token only. PATCH: email, full_name, phone, password.	Passenger profile data after retrieval or update.	200, 400, 401, 403, 500
/api/accounts/driver/profile	GET / PATCH	Accounts	View or update the driver profile, including driver documents.	GET: token only. PATCH: email, full_name, phone, password, ID images, license image.	Driver profile data with vehicle and route information.	200, 400, 401, 500
/api/passenger/complaints	POST	Passenger	Submit a passenger complaint.	message; optional image.	complaint_id, message, image_url, created_at, detail.	201, 400, 401, 403, 500
/api/passenger/trips/search	POST	Passenger Trips	Search for available routes between two points.	start and destination. Each point can be latitude/longitude or place_name.	start, destination, and matched routes.	200, 400, 401, 403, 500
/api/passenger/trips/<int:trip_id>/tracking	GET	Passenger Trips	Track an active trip.	Path: trip_id.	trip tracking details.	200, 401, 403, 404, 500
/api/passenger/trips/favorites	GET / POST	Passenger	List favorite routes or add a route to favorites.	GET: token only. POST: route_id.	favorites list or created favorite route details.	200, 201, 400, 401,

						403, 500
/api/passenger/routes/<int:route_id>/tracking	GET	Passenger Trips	Show active buses on a route based on passenger location.	Path: route_id. Query: latitude, longitude.	route_id, route_name, boarding_filter_applied, active_buses.	200, 400, 401, 403, 404, 500
/api/driver/trip/status	GET	Driver	Check the driver's current trip tracking status.	Authorization token.	is_tracking_active, trip, vehicle_id, route_id.	200, 401, 500
/api/driver/trip/start	POST	Driver	Start a driver trip and activate vehicle tracking.	Authorization token.	trip object and detail message.	200, 201, 400, 401, 500
/api/driver/trip/stop	POST	Driver	Stop the active driver tracking session.	Authorization token.	trip object and detail message.	200, 400, 401, 500
/api/driver/location	POST	Driver Tracking	Send the driver's live GPS location.	latitude, longitude, optional speed_kmh and heading.	location, alert, and detail message.	201, 400, 401, 500
/api/driver/vehicle/status	PATCH	Driver	Update the vehicle fullness status.	is_full.	vehicle data and detail message.	200, 400, 401, 500
/api/admin/statistics	GET	Admin	View system dashboard statistics.	Admin token.	accounts, network, vehicles, trips, tracking, favorites, complaints.	200, 401, 403, 500
/api/admin/accounts	GET / POST	Admin Accounts	List accounts or create a passenger/driver account by admin.	GET: admin token. POST: email, full_name, phone, password, account_type; driver fields if needed.	passengers/drivers list or created account data.	200, 201, 400, 401, 403, 500
/api/admin/drivers/<int:driver_id>/<str:action>	POST	Admin Drivers	Approve or reject a driver registration request.	Path: driver_id, action = approve/reject. Body may include vehicle_id or vehicle data and optional route_id.	driver, vehicle, and detail message.	200, 400, 401, 403, 404, 500
/api/admin/complaints	GET	Admin Complaints	View passenger complaints.	Admin token.	complaints list.	200, 401, 403, 500
/api/admin/routes	GET / POST	Admin Routes	List all routes or	GET: admin token. POST: route_name,	routes list or created route and detail message.	200, 201, 400,

			create a new route.	start/end coordinates, price, stop_ids, optional is_active.		401, 403, 500

Detailed API Specifications

The API endpoints are grouped according to the main service area they support:

•Authentication APIs: registration, login, and logout for passengers, drivers, and admins.

•Account APIs: passenger and driver profile retrieval and update operations.

•Passenger APIs: complaints, route search, trip tracking, favorite routes, and route tracking.

•Driver APIs: trip status, trip start, trip stop, GPS location updates, and vehicle fullness status.

•Admin APIs: dashboard statistics, account management, driver approval/rejection, complaint viewing, and route management.

3.5.2 WebSocket and Notification Integration

In addition to REST APIs, the system uses WebSocket communication for real-time vehicle tracking. Drivers send live location updates during active trips, and passengers can track active buses or specific trips. The WebSocket layer supports continuous communication between the client and backend without requiring repeated HTTP polling. Expo Notifications are used to support push notifications on the mobile application, while Expo push tokens are stored in the database for passengers and drivers.

3.6 Authentication & Security

Security and access control are essential because the platform manages user accounts, driver documents, vehicle information, route data, GPS locations, and administrative operations. The system uses Token Authentication provided through Django REST Framework. After login, the backend returns an authentication token that must be sent with protected requests.

5.4.1 Authentication Mechanism

Authentication method: Token Authentication.

Authorization header format: Authorization: Token <token>.

User roles: Passenger, Driver, and Admin.

Driver accounts require administrative approval before accessing driver-specific features.

Logout is implemented by deleting the active token.

3.6.1 User Roles and Permissions

Role	Permissions	Access Level
Admin	Manage accounts, approve/reject drivers, manage routes, view complaints, and access statistics.	Full administrative access
Driver	View/update profile, start/stop trips, send live GPS location, and update vehicle fullness status.	Driver operational access
Passenger	Register/login, search trips, track routes/trips, submit complaints, and manage favorite routes.	Passenger application access

Additional Security Measures

Protected endpoints require authentication using a valid token.

Administrative endpoints require admin privileges before allowing sensitive operations.

Driver approval status is checked before enabling driver-specific actions.

Input validation is applied through serializers to reduce invalid or unsafe data.

Sensitive data such as passwords are stored in hashed form rather than plain text.

Database relations and constraints prevent invalid references between users, drivers, vehicles, routes, trips, and complaints.

Deployment on Railway centralizes the backend environment and supports controlled production configuration.

Chapter 4 Test and Implement

1. Programming Languages and Frameworks:

This chapter documents the programming languages, frameworks, development tools, database technologies, deployment environment, and testing setup used to implement the Public Transportation Platform (PTP). The system is implemented as a multi-component platform consisting of a Django REST backend, a React and TypeScript administrator dashboard, a React Native mobile application developed with Expo, and a standalone Python-based artificial intelligence service for passenger counting.

The technology stack was selected to support modular development, RESTful API communication, cross-platform mobile applications, real-time vehicle tracking, push notifications, database management, and AI-based passenger occupancy monitoring. The backend provides the central communication layer between the mobile application, administrator dashboard, AI service, and bus scanner devices.

The backend is implemented using Python 3.11 and Django 4.2.7. Django REST Framework (DRF) provides the REST API layer, while Django Channels and Daphne are used to support WebSocket-based real-time communication. The system uses MySQL as its database server. The mobile application is implemented using React Native with Expo and TypeScript, while the administrator dashboard is implemented using React, TypeScript, and Vite. In addition, a separate Python service uses YOLOv8 and ByteTrack to automatically estimate the number of passengers inside active buses.

1.1. Mobile Frontend: TypeScript, React Native, and Expo:

The mobile application is implemented using React Native with Expo and TypeScript. The application supports three user roles: passengers, drivers, and merchants. Each role has its own set of screens and navigation structure, while authentication and shared functionality are handled through common application services and state management.

TypeScript was adopted to provide static typing, improve code maintainability, and reduce errors during development. Expo was selected to simplify React Native development, testing, and application deployment.

The passenger application provides functionality for searching public transportation routes, viewing route details, tracking active buses in real time, managing favorite routes, viewing transport cards and zone subscriptions, submitting complaints, and receiving notifications.

The driver application provides functionality for driver authentication and profile management, starting and stopping trips, and sending GPS location updates while a trip is active. Passenger occupancy is not entered manually by the driver; instead, it is provided automatically by the standalone AI passenger-counting service.

The merchant application provides functionality for looking up passenger transport cards using QR tokens, selling zone-based subscriptions, topping up card balances, and viewing transaction history.

Several libraries are used to support the mobile application. Expo Location is used to obtain GPS coordinates for driver tracking, Expo Notifications is used for push notifications, and React Native Maps is used for map visualization. Axios is used for HTTP communication with the backend, while TanStack React Query manages server-side data and caching. Zustand is used for client-side state management, and Expo Secure Store is used for secure local storage of authentication-related data and push notification tokens. Expo Router provides file-based navigation with separate route groups for authentication, passengers, drivers, and merchants.

1.2. 1.3. Backend Services: Python 3.11, Django 4.2.7, and Django REST The The backend is developed using Python 3.11 and Django 4.2.7. It acts as the central service of the platform and provides the REST APIs consumed by the mobile application and administrator dashboard.

Django was selected because it provides a mature web framework, database integration through its ORM, structured project organization, authentication support, and integration with Django REST Framework and Django Channels.

The backend is organized into a single Django application named PTP, with its functionality divided into models, serializers, services, views, URLs, authentication modules, and WebSocket consumers. This structure separates business logic from API views and provides a clear organization for the different platform domains.

Django REST Framework is used to implement API views, serializers, request validation, authentication, and structured JSON responses. The platform uses multiple authentication mechanisms according to the type of client. Passenger, merchant, and administrator accounts use Django REST Framework token authentication. Drivers use a dedicated driver token authentication mechanism, while physical bus scanner devices use a separate device authentication mechanism based on a secret key.

The backend uses MySQL as its database server. The database contains models representing users, drivers, vehicles, routes, stops, trips, vehicle locations, zones, transport cards, subscriptions, transactions, scanner devices, notifications, complaints, and favorite routes.

The backend also integrates external services such as OSRM for generating route paths and Nominatim/OpenStreetMap for geocoding location names.1.4 Real-Time Communication and Notifications:

Real-time communication is implemented using Django Channels and Daphne through WebSocket connections. This mechanism is used to deliver vehicle location and trip information to connected clients without requiring continuous HTTP polling.

When a driver is performing an active trip, the mobile application periodically sends GPS coordinates to the backend. The backend stores each location update as a VehicleLocation record and checks whether the vehicle has deviated from its assigned route. The resulting location, passenger occupancy information, and relevant tracking alerts are then broadcast through WebSocket connections.

The platform provides separate WebSocket channels for administrator vehicle tracking, passenger trip tracking, and user notifications. This allows passengers to monitor active trips while administrators can monitor vehicles across the transportation network in real time.

The current WebSocket implementation uses Django Channels with an in-memory channel layer. Therefore, the current configuration is designed for a single server process rather than a distributed multi-instance deployment.

For push notifications, the platform uses Expo Push Notifications rather than Firebase Cloud Messaging. The backend stores Expo push tokens and uses them to send notifications to passengers and drivers. Notifications are also stored in the database and can be delivered through the WebSocket notification channel when a client is connected.

Notification events implemented in the system include driver approval or rejection, administrator notifications, passenger broadcasts, successful boarding notifications, and tracking-related alertsTable summarizes the primary programming languages and frameworks used in the Public Transportation Platform:

1.5. AI Passenger-Counting Service

In addition to the main backend, the platform includes a standalone Python-based AI service responsible for estimating the number of passengers inside a bus.

The service uses YOLOv8 for person detection and ByteTrack for maintaining persistent identities of detected people across video frames. A virtual line is placed within the camera view, and the system detects when tracked individuals cross the line in either direction. These events are used to calculate the number of passengers entering and leaving the vehicle.

The resulting passenger count is periodically sent to the Django backend through an authenticated API endpoint. The backend associates the count with the driver's currently active trip and updates the corresponding vehicle's passenger count and occupancy status.

This information is then included in the real-time vehicle tracking updates, allowing connected applications to receive both the current vehicle location and passenger occupancy information.

1.6. Database and Development Environment

The platform uses MySQL as its primary relational database. Django's ORM is responsible for defining and interacting with the database models.

The backend is served through Daphne because the system requires ASGI support for Django Channels and WebSocket communication. The project also includes OpenAPI documentation through drf-spectacular, providing automatically generated API documentation.

The mobile application uses Expo Application Services (EAS) for application building and distribution. During development, tools such as Expo development tooling and ngrok were used to facilitate application testing and communication with the locally running backend.

The repository also contains a standalone QR scanner simulator that can be used to manually test the bus card scanning and subscription-validation workflow without requiring a physical scanner device.

1.7. Testing Setup

The current repository does not contain an automated unit, integration, or end-to-end testing suite. No dedicated test directories, Postman collections, CI test pipelines, or recorded automated test results were found in the inspected project.

However, the project includes tools that support manual testing and system validation. The scanner simulator can be used to test the QR-based boarding workflow against the backend, while the validate_setup.py script performs basic Django configuration and import checks.

Therefore, testing in the current implementation primarily relies on manual functional verification of the backend API, mobile application, dashboard, real-time tracking, payment workflow, scanner simulation, and AI passenger-counting service.

Table summarizes the primary programming languages and frameworks used in the Public Transportation Platform:

Component	Language	Framework / Tool	Role
Mobile Application	TypeScript	React Native 0.81.5, Expo 54, Expo Router	Passenger and driver mobile interfaces, live location, notifications
Admin Dashboard	TypeScript	React.js 18, Vite 5	Administrator web dashboard, maps, statistics, management screens
Backend API	Python 3.11	Django 4.2.7, Django REST Framework 3.14	Authentication, route management, trips, vehicles, complaints, REST APIs
Real-Time service	Python 3.11	Django Channels 4.1.0, Daphne 4.1.2	WebSocket communication for real-time bus tracking
Database	SQL	MySQL with mysqlclient 2.2.4	Structured storage for users, drivers, vehicles, routes, stops, trips, and locations
Notification service	TypeScript	Expo Notifications 0.32.17	Mobile push notification support

2. Development Environment and Project Structure:

A reproducible development environment was essential because the project contains multiple components: a Django backend, an administrator web dashboard, and a mobile application. The team developed the system on Windows workstations using Visual Studio Code as the primary integrated development environment. The backend was executed using a Python virtual environment, while the dashboard and mobile application were managed through Node.js package scripts.

2.1 Operating System and Hardware:

Development was carried out on Windows-based machines. Windows was suitable for this project because it supported Visual Studio Code, Python 3.11, Node.js, MySQL Workbench, Postman, Expo CLI, and browser-based dashboard testing. The development machines were sufficient to run the Django backend server, MySQL database tools, the Vite development server, and the Expo mobile development environment simultaneously during integration testing.

2.2 Integrated Development Environments:

Visual Studio Code was used as the main development environment for backend, dashboard, and mobile application development. VS Code provided extensions for Python, Django, TypeScript, React, React Native, Git, and formatting. MySQL Workbench was used to inspect and manage database tables, verify inserted records, and validate relational data during development. Postman was used to test REST API endpoints before connecting them to the frontend applications.

2.3 Deployment and Runtime Environment:

The backend was deployed using Railway, which provides a managed cloud environment suitable for student and prototype projects. Railway simplified environment variable management, backend hosting, and database connectivity. During mobile development, Expo Go was used as the primary runtime testing environment. The administrator dashboard was tested locally during development using Vite and connected to the backend APIs for integration testing.

The project did not rely on a full Docker-based deployment workflow or GitHub Actions CI/CD pipeline. Instead, the team followed a manual build, test, and deployment workflow. This approach was appropriate for the project scope and timeline, while still allowing future enhancement toward automated CI/CD if the platform is expanded.

2.4 Project Structure:

The source code is organized in a GitHub repository containing the main system components. The structure separates the backend, administrator dashboard, and mobile application to maintain clear responsibility boundaries and reduce development conflicts. This organization also supports independent development and testing of each subsystem.

```
public-transportation-platform/
├── PTP/                          # Django backend project and API implementation
├── p_transportation_p/           # Backend project configuration / related Django structure
├── Senior1_dashboard/            # React TypeScript admin dashboard
├── Senior1_mobile/               # Expo React Native mobile application
└── requirements.txt              # Backend Python dependencies
```

2.4.1 Backend Project Structure

The backend follows a structured Django project architecture. The main Django application is organized into modules responsible for models, serializers, services, API views, URL routing, authentication, and WebSocket communication. The project contains functionality related to users, drivers, merchants, vehicles, routes, stops, trips, live vehicle tracking, transport cards, subscriptions, transactions, notifications, complaints, and administrative operations.

Django models define the relational database structure and represent the main entities of the transportation platform. Serializers are responsible for converting model instances and validated request data into structured JSON representations. API views expose RESTful endpoints that are consumed by both the mobile application and the administrator dashboard.

Business logic is separated into service classes where appropriate, including services related to authentication, trip management, vehicle tracking, route deviation detection, notifications, payments, subscriptions, and route generation. The backend also uses Django Channels and WebSocket consumers to provide real-time vehicle tracking, passenger trip tracking, and notification communication.

2.4.2 Admin Dashboard Structure:

2.4.2 Admin Dashboard Structure

The administrator dashboard is implemented as a React 18 and TypeScript application using Vite. Its structure is organized around functional features such as authentication, dashboard statistics, users, drivers, vehicles, bus routes, bus stops, zones, merchants, transactions, complaints, and notifications.

The dashboard uses reusable components and feature-specific modules to provide administrative interfaces for managing the transportation platform. Axios is used for communication with the Django REST API, while TanStack React Query is used for server-state management and caching. TanStack Router handles client-side navigation, and TanStack Table is used for structured administrative data. Form handling and validation are supported through Formik and Yup, while Recharts and Google Maps integration are used for statistical and map-based interfaces.

The dashboard also provides live vehicle monitoring functionality, allowing administrators to observe active vehicles and their current locations through the platform's real-time tracking system.

2.4.3 Mobile Application Structure:

The mobile application is implemented using React Native, Expo, and TypeScript, with Expo Router used for application navigation. The application is organized according to the supported user roles: passengers, drivers, and merchants.

Passenger functionality includes route search, route and trip details, live vehicle tracking, favorite routes, transport cards, zone subscriptions, complaints, and notifications. Driver functionality includes authentication, profile management, starting and stopping trips, and sending live GPS location updates during active trips. Merchant functionality includes transport card lookup using QR tokens, selling zone subscriptions, topping up card balances, and viewing transaction history.

The mobile application communicates with the backend through Axios and uses TanStack React Query for server-state management. Expo Location provides GPS functionality for driver tracking, while React Native Maps is used for map visualization. Expo Secure Store is used for secure local storage of authentication-related information and Expo push notification tokens. Real-time tracking and notification updates are received through WebSocket connections provided by the backend.

3. Database Servers:

The Public Transportation Platform uses MySQL as its primary database server. MySQL was selected because the project data is highly structured and relational: users, drivers, vehicles, routes, stops, trips, complaints, favorite trips, and vehicle locations all have clear relationships that can be represented efficiently using relational tables and foreign keys. Django ORM is used as the main data access layer, allowing the backend to define models in Python while Django handles SQL generation and database interaction.

Service / Component	Database / Storage	Access Layer	Main Stored Data
Django Backend	MySQL	Django ORM	Users, drivers, vehicles, routes, stops, trips, complaints, favorites, statistics
Tracking Module	MySQL	Django ORM + API/WebSocket layer	Vehicle locations, driver trip records, tracking timestamps
Admin Dashboard	Backend API	Axios + React Query	Displays and manages backend data through REST endpoints
Mobile Application	Local secure/session storage + Backend API	Expo Secure Store, Async Storage, Axios	Token/session data, mobile state, server data retrieval

3.1 MySQL Database:

MySQL stores the structured operational data of the platform. The most important entities include User, Driver, Vehicle, Route, Stop, RouteStop, DriverTrip, FavoriteTrip, and Complaint. These tables support the main business logic of the platform: passenger route discovery, admin management of routes and stops, driver trip tracking, and complaint handling. Relational integrity is maintained through primary keys, foreign keys, and Django model constraints.

3.2 Database Management Tools:

MySQL Workbench was used during development to inspect tables, verify migrations, review inserted data, and troubleshoot database issues. Django migrations were used to create and update the schema during development. This workflow allowed the team to maintain consistency between Django models and the actual MySQL database schema.

4. Libraries and Dependencies:

Each subsystem uses a different dependency set based on its responsibility. The backend dependencies are listed in requirements.txt, while the admin dashboard and mobile application dependencies are managed through package.json files. The following sections summarize the most important dependencies rather than listing every package, focusing on the libraries that directly support the core features of the Public Transportation Platform.

4.1 Backend Dependencies (requirements.txt):

The backend requirements file contains the core dependencies required to run the Django API server, REST framework, MySQL integration, and WebSocket communication layer.

Library	Version	Role
Django	4.2.7	Core backend web framework
djangorestframework	3.14.0	REST API serialization, views, and response handling
channels	4.1.0	WebSocket support for real-time tracking
daphne	4.1.2	ASGI server for Django Channels and WebSocket communication
mysqlclient	2.2.4	MySQL database adapter for Django ORM

4.2 Admin Dashboard Dependencies:

The admin dashboard uses React, TypeScript, and Vite as the foundation. The most important libraries include Axios for API communication, TanStack React Query for server-state management, TanStack Router for routing, TanStack Table for tabular data, Formik and Yup for forms and validation, Recharts for statistics visualization, @react-google-maps/api for map integration, and i18next for internationalization support.

4.3 Mobile Application Dependencies:

The mobile application uses Expo, React Native, TypeScript, and Expo Router. The most important mobile libraries include Axios and React Query for backend communication, Expo Location for GPS access, Expo Notifications for push notifications, React Native Maps and React Native Maps Directions for map and route visualization, Expo Secure Store and Async Storage for local storage, Zustand for state management, NativeWind for styling, and Formik/Yup for form validation.

Component	Main Libraries	Purpose
Backend	Django, DRF, channels, daphne, mysqlclient	REST APIs, authentication, MySQL integration, WebSocket tracking
Admin Dashboard	React, Vite, TypeScript, Axios, React Query, TanStack Router/Table	Admin interface, API communication, routing, server-state handling, data tables
Admin Dashboard	@react-google-maps/api, Recharts, Formik, Yup, i18next	Map display, statistics charts, forms, validation, multilingual support
Mobile Application	Expo, React Native, Expo Router, Axios, React Query	Cross-platform mobile application and API communication
Mobile Application	Expo Location, Expo Notifications, React Native Maps	GPS tracking, push notifications, and map-based interaction
Mobile Application	Expo Secure Store, Async Storage, Zustand, NativeWind	Secure token storage, local persistence, state management, styling

5. Testing Environment:

Testing was conducted throughout the development lifecycle to validate API behavior, database operations, frontend integration, mobile workflows, and real-time tracking. Because the system consists of multiple clients communicating with a backend server, testing focused on both individual components and end-to-end integration scenarios.

5.1 Local Development Testing:

Local testing was performed on Windows machines using the Django development server, Vite development server, and Expo Go. Developers verified that backend endpoints responded correctly, dashboard screens displayed expected data, and mobile screens communicated properly with the backend APIs. Local testing was especially important before connecting features across the three subsystems.

5.2 API Testing with Postman:

Postman was used as the primary API testing tool. The team tested authentication endpoints, route and stop management endpoints, driver and vehicle APIs, complaint APIs, favorite trip APIs, and tracking-related endpoints. Token Authentication was validated by sending requests with the appropriate Authorization header. Postman helped detect request/response errors before frontend integration.

5.3 Mobile Testing with Expo Go:

Expo Go was used to test the passenger and driver mobile application on real devices. This was important for testing location-based features, map rendering, navigation flows, and push notification behavior. Driver tracking scenarios were tested by sending location updates from the mobile application and observing whether the backend and user-facing interfaces reflected the updates correctly.

5.4 Database Testing with MySQL Workbench:

MySQL Workbench was used to verify database records generated during testing. The team inspected tables related to users, drivers, vehicles, routes, stops, trips, and locations to ensure that backend operations correctly persisted data. This was particularly useful when debugging route definitions, ordered stops, driver trips, and location history.

5.5 Real-Time Tracking Testing:

Real-time tracking testing focused on verifying WebSocket connectivity, driver location transmission, and client-side update behavior. The team validated that active driver trips could send location information and that tracking data could be delivered to the relevant client views.

6. Software implementation

The system is built using a well-structured and organized architecture following the principles of Layered Architecture and Feature-Based Structure. This design approach aims to improve maintainability, scalability, and separation of concerns across the system components. The project consists of three main applications: the **Backend system**, the **Mobile application**, and the **Admin dashboard**, each designed as an independent module while remaining fully integrated through APIs.

6. Software implementation

The Public Transportation Platform is implemented using a modular architecture that separates the main system components according to their responsibilities. The platform consists of a Django-based backend, a React Native and Expo mobile application, a React and TypeScript administrator dashboard, and a standalone Python-based AI passenger-counting service.

The backend acts as the central communication and business-logic layer. It provides RESTful APIs for the mobile application and administrator dashboard, manages the MySQL database through Django ORM, handles authentication and business operations, and provides WebSocket communication for real-time tracking and notifications.

The mobile application provides role-based functionality for passengers, drivers, and merchants. The administrator dashboard provides centralized management and monitoring functionality, while the AI service processes camera input to estimate passenger occupancy and communicates the resulting passenger count to the backend.

The components remain logically separated while being integrated through REST APIs, WebSocket connections, and dedicated service interfaces. This structure improves separation of concerns and allows each component to be developed and maintained according to its specific responsibilities.

The backend itself follows a layered organization in which models represent the database entities, serializers handle data transformation and validation, views expose API endpoints, and service modules contain business logic for specific platform operations. This organization improves code maintainability and provides a clear separation between data models, API handling, and business operations.

Backend Architecture Layers

The backend system follows a structured layered organization that separates API handling, business logic, data representation, and database operations. The main layers and components are organized as follows:

- **Presentation / API Layer:**
  Responsible for receiving and processing HTTP requests from the Mobile Application and Administrator Dashboard. Django REST Framework views and URL configurations expose the required API endpoints and route incoming requests to the appropriate application logic.
- **Service Layer:**
  Contains the main business logic of the platform. Service modules are responsible for operations such as authentication, driver trips, vehicle tracking, route deviation detection, notifications, payments, subscriptions, and other transportation-related operations. Separating business logic into services helps keep API views focused on request handling.

- **Serialization Layer:**
  Django REST Framework serializers are responsible for validating incoming data and converting Django model instances into structured JSON responses. They also define how request data is transformed into data that can be processed by the backend.
- **Data and Model Layer:**
  Django models define the entities and relationships used by the system and provide the main interface for database operations through Django ORM. The model layer represents entities such as Users, Drivers, Merchants, Vehicles, Routes, Stops, Driver Trips, Vehicle Locations, Transport Cards, Subscriptions, Transactions, Notifications, and Complaints.
- **Real-Time Communication Layer:**
  Django Channels and WebSocket consumers provide real-time communication between the backend and connected clients. This layer is responsible for features such as live vehicle tracking, passenger trip tracking, and real-time notifications.
- **Authentication Layer:**
  The backend contains dedicated authentication mechanisms according to the type of client. Passenger, merchant, and administrator accounts use token-based authentication, while drivers and scanner devices use dedicated authentication mechanisms.

This separation improves code organization, reduces coupling between components, and enhances system scalability and maintainability.

Admin Dashboard Structure

The Admin Dashboard is developed using **React with TypeScript**, following a **Feature-Based Architecture**. Each feature is grouped with its related files such as API calls, UI components, validation rules, and interfaces.

The structure is organized into:

- Shared layout components (Header, Sidebar, Layout)
- Reusable components (Forms, Tables, Modals, Loaders)

- Feature modules including:
    - User Management
    - Zone Management
    - Merchant Management
    - Driver Management
    - Bus Routes Management
    - Bus Stops Management
    - Complaints Handling
    - Notifications
    - Dashboard Analytics

This modular structure allows for easy scalability, as new features can be added without affecting existing functionality.

Mobile Application Structure

The Mobile Application is designed with a focus on user experience and performance. It also follows a modular approach where each feature is isolated into its own unit.

Main modules include:

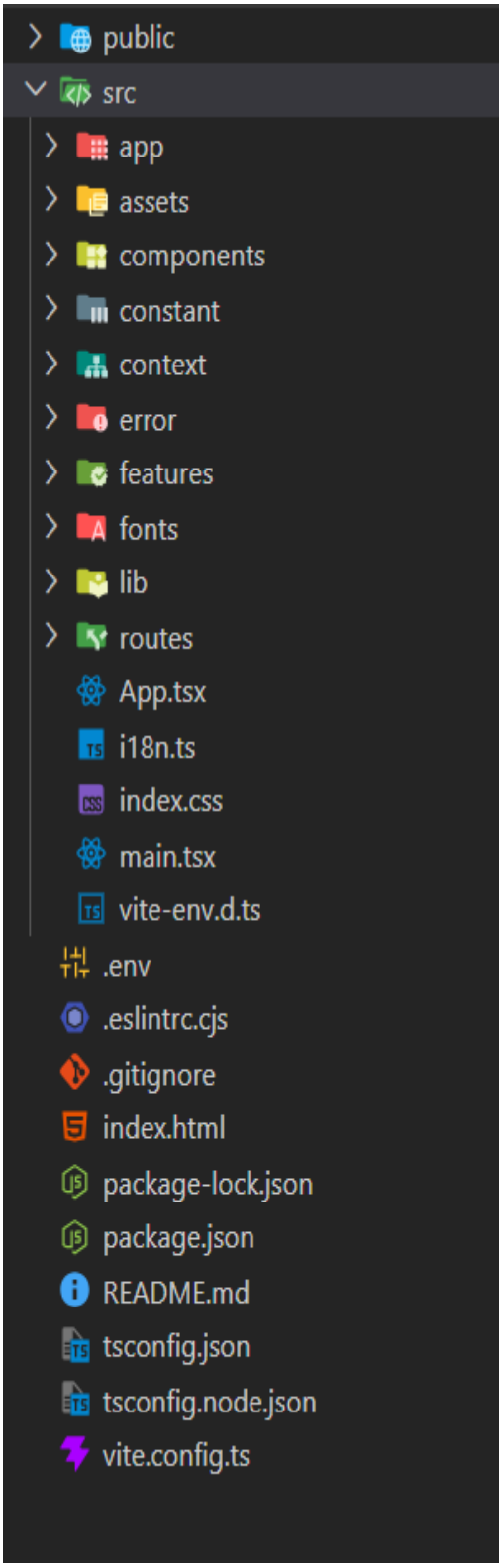
- Authentication and user login
- Bus routes and booking features
- Live bus tracking
- Notifications system
- Complaints and support system

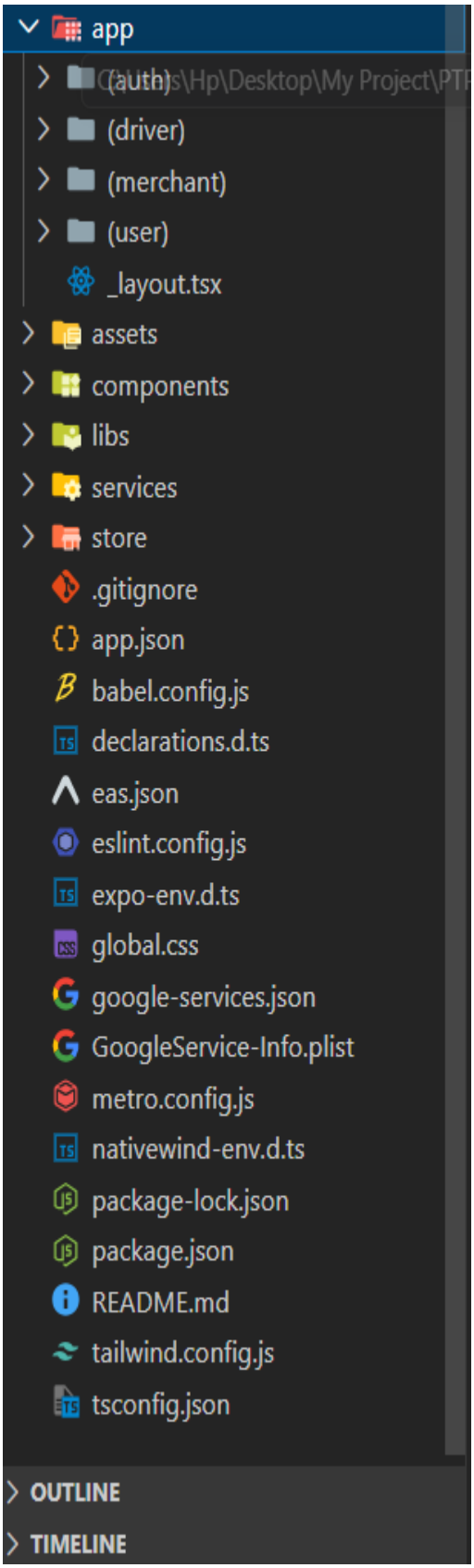
The application is structured into:

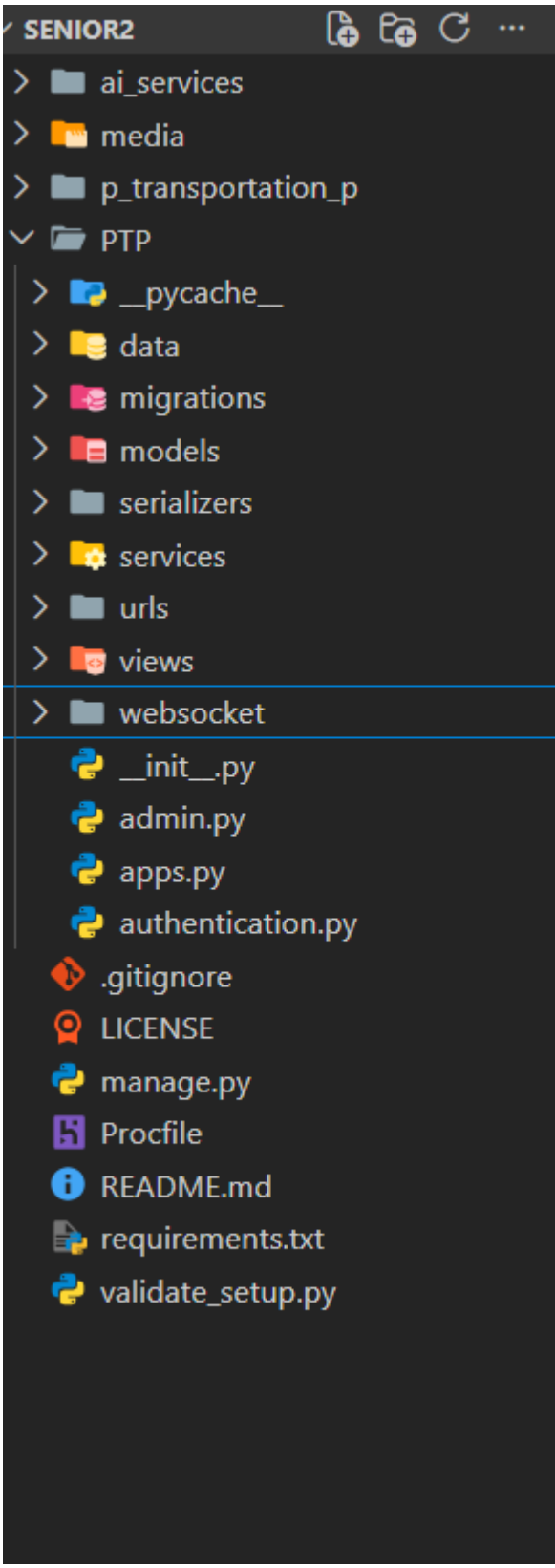
- UI Screens
- API Services
- State Management

This ensures a clean separation between UI logic and business logic, improving maintainability and performance.

System Structure Diagram:


public
src
app
assets
components
constant
context
error
features
fonts
lib
routes
App.tsx
i18n.ts
index.css
main.tsx
vite-env.d.ts
.env
.eslintrc.cjs
.gitignore
index.html
package-lock.json
package.json
README.md
tsconfig.json
tsconfig.node.json
vite.config.ts


app
(auth)
(driver)
(merchant)
(user)
_layout.tsx
assets
components
libs
services
store
.gitignore
app.json
babel.config.js
declarations.d.ts
eas.json
eslint.config.js
expo-env.d.ts
global.css
google-services.json
GoogleService-Info.plist
metro.config.js
nativewind-env.d.ts
package-lock.json
package.json
README.md
tailwind.config.js
tsconfig.json
OUTLINE
TIMELINE


SENIOR2
ai_services
media
p_transportation_p
PTP
__pycache__
data
migrations
models
serializers
services
urls
views
websocket
__init__.py
admin.py
apps.py
authentication.py
.gitignore
LICENSE
manage.py
Procfile
README.md
requirements.txt
validate_setup.py

The following diagram illustrates the overall project structure, showing the hierarchical organization of files and modules across the Backend, Admin Dashboard, and Mobile Application.

The system was implemented using a modular approach, where the system is divided into independent functional modules that work together to provide the full system functionality. The backend follows a **Layered Architecture**, while both the Admin Dashboard and Mobile Application follow a **Feature-Based Modular Structure**. This approach improves maintainability, scalability, and separation of concerns across the system.

1. Authentication Module

The Authentication module is responsible for handling user registration and login processes.

Backend Implementation:

User registration and login are implemented by validating user credentials such as email and password.
During account creation, the user type is directly assigned as either **Driver or Passenger**, and this role is stored as part of the user data without implementing a complex role-based access control system.

Admin & Mobile Implementation:

Users can register and log in through the interface, where the data is sent to the backend API for validation. Upon successful authentication, user data is stored and used for accessing system features.

2. User Management Module

This module is responsible for managing system users, including drivers and passengers.

Backend:

The backend handles basic CRUD operations for user data, including creating, retrieving, and updating user information when required.

Admin Dashboard:

The admin interface allows administrators to view all users and display their details based on their type (Driver or Passenger).

3. Drivers Management Module

This module manages driver registration requests and their approval process.

Backend:

Driver data is stored in the system, and the backend handles the process of accepting or rejecting driver registration requests.

Admin Dashboard:

The admin panel allows administrators to review driver applications and make decisions to either approve or reject them, which determines the driver's activation status in the system.

4. Vehicles Module

This module is responsible for managing vehicle information in the system.

Backend:

Vehicle data such as license plate number is stored and linked to drivers.

Admin Dashboard:

Administrators can add vehicles and assign them to drivers for management purposes.

5. Complaints Module

This module handles user-submitted complaints.

Backend:

Complaints are stored in the database without multiple processing states. They are recorded and later reviewed by administrators.

Admin Dashboard:

The admin panel allows administrators to view complaints

6. Notification Module

This module manages system notifications.

Backend:

Notifications are triggered only in specific events, such as:

- Approval of a driver registration request
- Rejection of a driver registration request

Mobile Application:

Drivers receive real-time notifications when their registration request status changes (approved or rejected).

7. Real-Time Bus Tracking Module (WebSocket)

This is one of the key modules of the system, responsible for real-time tracking of buses.

Backend:

The backend uses **WebSocket communication** to continuously broadcast the live location of buses. The driver's GPS coordinates are sent to the server in real time and then distributed to connected clients.

Mobile Application (Passenger):

Passengers can track buses live on the map. The application receives real-time updates via WebSocket and displays the current bus location instantly.

Functionality:

This module enables passengers to:

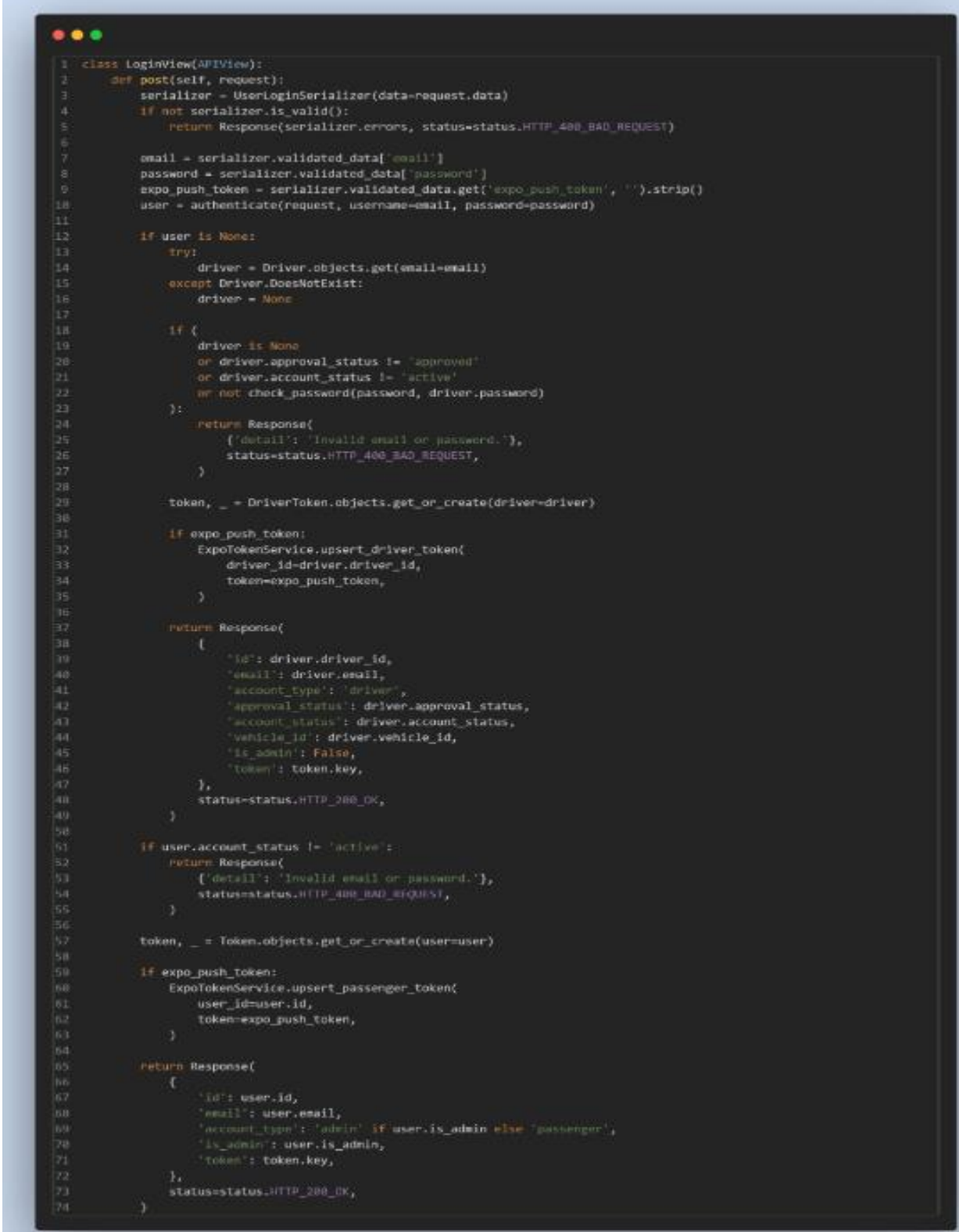
- View live bus movement on the map
- Track the nearest or selected bus in real time
- Receive continuous location updates without refreshing the application

This significantly improves user experience by providing accurate and up-to-date transportation tracking.

- The APIs in this system were developed using RESTful architecture, which relies on HTTP protocols to enable communication between the backend and both the Mobile Application and Admin Dashboard. These APIs handle incoming requests from the frontend applications, process them in the backend server, and return responses in JSON format.

The APIs act as the core communication layer between all system components, enabling secure and efficient execution of operations such as user authentication, data management, and real-time processing. The design follows a separation of concerns approach, where business logic is isolated from the user interface to improve maintainability and scalability.

The Login API is responsible for authenticating users, including drivers, passengers, and administrators.



```python
class LoginView(APIView):
    def post(self, request):
        serializer = UserLoginSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

        email = serializer.validated_data['email']
        password = serializer.validated_data['password']
        expo_push_token = serializer.validated_data.get('expo_push_token', '').strip()
        user = authenticate(request, username=email, password=password)

        if user is None:
            try:
                driver = Driver.objects.get(email=email)
            except Driver.DoesNotExist:
                driver = None

            if (
                driver is None
                or driver.approval_status != 'approved'
                or driver.account_status != 'active'
                or not check_password(password, driver.password)
            ):
                return Response(
                    {'detail': 'Invalid email or password.'},
                    status=status.HTTP_400_BAD_REQUEST,
                )

            token, _ = DriverToken.objects.get_or_create(driver=driver)

            if expo_push_token:
                ExpoTokenService.upsert_driver_token(
                    driver_id=driver.driver_id,
                    token=expo_push_token,
                )

            return Response(
                {
                    'id': driver.driver_id,
                    'email': driver.email,
                    'account_type': 'driver',
                    'approval_status': driver.approval_status,
                    'account_status': driver.account_status,
                    'vehicle_id': driver.vehicle_id,
                    'is_admin': False,
                    'token': token.key,
                },
                status=status.HTTP_200_OK,
            )

        if user.account_status != 'active':
            return Response(
                {'detail': 'Invalid email or password.'},
                status=status.HTTP_400_BAD_REQUEST,
            )

        token, _ = Token.objects.get_or_create(user=user)

        if expo_push_token:
            ExpoTokenService.upsert_passenger_token(
                user_id=user.id,
                token=expo_push_token,
            )

        return Response(
            {
                'id': user.id,
                'email': user.email,
                'account_type': 'admin' if user.is_admin else 'passenger',
                'is_admin': user.is_admin,
                'token': token.key,
            },
            status=status.HTTP_200_OK,
        )
```

Request Body

```
{
"email": "user@example.com",
"password": "123456",
"expo_push_token": "optional_device_token"
}
```

Success Response (Driver)

```
{
  "id": 5,
  "email": "driver@example.com",
  "account_type": "driver",
  "approval_status": "approved",
  "account_status": "active",
  "vehicle_id": 12,
  "is_admin": false,
  "token": "TOKEN_VALUE"
}
```

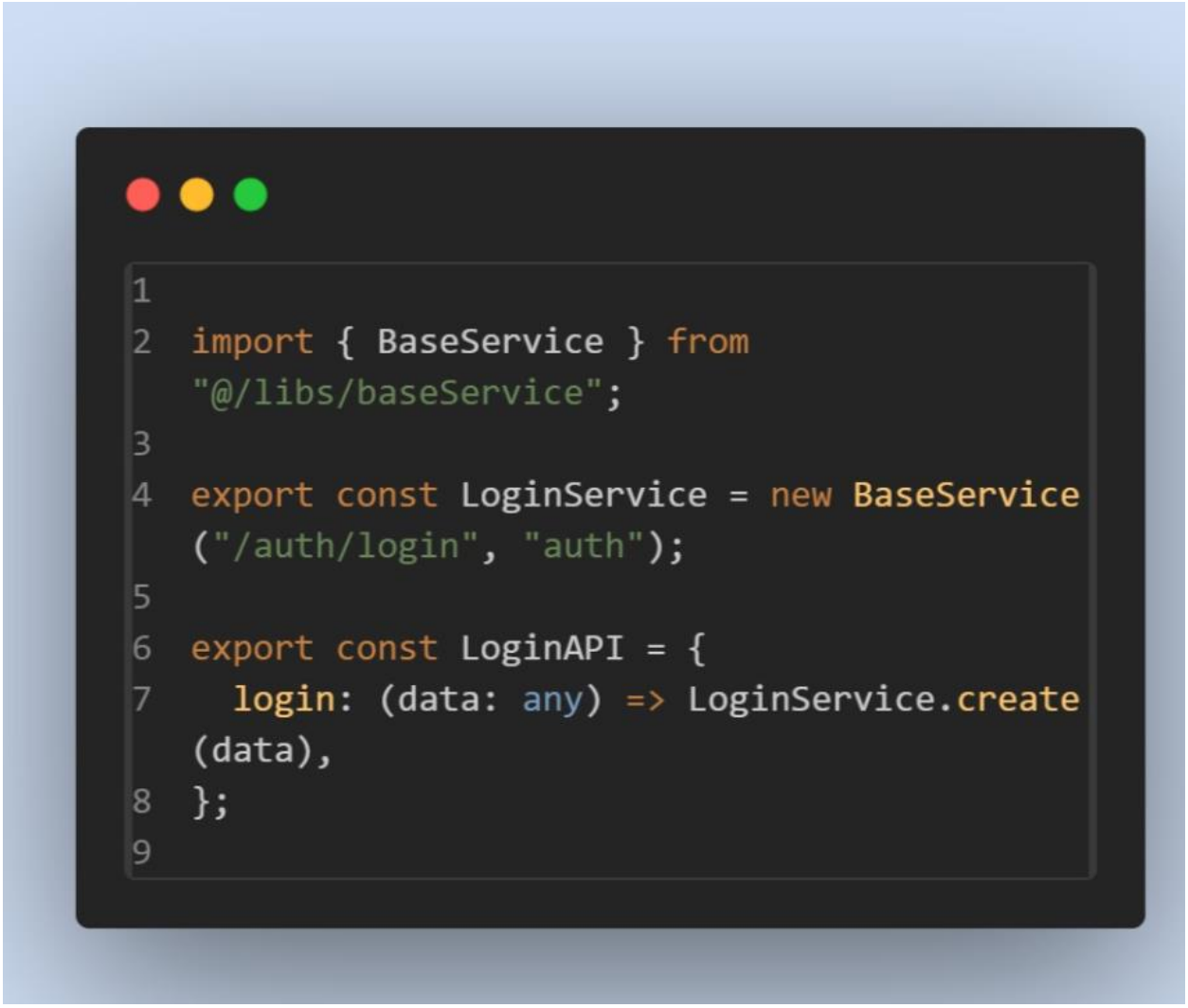
Success Response (Passenger / Admin)

```
{
  "id": 1,
  "email": "user@example.com",
```

```
  "account_type": "admin",
  "is_admin": true,
  "token": "TOKEN_VALUE"
}
```


Error Response


```
{
  "detail": "Invalid email or password."
}
```



```
1
2  import { BaseService } from
   "@/libs/baseService";
3
4  export const LoginService = new BaseService
   ("/auth/login", "auth");
5
6  export const LoginAPI = {
7    login: (data: any) => LoginService.create
   (data),
8  };
9
```

```
onst navigate = useNavigate();

  const handleLogin = async (values: any
) => {
    const response: any = await AuthAPI.
login(values);
    if (response) {
      const { token, id } = response;
      tokenStorage.setAccessToken(token);
      const userData = { id: id };
      const encryptedData = CryptoJS.AES.
encrypt(
        JSON.stringify(userData),
        import.meta.env
.VITE_SECRET_KEY_FOR_DATA
      ).toString();
      sessionStorage.setItem("userData"
, encryptedData);

      navigate({ to: "/dashboard" });
    }
  };

  const fields = [
    [
      {
        name: "email",
        label: "Administrator Email",
        type: "email",
        placeholder:
"admin@damascus-transport.sy",
        wrapperClass: "col-span-2",
      },
      {
        name: "password",
        label: "Access Password",
        type: "password",
        placeholder: "••••••••",
        wrapperClass: "col-span-2",
      },
    ],
  ];
```

```

  const loginFields = useMemo(() => [
    { name: "email", type: "email",
placeholder: "Email", label: "Email" },
    { name: "password", type: "password"
, placeholder: "Password", label:
"Password" }
  ], []);
const handleAuth = async (values: any) =>
 {
  const token = await SecureStore.
getItemAsync('expo_push_token');
  const signupPayload = {
    ...values,
    expo_push_token:token,
    account_type: signupRole,
  };
  const loginPayload = {
    ...values,
    expo_push_token:token,
  };
  const res =
    authMode === 'login'
      ? await LoginAPI.login
(loginPayload)
      : await AuthAPI.signup
(signupPayload);

  await setAuth(res);
  closeModal();
  };

```

Business Logic Layer Implementation

The Business Logic Layer represents the core functionality of the system, where all rules, conditions, and decision-making processes are implemented. It acts as an intermediary layer between the Data Access Layer and the API Layer, ensuring that all system operations follow defined business rules before interacting with the database or being exposed to the frontend applications.

In this system, the business logic is implemented within the backend views and supporting services. It is responsible for controlling how data is processed, validated, and transformed according to system requirements.

1. Authentication Logic

The authentication process includes verifying user credentials and determining the type of user (Driver, Passenger, or Admin). The system also checks the account status before allowing access. Drivers are only allowed to log in if their account is approved and active. Upon successful authentication, a token is generated for secure communication between the client and server.

2. Driver Approval Logic

The system enforces a business rule where drivers cannot access the system until their accounts are approved by the administrator. This ensures that only verified drivers are allowed to operate within the system.

3. Notification Logic

Notifications are triggered based on specific system events. For example, when a driver account is approved or rejected by the administrator, the system automatically sends a notification to inform the user about the decision.

4. Real-Time Tracking Logic (WebSocket)

The system implements real-time bus tracking using WebSocket communication. The driver continuously sends GPS location updates to the server, which are then broadcasted to passengers. This allows users to track buses live on the map in real time.

5. Device Token Management Logic

During login, the system stores the Expo push token associated with the user device. This token is used later to send push notifications to the correct device when system events occur

7. Testing and Evaluation

7.1 Test Plan

The test plan was prepared to verify that the Public Transportation Platform operates according to the specified requirements and that all core functionalities perform correctly. The testing process focuses on validating user registration and login, trip planning, favorite trip management, user management, route management, vehicle management, driver ride management, notification services, and reporting functionalities.

Testing was conducted using the system development and deployment environment. Manual testing techniques were used to verify system functionality and user interface behavior. In addition, database validation was performed to ensure the correct storage, retrieval, and updating of data during different system operations

Test Area	Testing Method	Tool
User Registration	Functional Testing	Manual Testing
User Login	Functional Testing	Manual Testing
Trip Planning	Functional Testing	Manual Testing
Favorite Trips	Functional Testing	Manual Testing
Route Management	Functional Testing	Manual Testing
Vehicle Management	Functional Testing	Manual Testing
Notifications	Functional Testing	Manual Testing
Dashboard & Reports	Functional Testing	Manual Testing
Database Operations	Data Validation	MySQl
API Services	API Testing	Postman

7.2 Types of Testing

- Unit Testing

Unit testing was performed to verify the functionality of individual system components independently. Each module, such as user authentication, route management, vehicle management, and trip planning, was tested separately to ensure correct operation.

- Integration Testing

Integration testing was conducted to verify the interaction between different modules of the system. For example, the integration between trip planning and route management, as well as the interaction between bus tracking and vehicle management, was tested to ensure proper data flow.

- System Testing

System testing was performed on the complete Public Transportation Platform to ensure that all functionalities work together correctly and satisfy the specified system requirements.

- Acceptance Testing

Acceptance testing was carried out to verify that the system meets stakeholder requirements and provides the expected transportation services from the perspective of end users and administrators.

- Performance and Load Testing

Performance testing was conducted to evaluate system responsiveness under normal operating conditions. Load testing was used to assess the platform's ability to handle multiple users accessing services simultaneously.

- Usability Testing

Usability testing was conducted to evaluate the ease of use of the platform. Users were able to navigate between features such as trip planning, route selection, and bus tracking with minimal effort.

- Accessibility Testing

Accessibility testing was performed to ensure that the system interface remains understandable and usable for users with different levels of technical experience and across different device sizes.

- Compatibility Testing

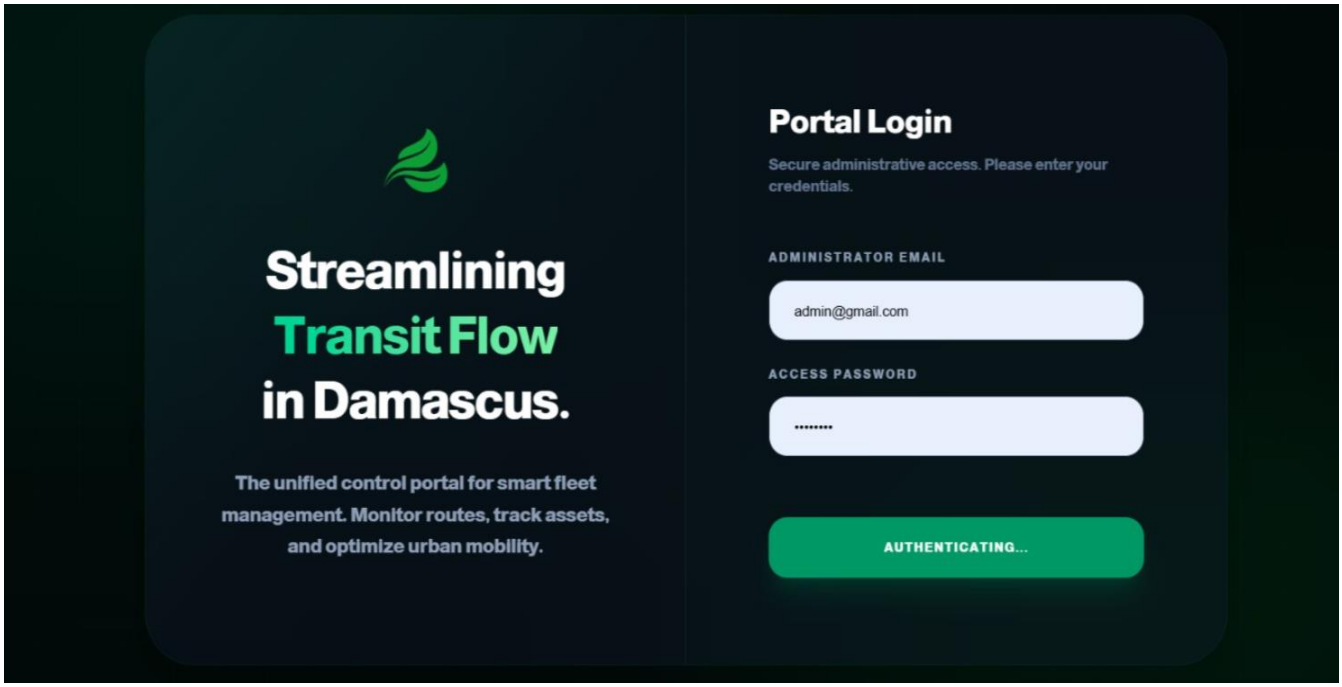
Compatibility testing was conducted to ensure that the platform operates correctly across different web browsers, mobile devices, and operating systems supported by the application.

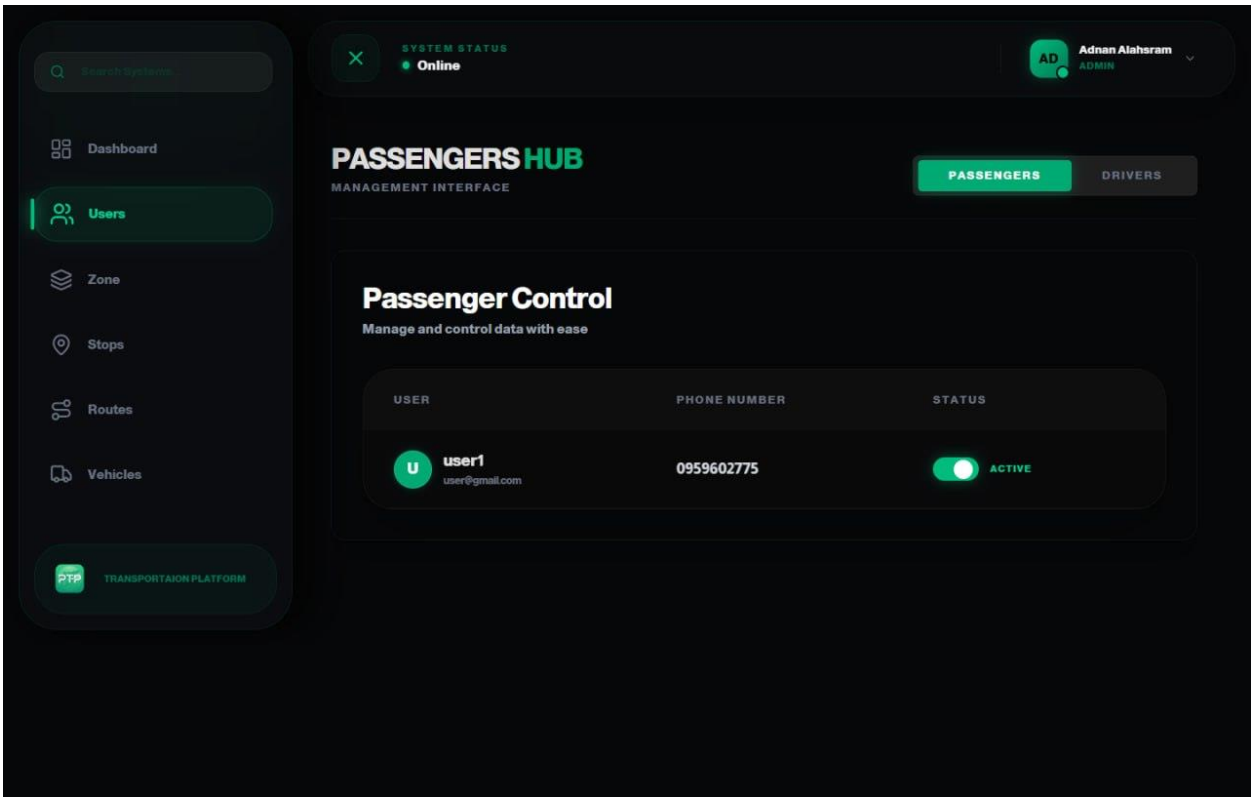
Table 16 Types of testing

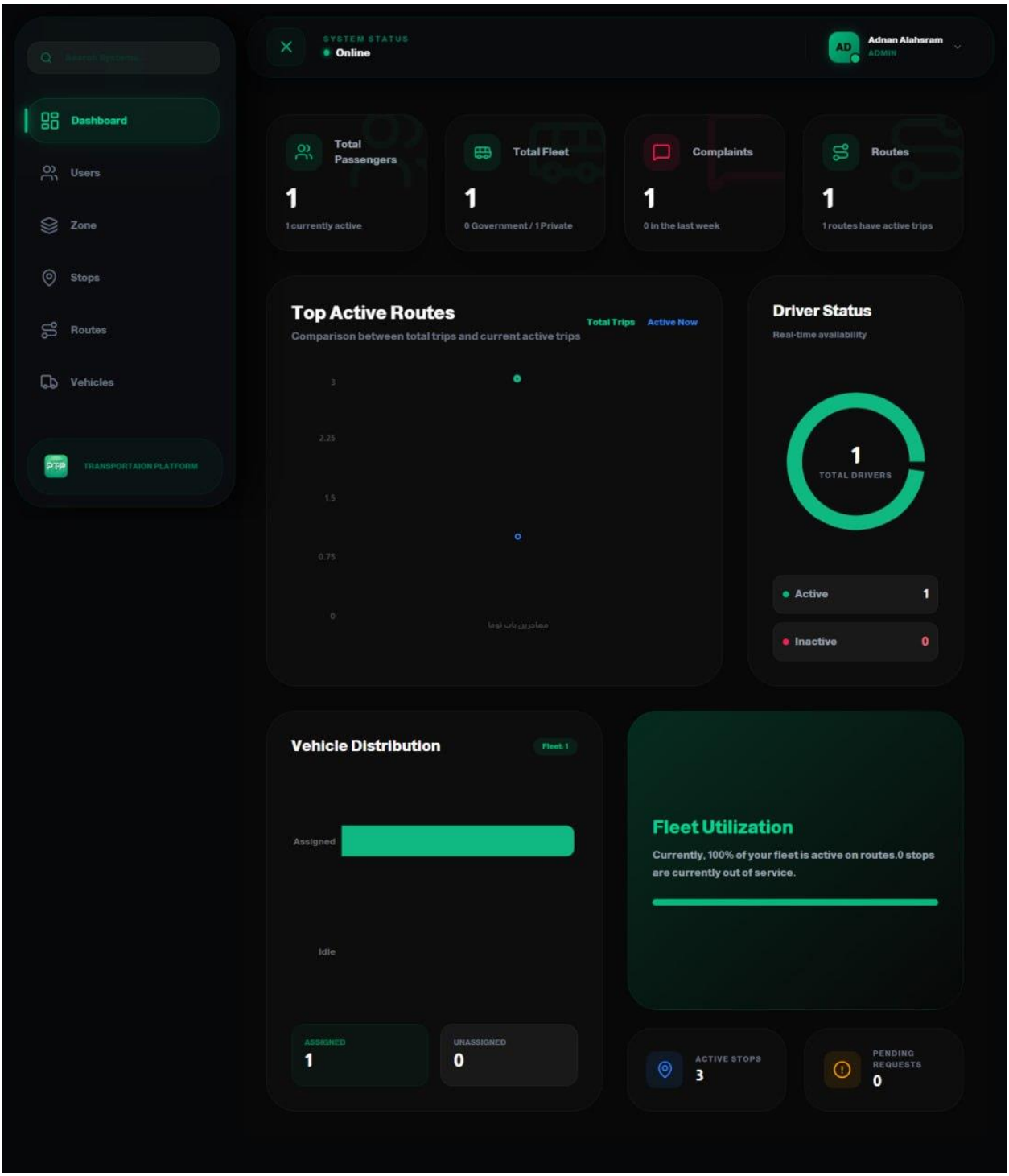
Testing Type	Purpose
Unit Testing	Verify individual modules independently
Integration Testing	Verify interaction between modules
System Testing	Verify the complete system
Acceptance Testing	Validate stakeholder requirements
Performance & Load Testing	Evaluate response time and scalability
Usability Testing	Evaluate ease of use
Accessibility Testing	Ensure accessibility for different users
Compatibility Testing	Verify operation across devices and platforms

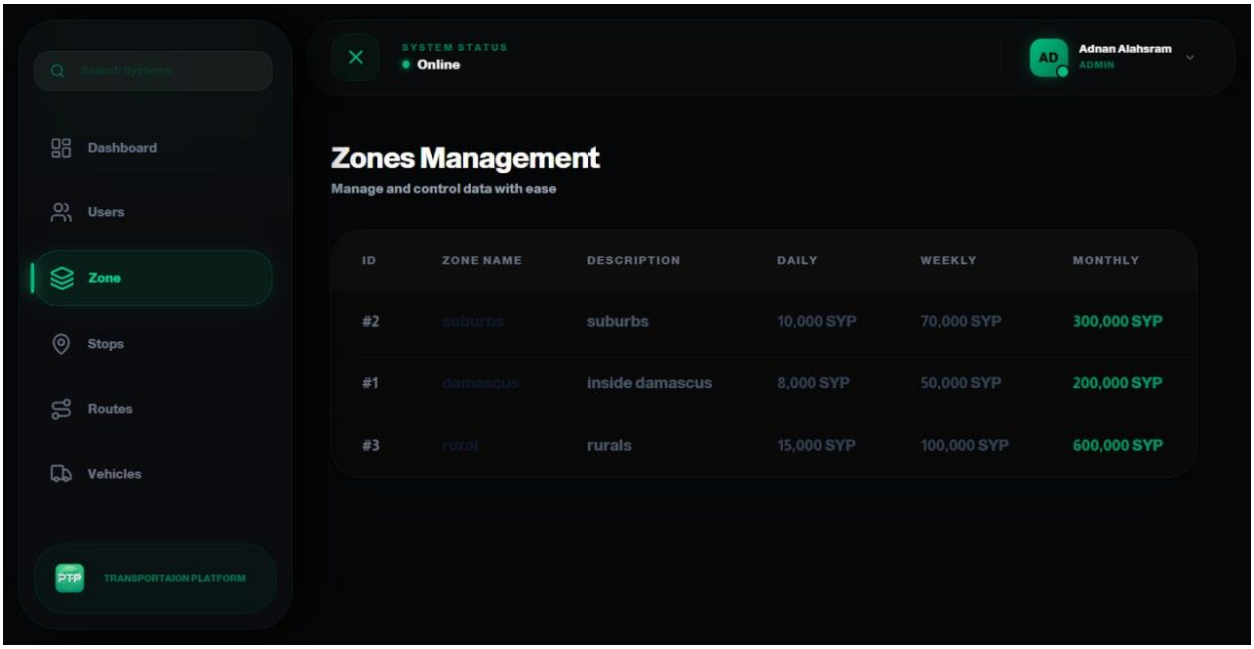
Chapter 5 Results and Recommendations

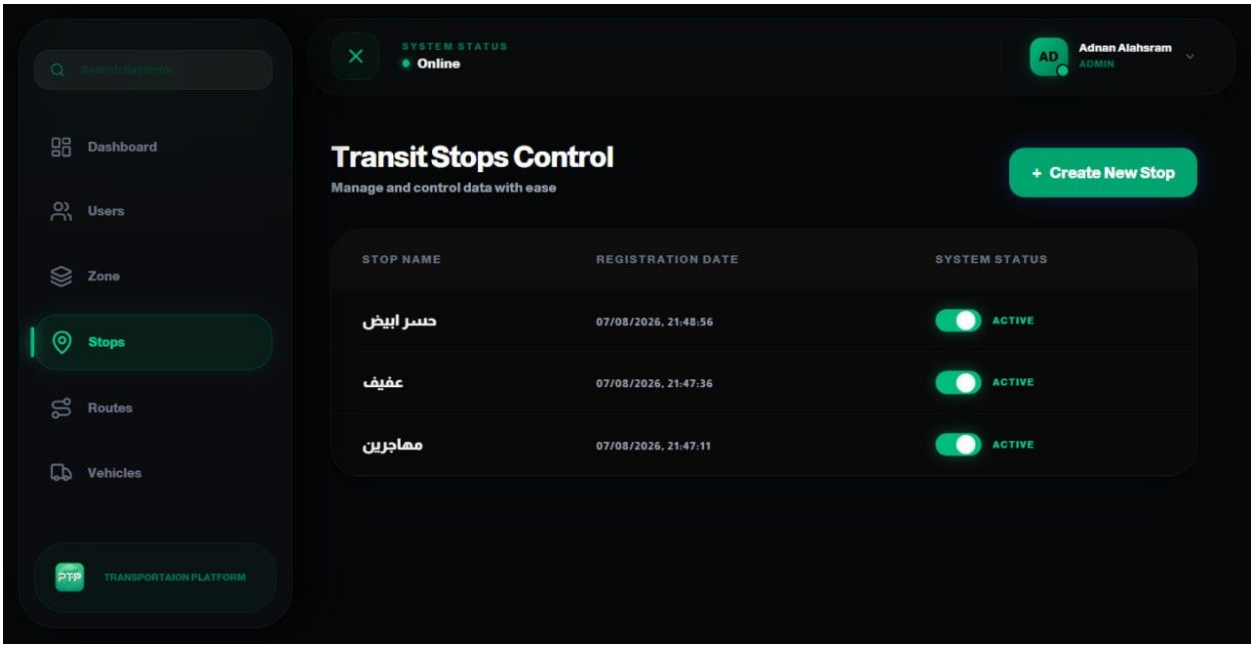
1. Results of implementing the system

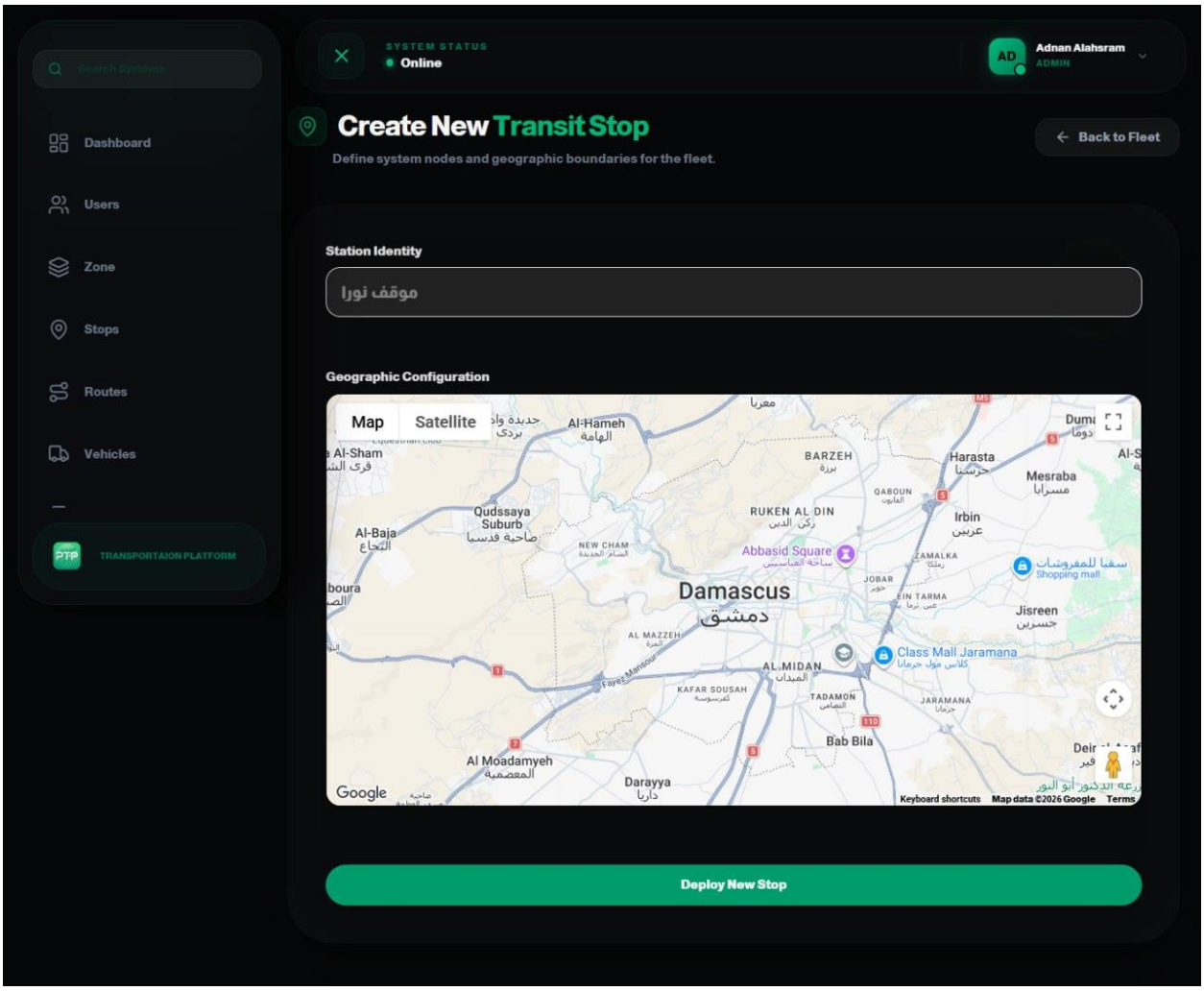


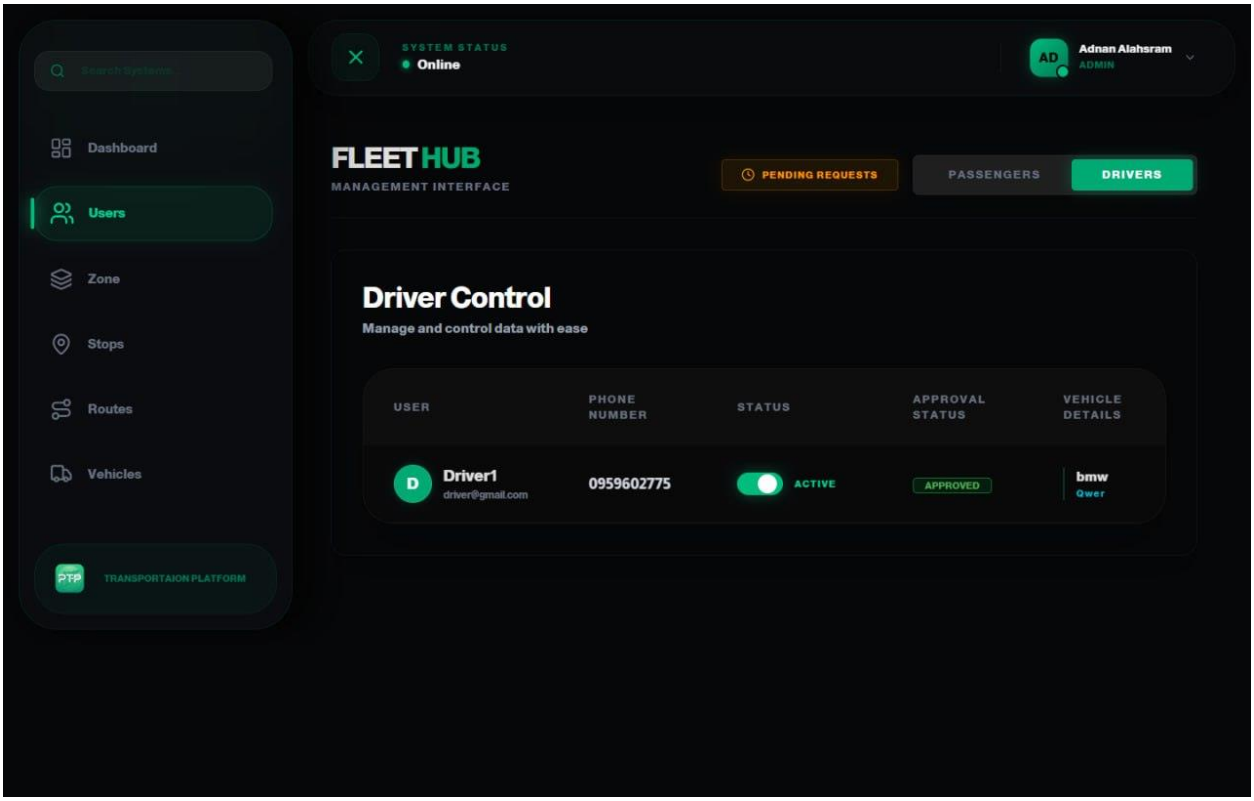


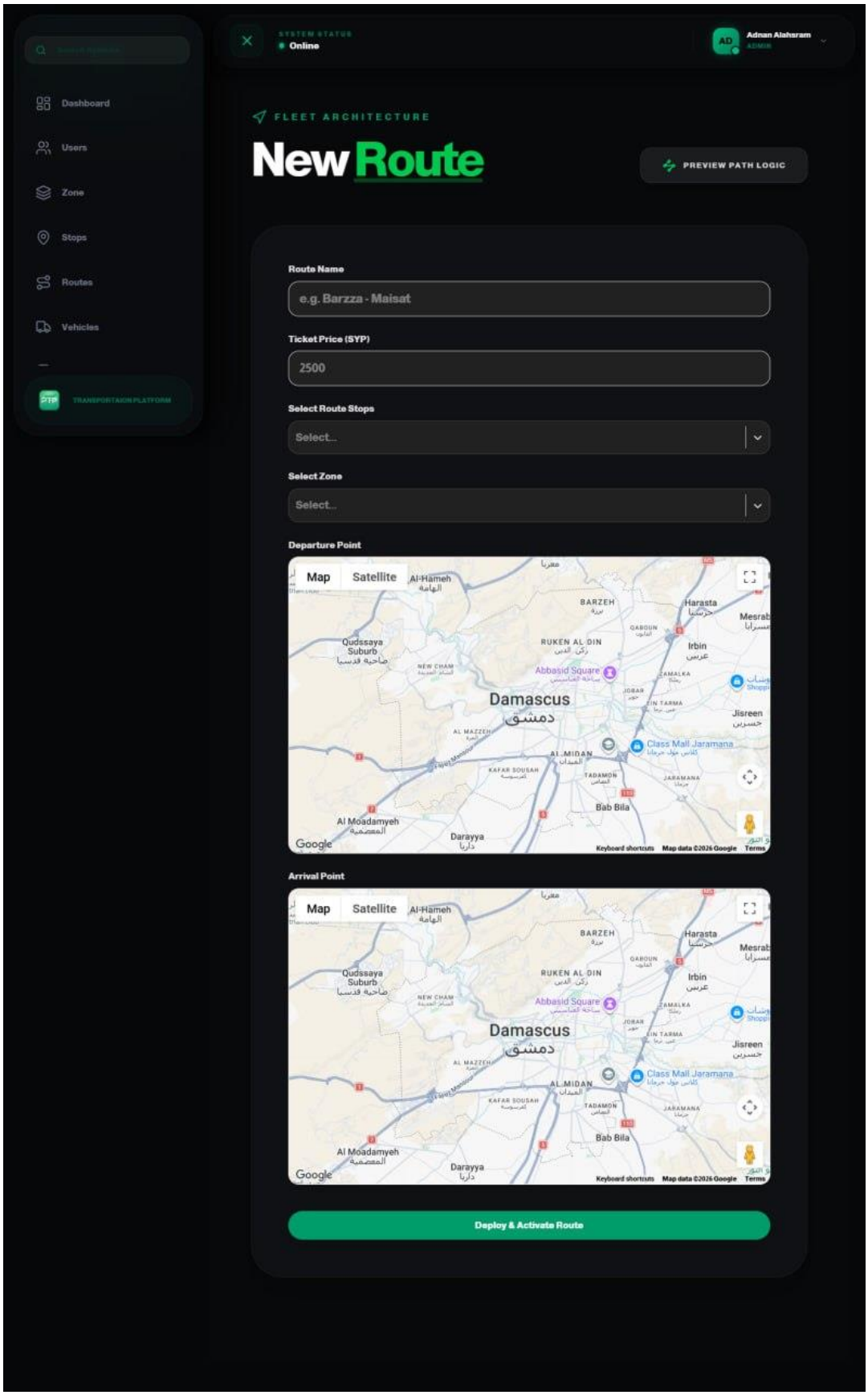

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
Total Passengers
1
1 currently active
Total Fleet
1
0 Government / 1 Private
Complaints
1
0 in the last week
Routes
1
1 routes have active trips
Top Active Routes
Comparison between total trips and current active trips
Total Trips
Active Now
3
2.25
1.5
0.75
0
Driver Status
Real-time availability
1
TOTAL DRIVERS
Active 1
Inactive 0
Vehicle Distribution
Fleet 1
Assigned
Idle
ASSIGNED
1
UNASSIGNED
0
Fleet Utilization
Currently, 100% of your fleet is active on routes.0 stops are currently out of service.
ACTIVE STOPS
3
PENDING REQUESTS
0

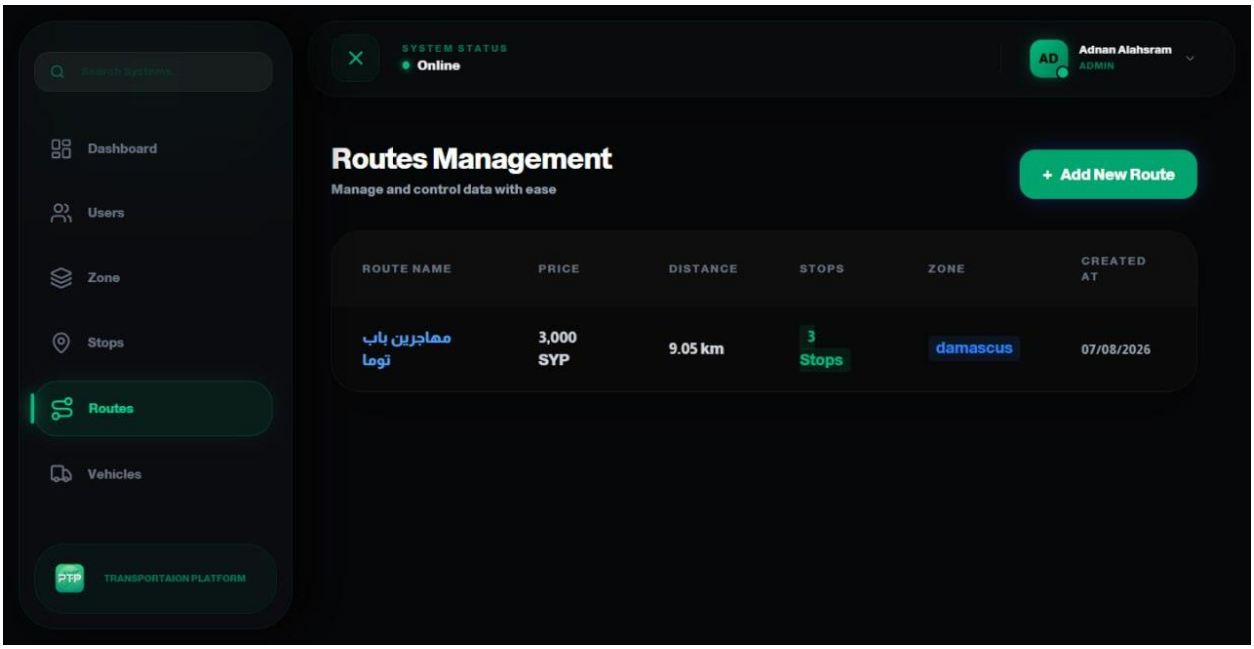

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
Zones Management
Manage and control data with ease
ID
ZONE NAME
DESCRIPTION
DAILY
WEEKLY
MONTHLY
#2
suburbs
suburbs
10,000 SYP
70,000 SYP
300,000 SYP
#1
damascus
inside damascus
8,000 SYP
50,000 SYP
200,000 SYP
#3
rural
rurals
15,000 SYP
100,000 SYP
600,000 SYP

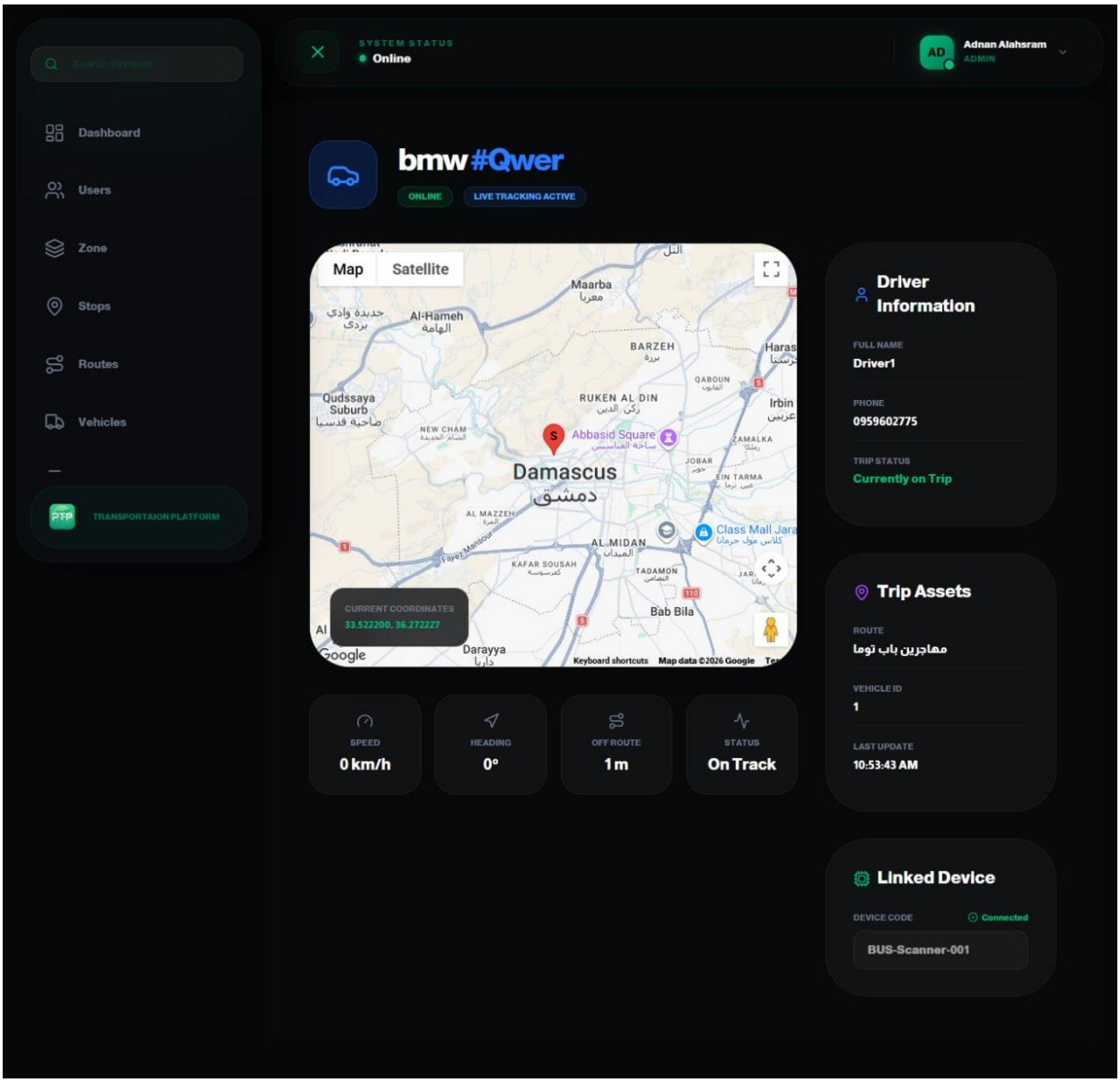

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
Transit Stops Control
Manage and control data with ease
+ Create New Stop
STOP NAME
REGISTRATION DATE
SYSTEM STATUS
حسر ابيض
07/08/2026, 21:48:56
ACTIVE
عفيف
07/08/2026, 21:47:36
ACTIVE
مهاجرين
07/08/2026, 21:47:11
ACTIVE

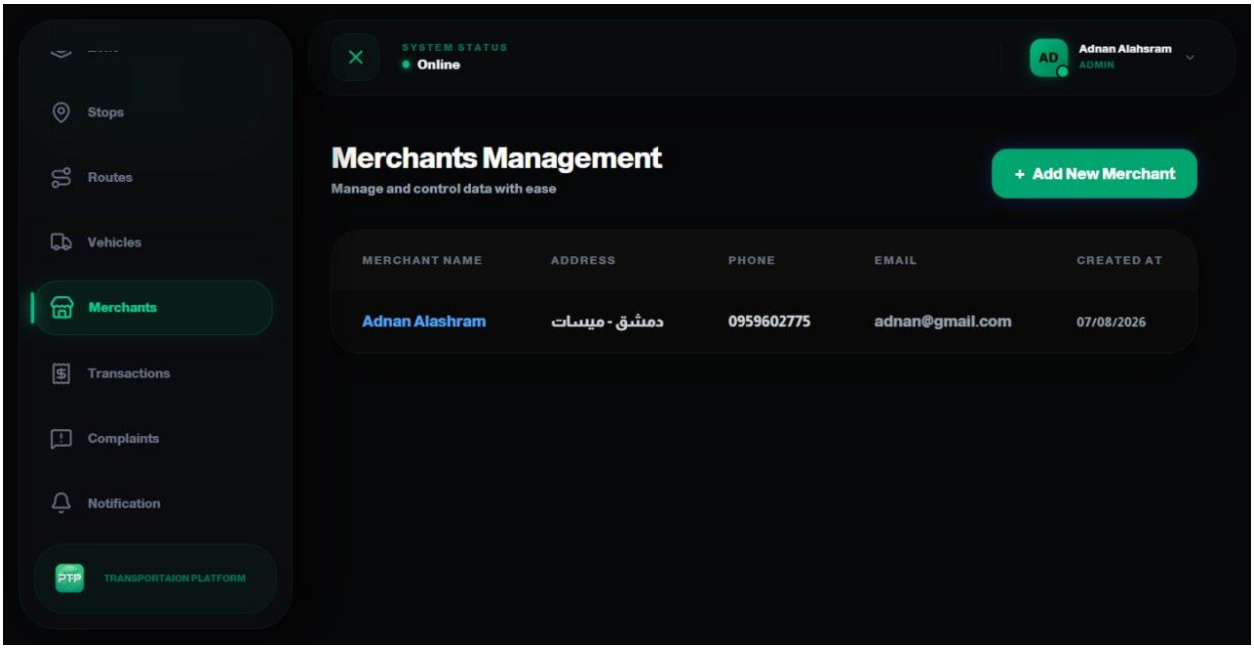

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
Create New Transit Stop
Define system nodes and geographic boundaries for the fleet.
Back to Fleet
Station Identity
موقف نورا
Geographic Configuration
Map
Satellite
Damascus
دمشق
Abbasid Square
Class Mall Jaramana
Shopping mall
Jisreen
Harasta
Mesraba
Irbin
Qudssaya Suburb
Al-Baja
Al Moadamyeh
Darayya
Bab Bila
Google
Keyboard shortcuts
Map data ©2026 Google
Terms
Deploy New Stop

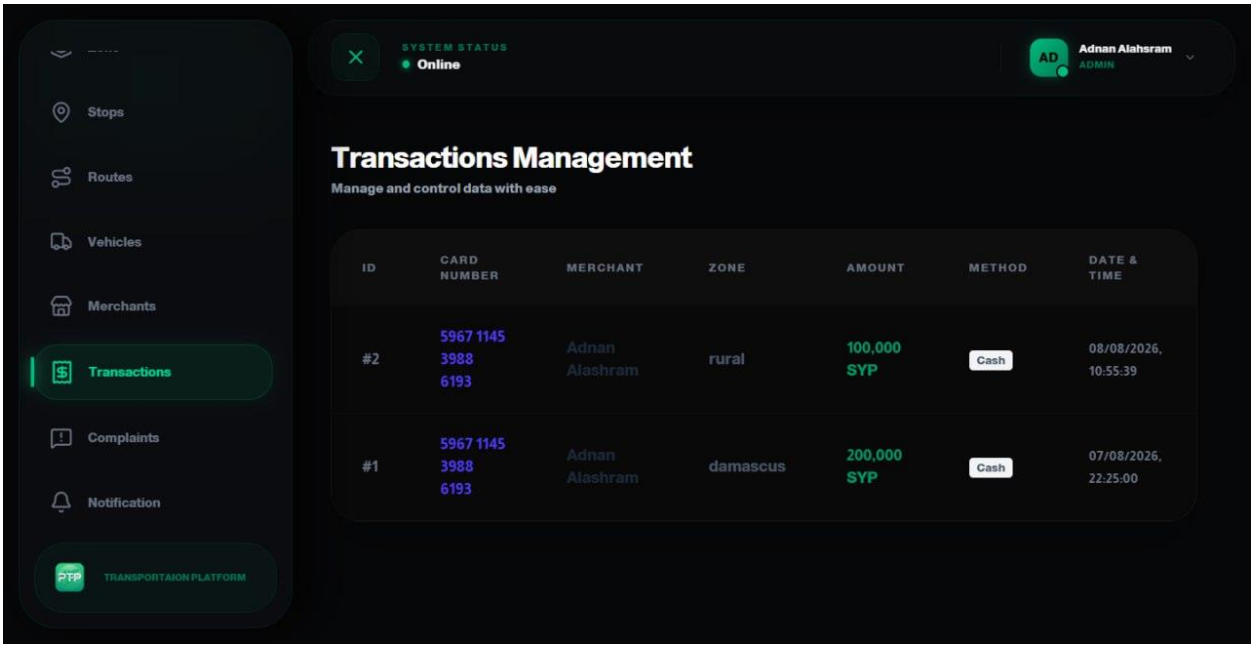

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Search Systems...
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
FLEET HUB
MANAGEMENT INTERFACE
PENDING REQUESTS
PASSENGERS
DRIVERS
Driver Control
Manage and control data with ease
USER
PHONE NUMBER
STATUS
APPROVAL STATUS
VEHICLE DETAILS
Driver1
driver@gmail.com
0959602775
ACTIVE
APPROVED
bmw
Qwer

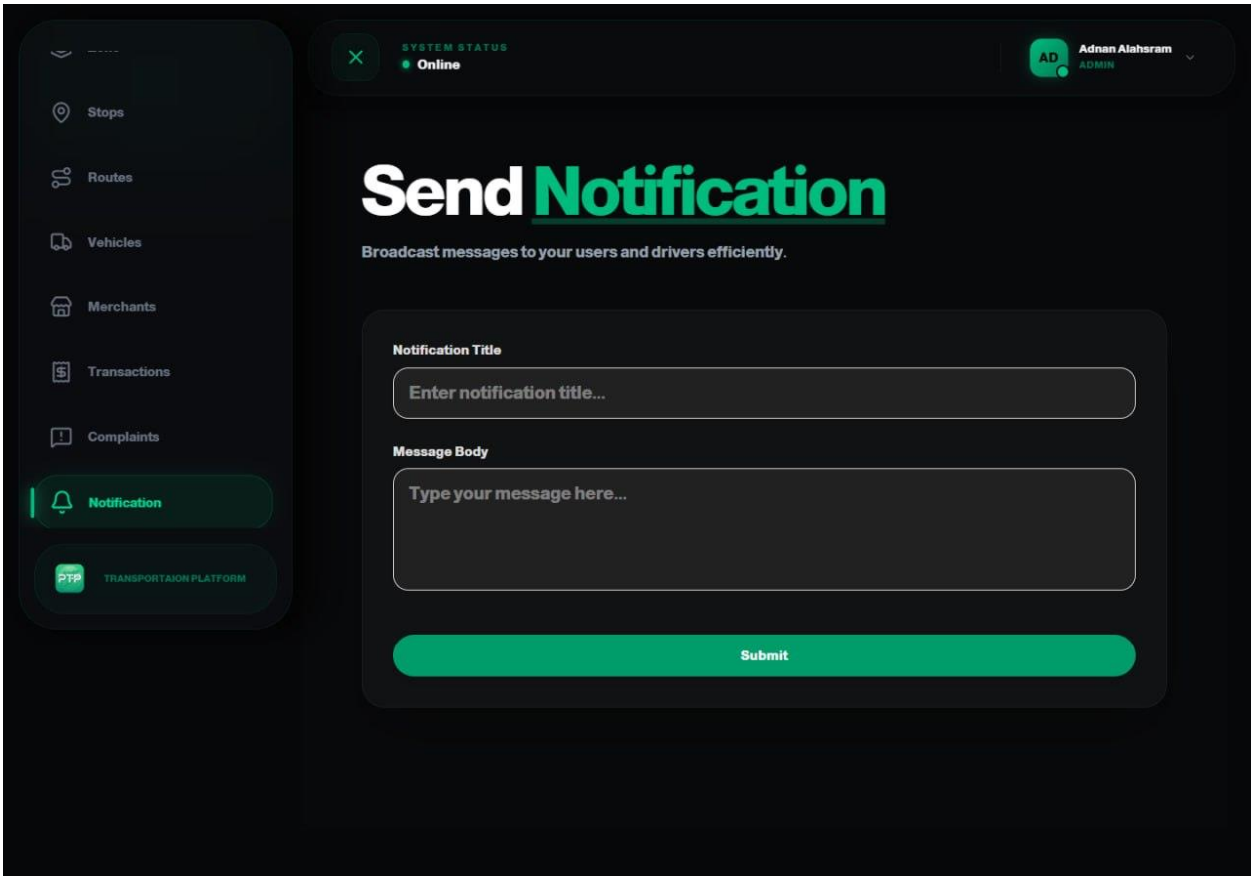

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTATION PLATFORM
FLEET ARCHITECTURE
New Route
PREVIEW PATH LOGIC
Route Name
e.g. Barzza - Maisat
Ticket Price (SYP)
2500
Select Route Stops
Select...
Select Zone
Select...
Departure Point
Map
Satellite
Damascus
دمشق
Arrival Point
Deploy & Activate Route

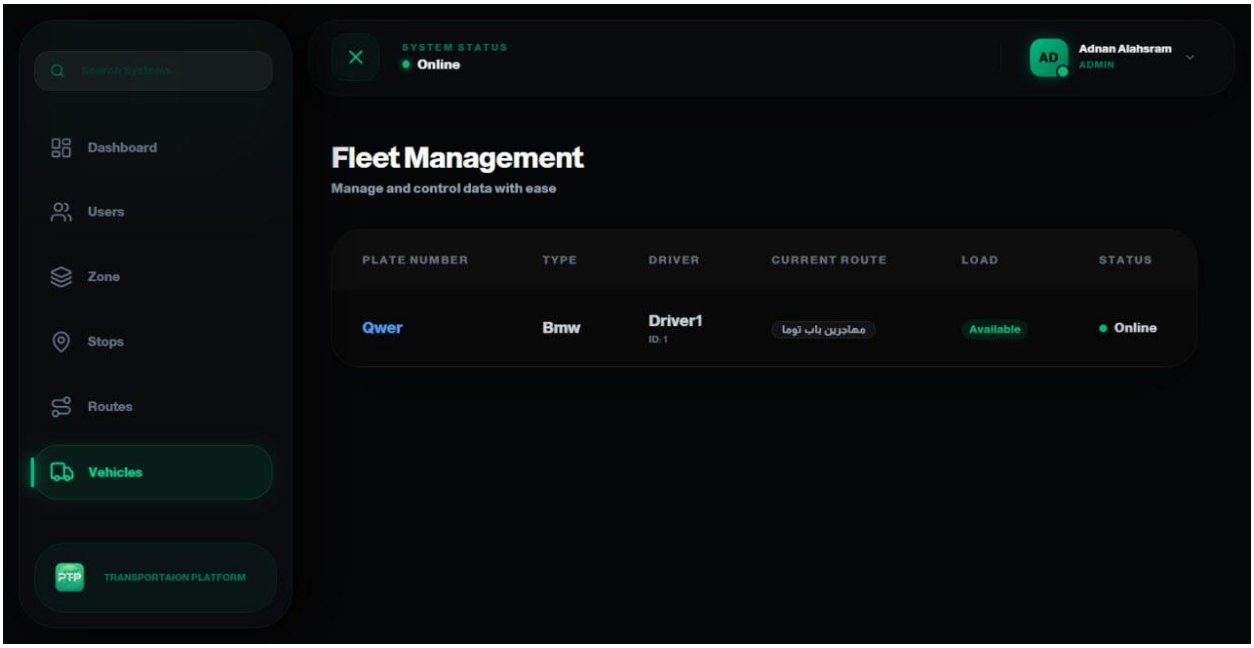

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
Routes Management
Manage and control data with ease
+ Add New Route
ROUTE NAME
PRICE
DISTANCE
STOPS
ZONE
CREATED AT
مهاجرين باب توما
3,000 SYP
9.05 km
3 Stops
damascus
07/08/2026

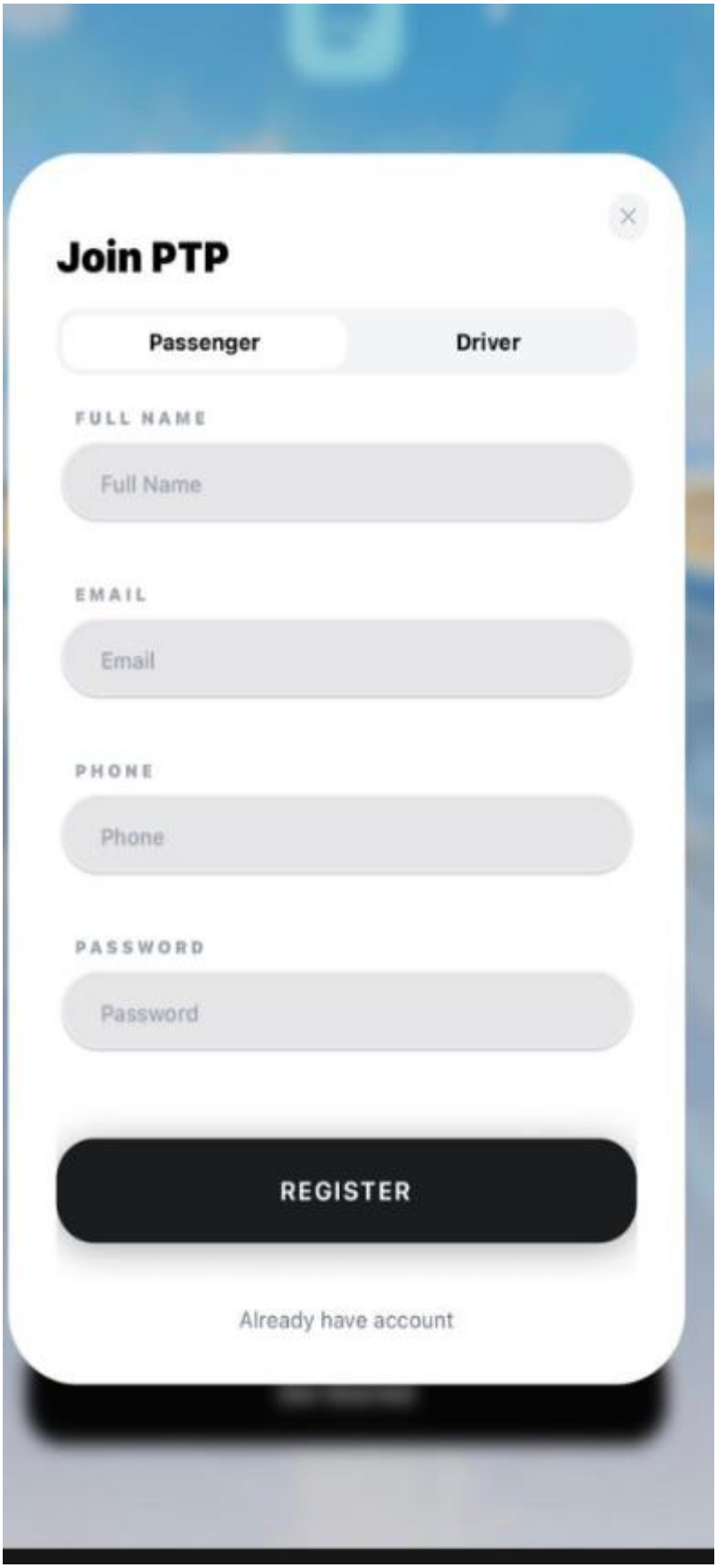

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
bmw#Qwer
ONLINE
LIVE TRACKING ACTIVE
Map
Satellite
Damascus
دمشق
Abbasid Square
Class Mall Jara
Bab Bila
CURRENT COORDINATES
33.522200, 36.272227
Keyboard shortcuts
Map data ©2026 Google
SPEED
0 km/h
HEADING
0°
OFF ROUTE
1 m
STATUS
On Track
Driver Information
FULL NAME
Driver1
PHONE
0959602775
TRIP STATUS
Currently on Trip
Trip Assets
ROUTE
مهاجرين باب توما
VEHICLE ID
1
LAST UPDATE
10:53:43 AM
Linked Device
DEVICE CODE
Connected
BUS-Scanner-001

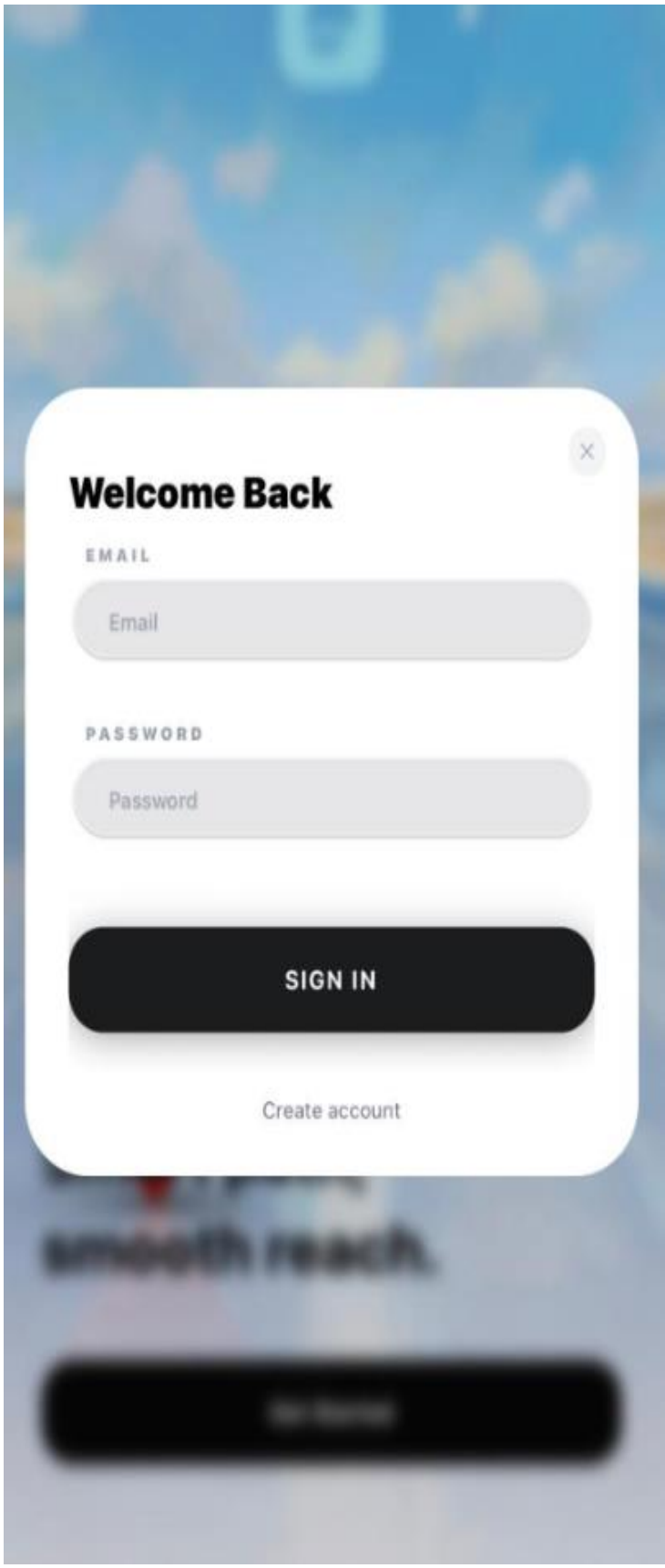

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Stops
Routes
Vehicles
Merchants
Transactions
Complaints
Notification
TRANSPORTAION PLATFORM
Merchants Management
Manage and control data with ease
+ Add New Merchant
MERCHANT NAME
ADDRESS
PHONE
EMAIL
CREATED AT
Adnan Alashram
دمشق - ميسات
0959602775
adnan@gmail.com
07/08/2026

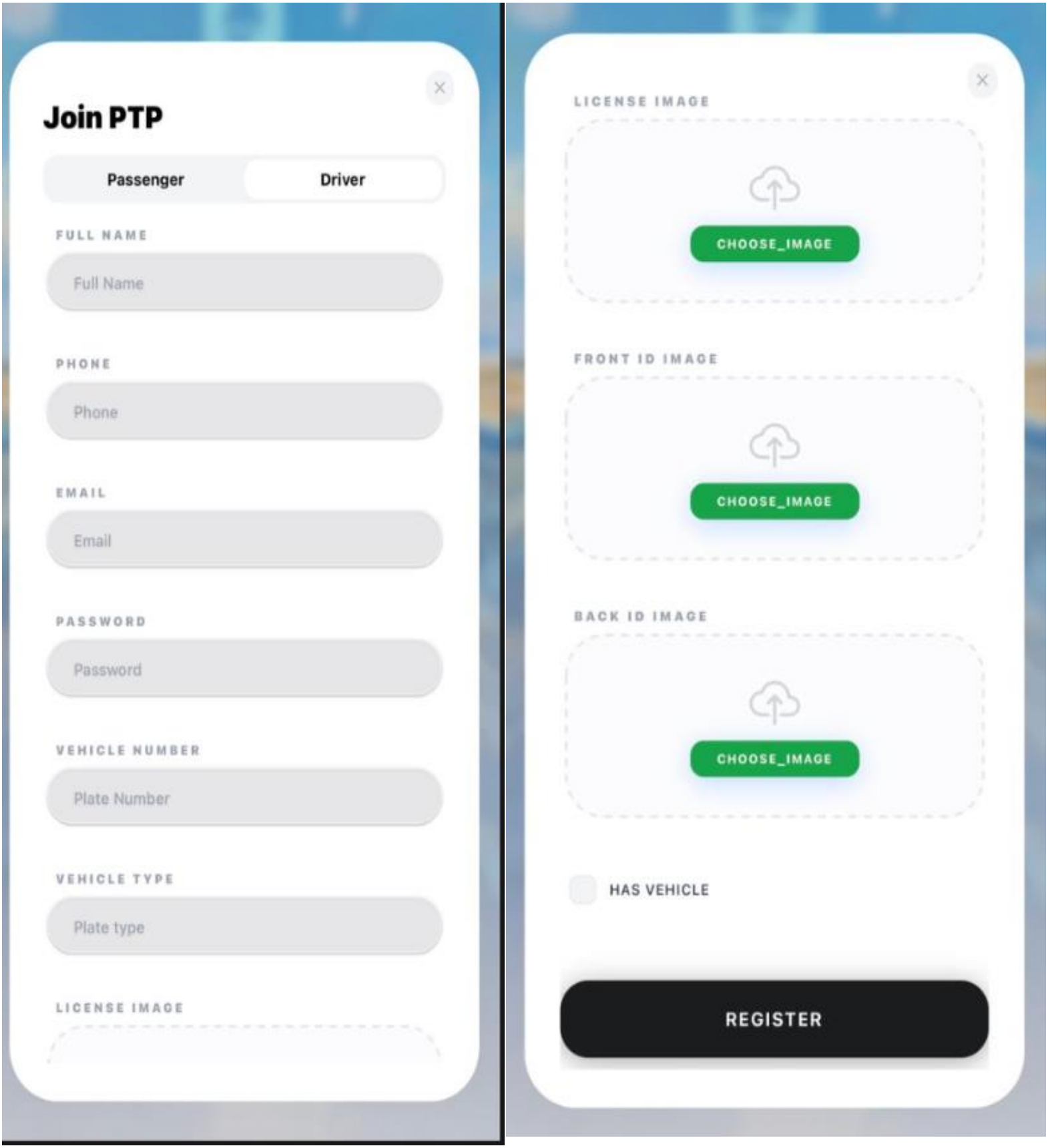

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Stops
Routes
Vehicles
Merchants
Transactions
Complaints
Notification
TRANSPORTAION PLATFORM
Transactions Management
Manage and control data with ease
ID
CARD NUMBER
MERCHANT
ZONE
AMOUNT
METHOD
DATE & TIME
#2
5967 1145 3988 6193
Adnan Alashram
rural
100,000 SYP
Cash
08/08/2026, 10:55:39
#1
5967 1145 3988 6193
Adnan Alashram
damascus
200,000 SYP
Cash
07/08/2026, 22:25:00

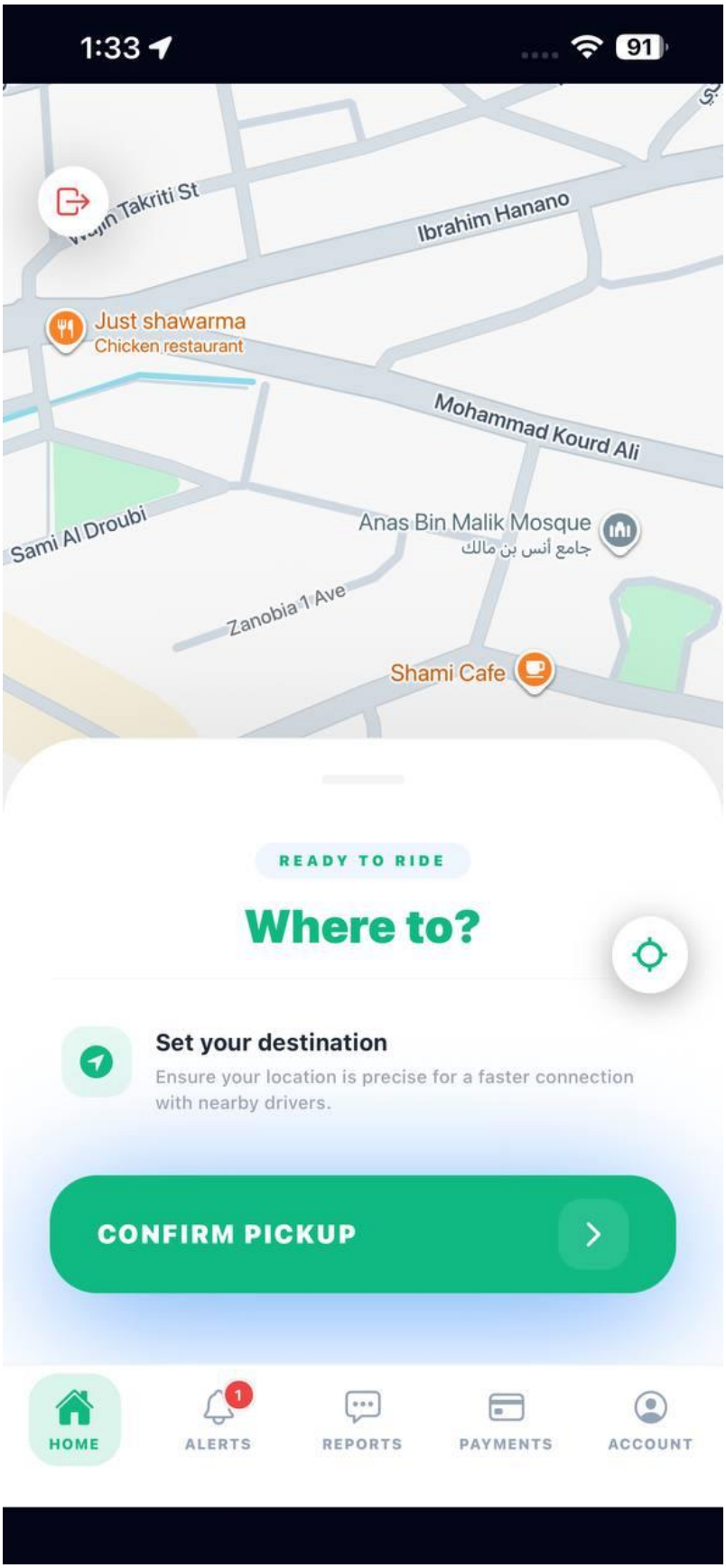

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Stops
Routes
Vehicles
Merchants
Transactions
Complaints
Notification
TRANSPORTAION PLATFORM
Send Notification
Broadcast messages to your users and drivers efficiently.
Notification Title
Enter notification title...
Message Body
Type your message here...
Submit

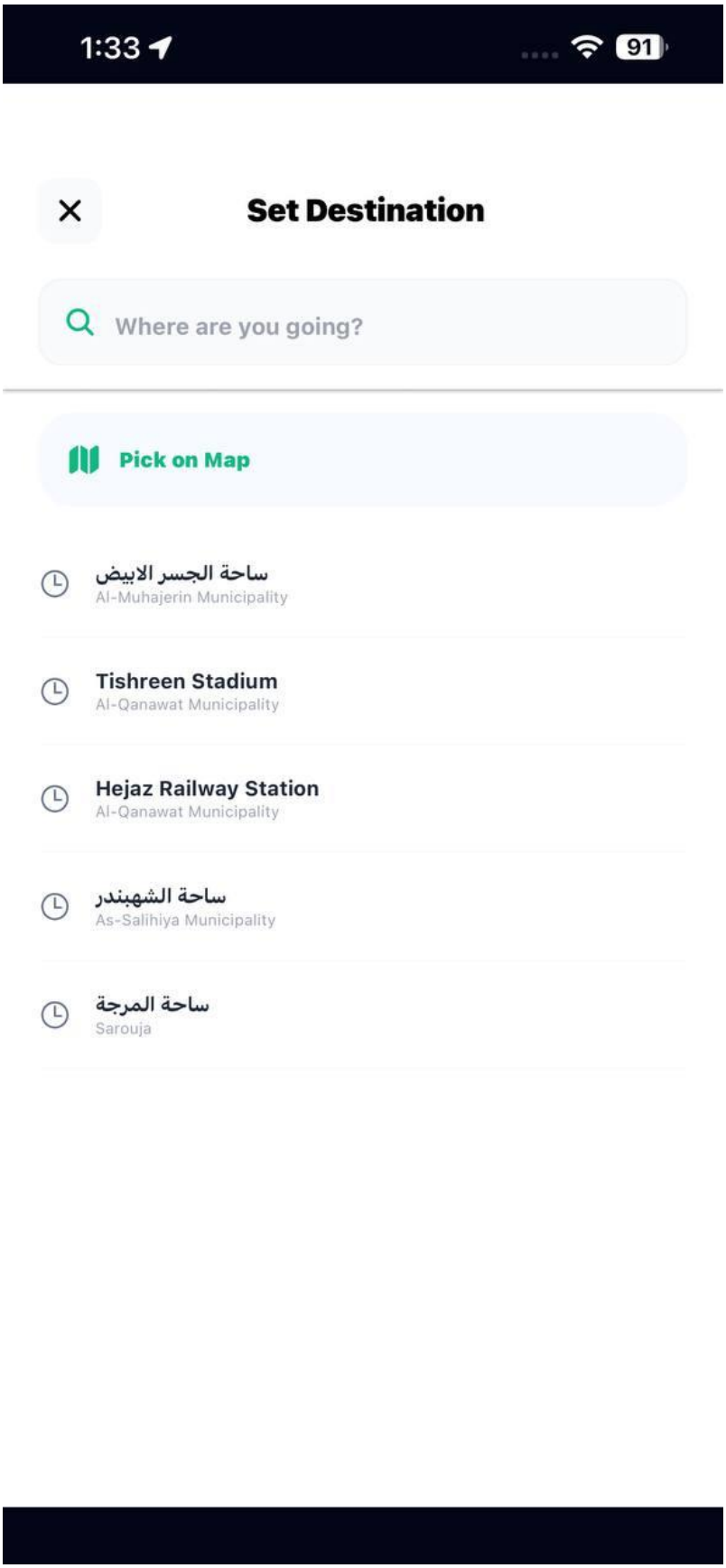

SYSTEM STATUS
Online
Adnan Alahsram
ADMIN
Dashboard
Users
Zone
Stops
Routes
Vehicles
TRANSPORTAION PLATFORM
Fleet Management
Manage and control data with ease
PLATE NUMBER
TYPE
DRIVER
CURRENT ROUTE
LOAD
STATUS
Qwer
Bmw
Driver1
ID: 1
مهاجرين باب توما
Available
Online

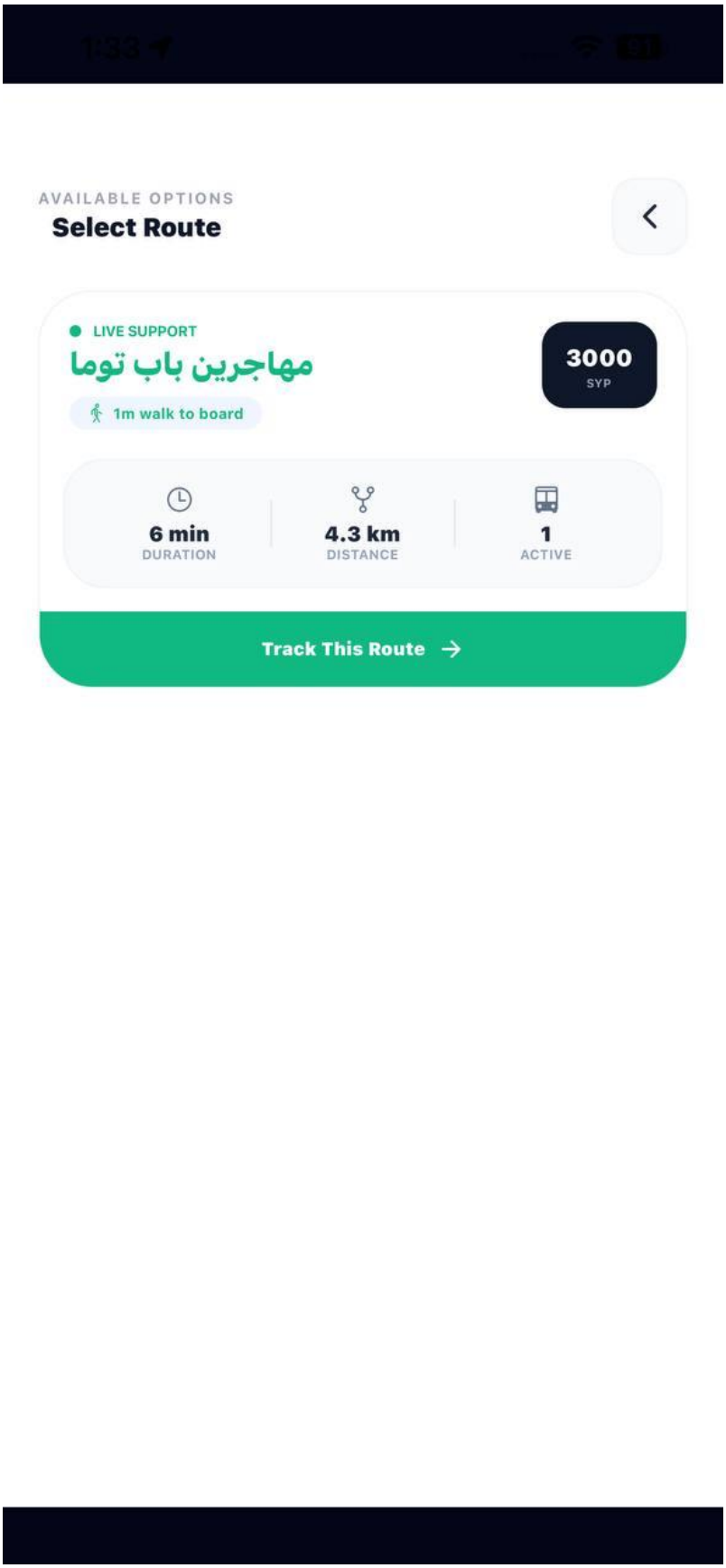

Join PTP
Passenger
Driver
FULL NAME
Full Name
EMAIL
Email
PHONE
Phone
PASSWORD
Password
REGISTER
Already have account

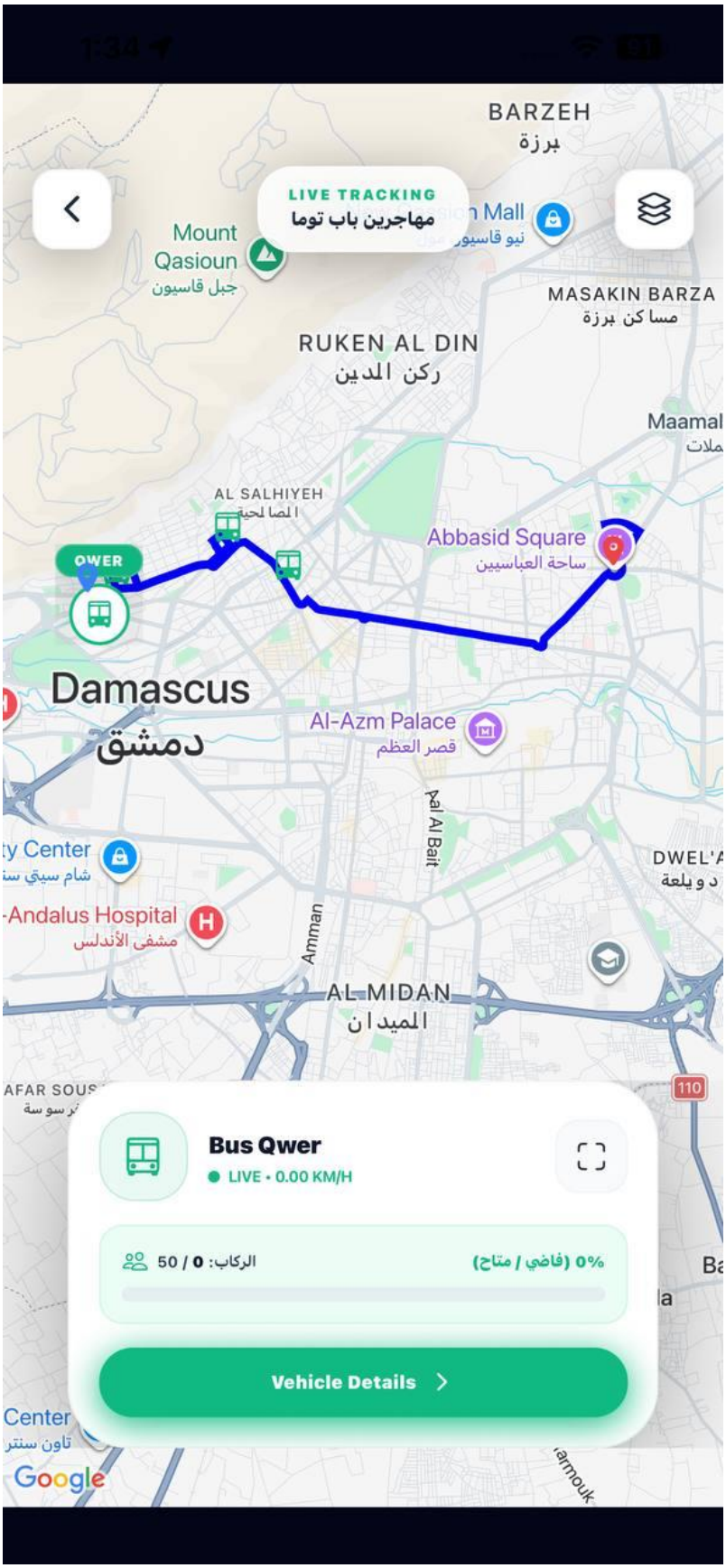

Welcome Back
EMAIL
Email
PASSWORD
Password
SIGN IN
Create account

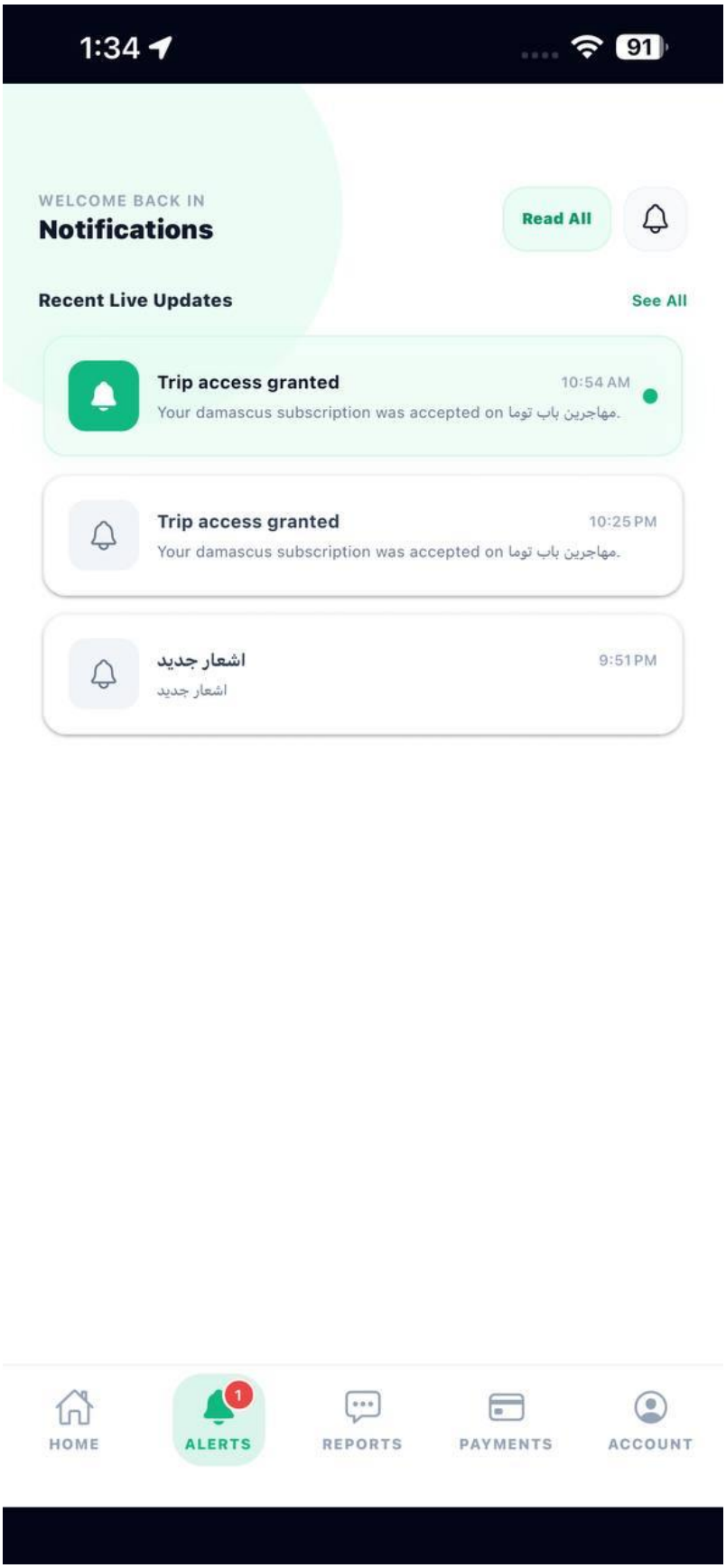

Join PTP
Passenger
Driver
FULL NAME
Full Name
PHONE
Phone
EMAIL
Email
PASSWORD
Password
VEHICLE NUMBER
Plate Number
VEHICLE TYPE
Plate type
LICENSE IMAGE
LICENSE IMAGE
CHOOSE_IMAGE
FRONT ID IMAGE
CHOOSE_IMAGE
BACK ID IMAGE
CHOOSE_IMAGE
HAS VEHICLE
REGISTER

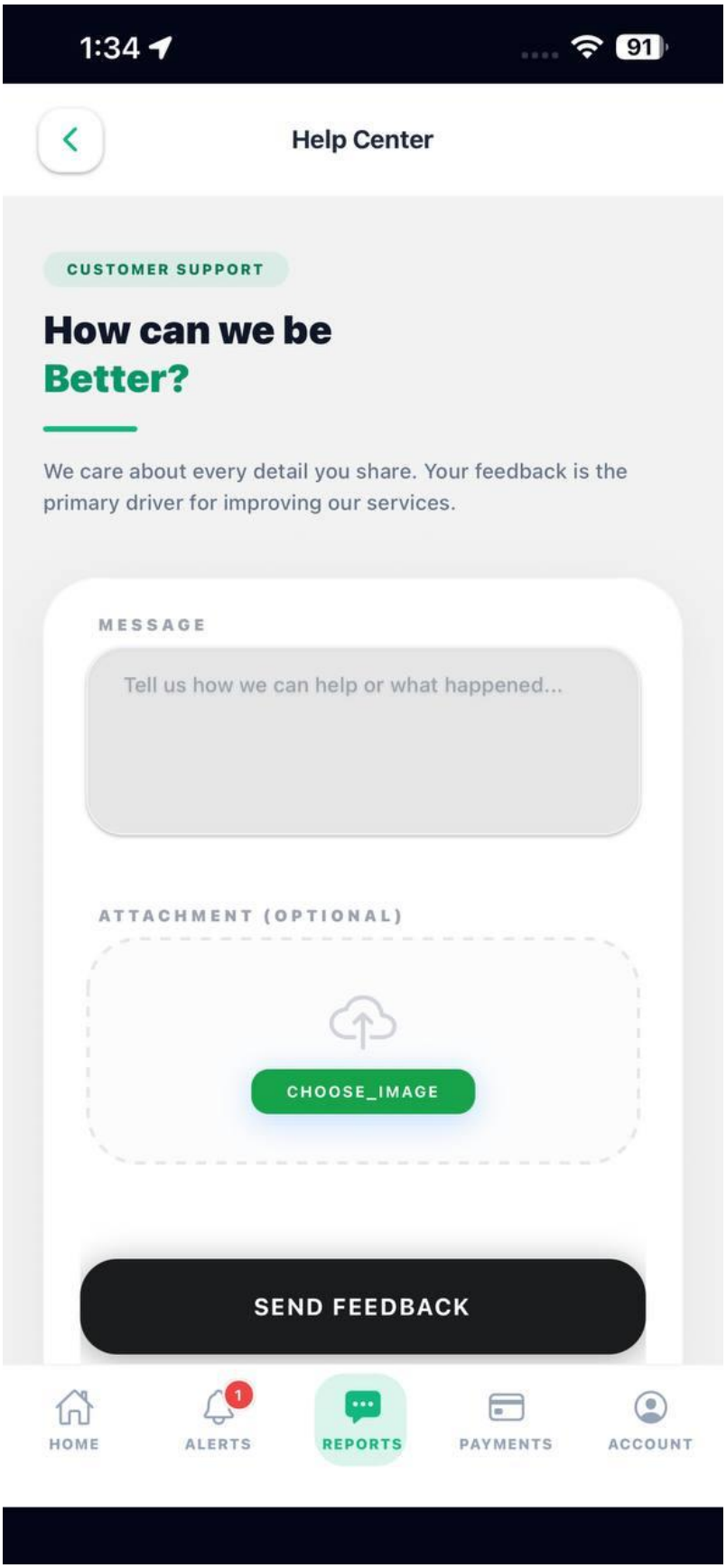

1:33
91
Takriti St
Ibrahim Hanano
Just shawarma
Chicken restaurant
Mohammad Kourd Ali
Sami Al Droubi
Anas Bin Malik Mosque
جامع أنس بن مالك
Zanobia 1 Ave
Shami Cafe
READY TO RIDE
Where to?
Set your destination
Ensure your location is precise for a faster connection with nearby drivers.
CONFIRM PICKUP
HOME
ALERTS
REPORTS
PAYMENTS
ACCOUNT

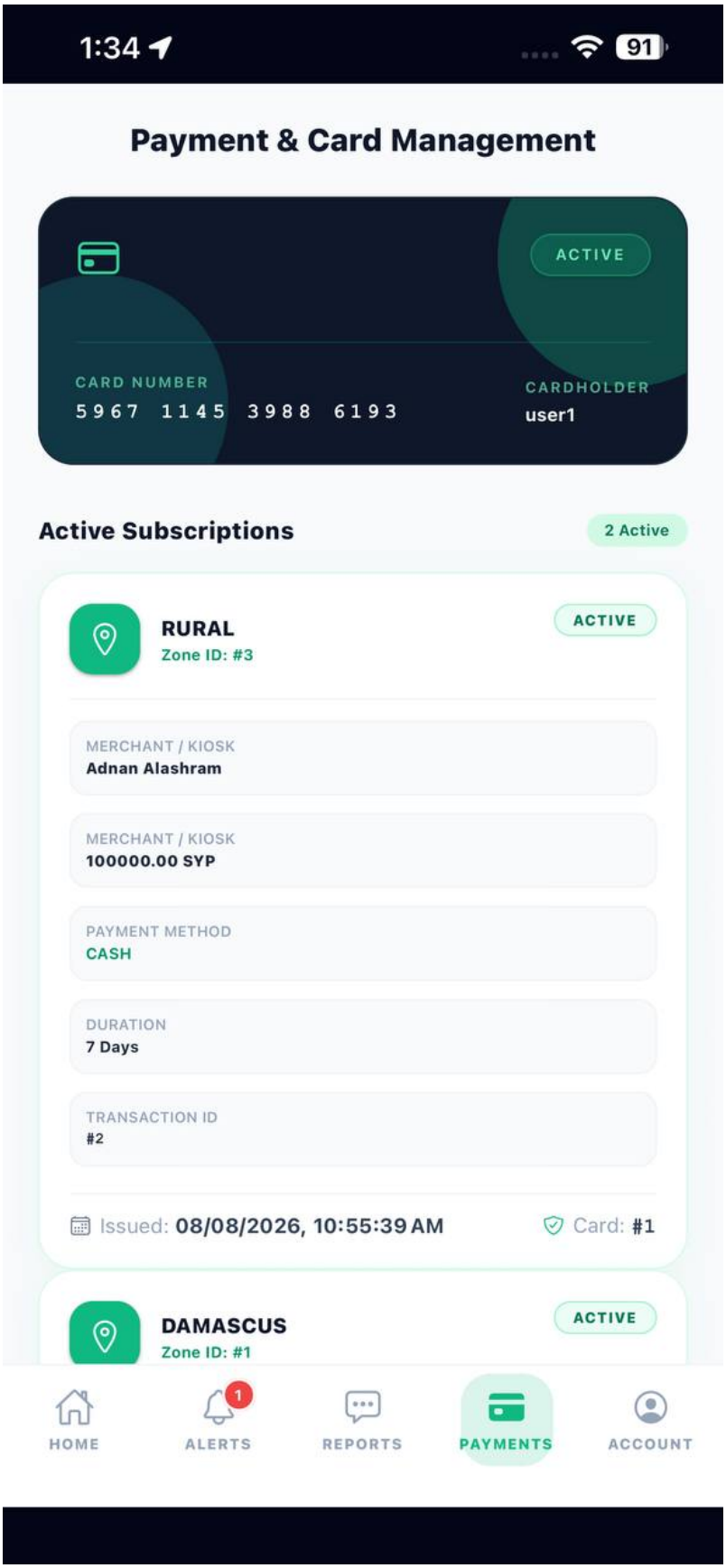

1:33
91
Set Destination
Where are you going?
Pick on Map
ساحة الجسر الابيض
Al-Muhajerin Municipality
Tishreen Stadium
Al-Qanawat Municipality
Hejaz Railway Station
Al-Qanawat Municipality
ساحة الشهبندر
As-Salihiya Municipality
ساحة المرجة
Sarouja

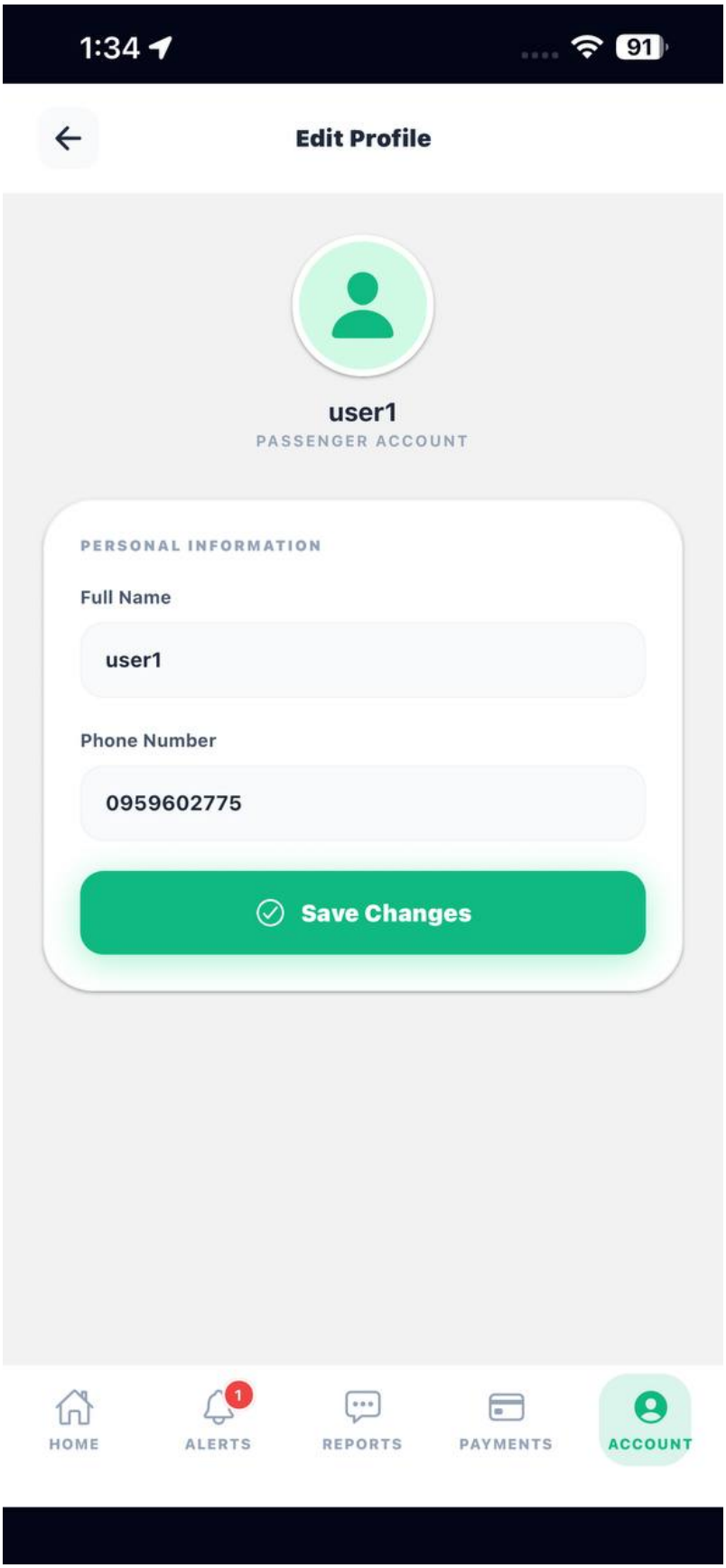

AVAILABLE OPTIONS
Select Route
LIVE SUPPORT
مهاجرين باب توما
1m walk to board
3000
SYP
6 min
DURATION
4.3 km
DISTANCE
1
ACTIVE
Track This Route

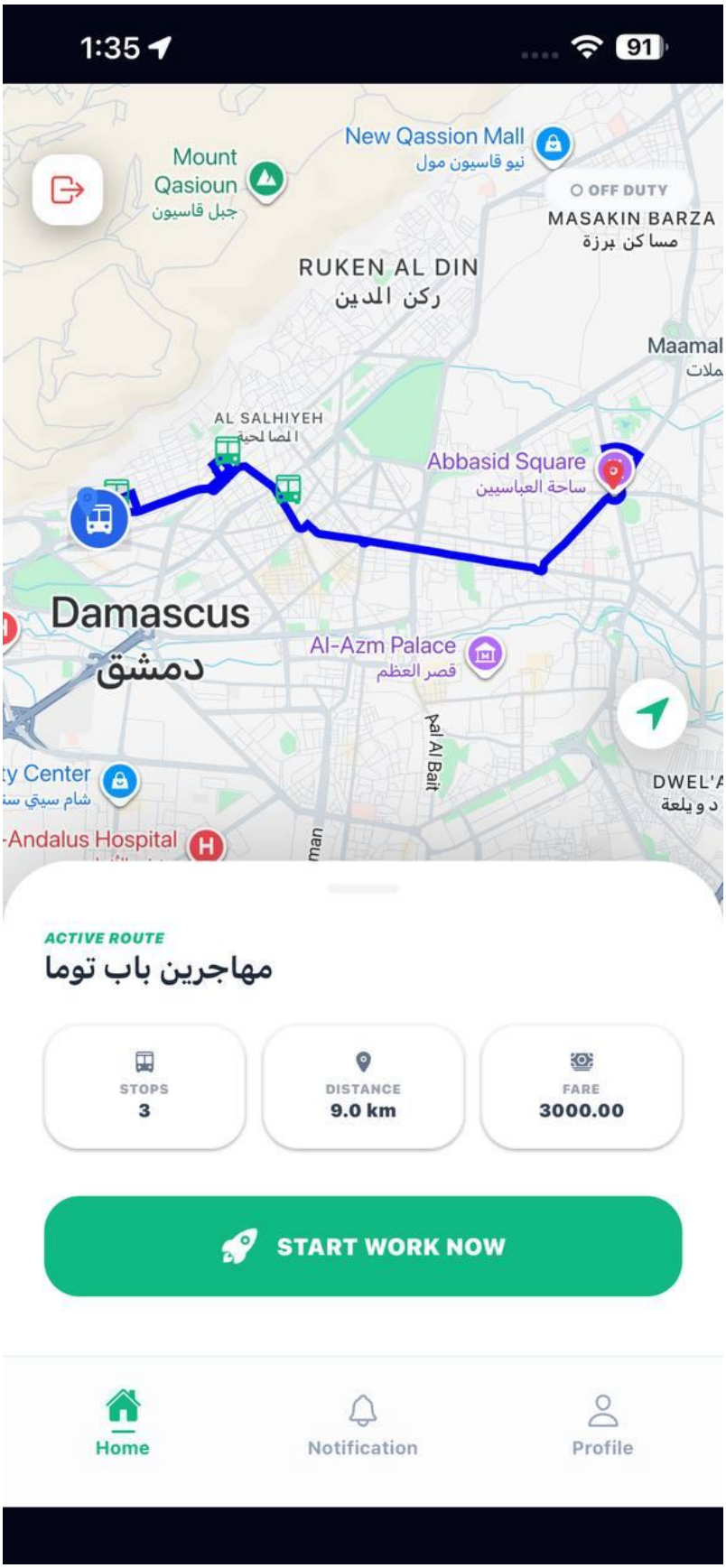

BARZEH
برزة
LIVE TRACKING
مهاجرين باب توما
Mount Qasioun
جبل قاسيون
MASAKIN BARZA
مساكن برزة
RUKEN AL DIN
ركن الدين
Maamal
AL SALHIYEH
Abbasid Square
ساحة العباسيين
QWER
Damascus
دمشق
Al-Azm Palace
قصر العظم
Amman
AL-MIDAN
الميدان
Bus Qwer
LIVE • 0.00 KM/H
الركاب: 0 / 50
0% (فاضي / متاح)
Vehicle Details
Google

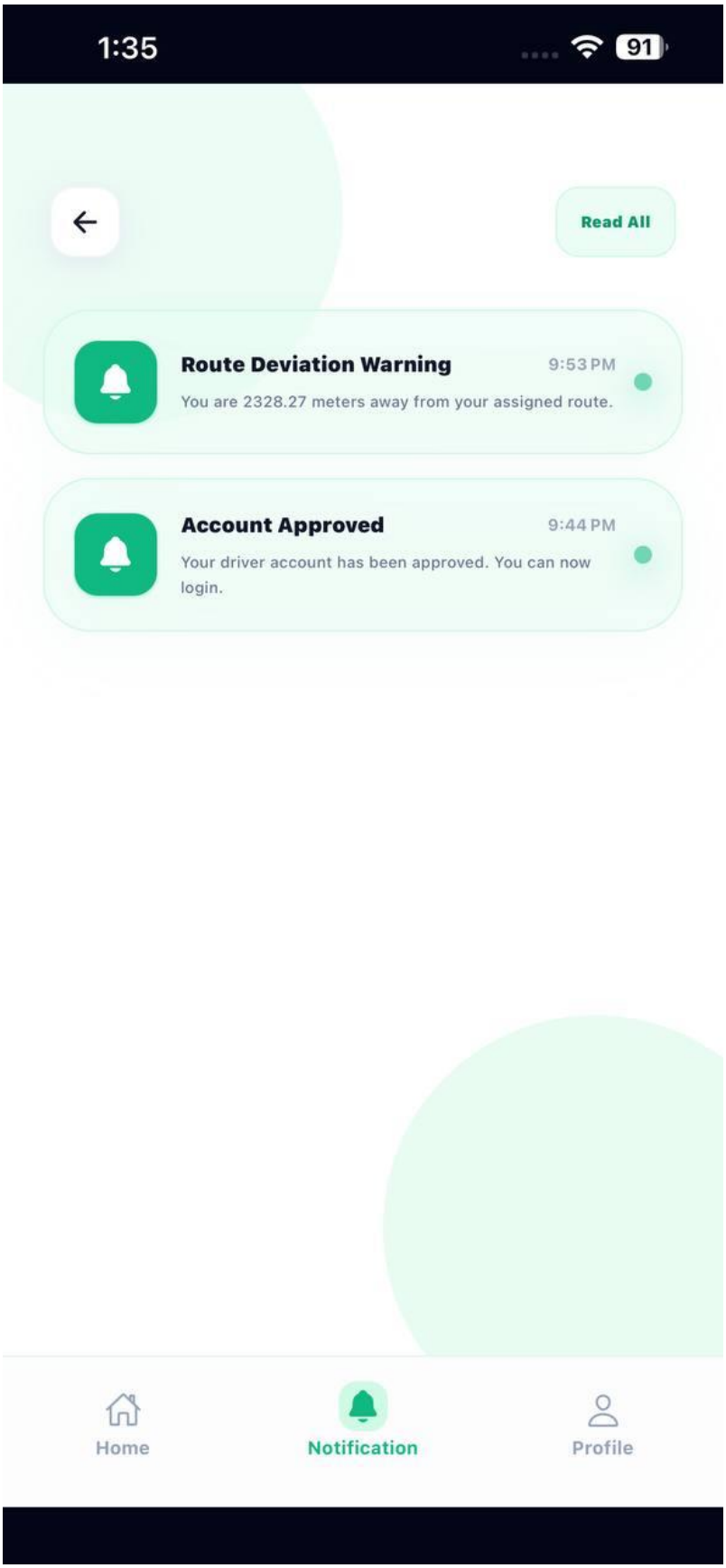

1:34
91
WELCOME BACK IN
Notifications
Read All
Recent Live Updates
See All
Trip access granted
10:54 AM
Your damascus subscription was accepted on مهاجرين باب توما.
Trip access granted
10:25 PM
Your damascus subscription was accepted on مهاجرين باب توما.
اشعار جديد
9:51 PM
اشعار جديد
HOME
ALERTS
REPORTS
PAYMENTS
ACCOUNT

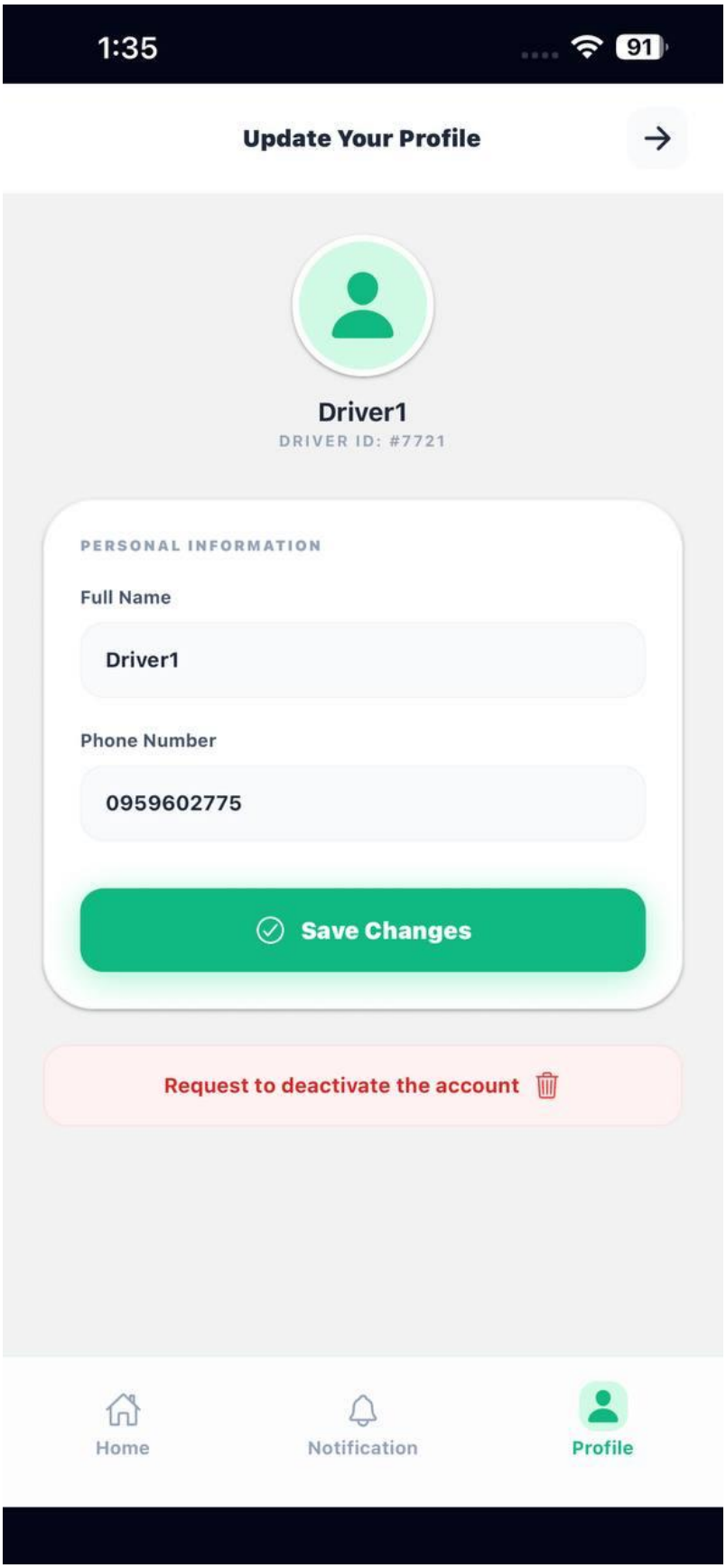

1:34
91
Help Center
CUSTOMER SUPPORT
How can we be
Better?
We care about every detail you share. Your feedback is the primary driver for improving our services.
MESSAGE
Tell us how we can help or what happened...
ATTACHMENT (OPTIONAL)
CHOOSE_IMAGE
SEND FEEDBACK
HOME
ALERTS
1
REPORTS
PAYMENTS
ACCOUNT

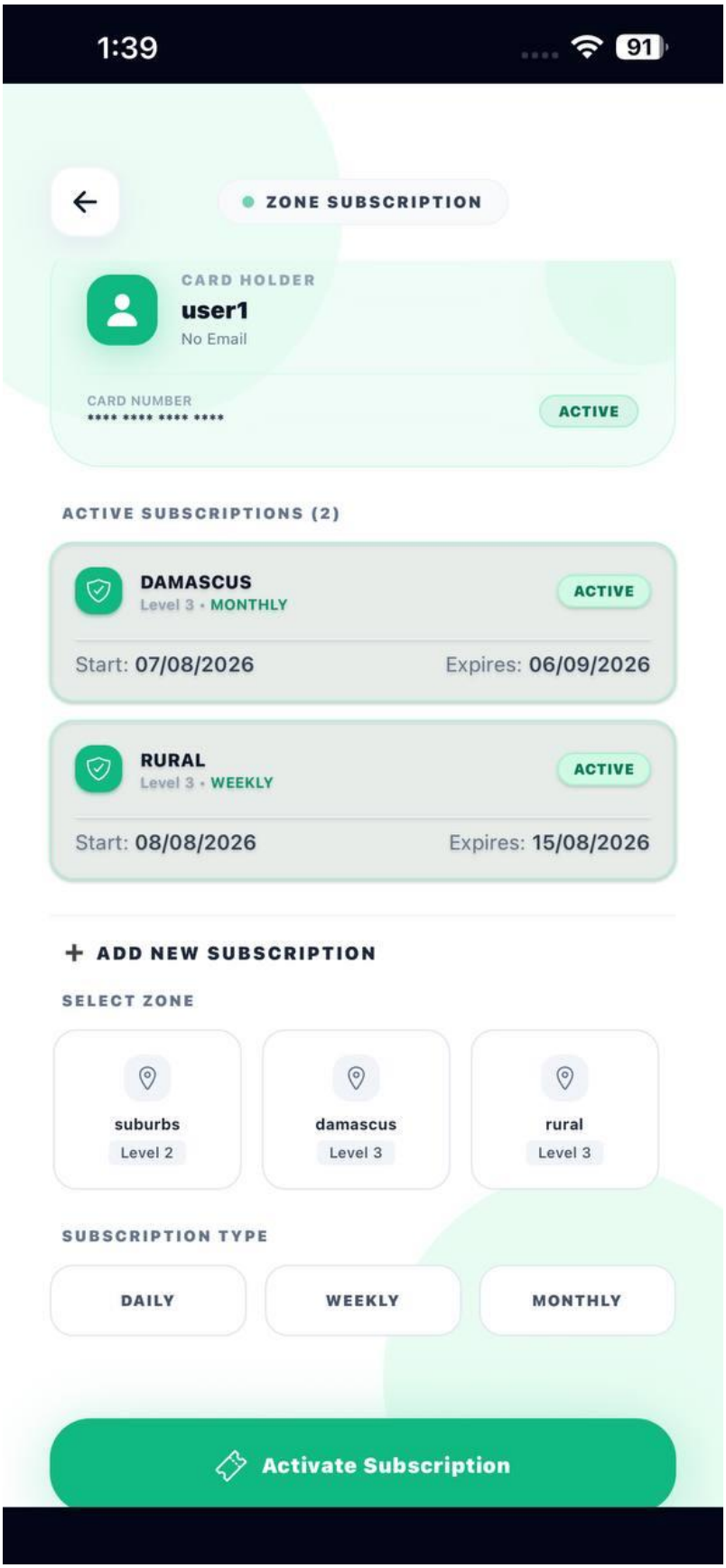

1:34
91
Payment & Card Management
ACTIVE
CARD NUMBER
5967 1145 3988 6193
CARDHOLDER
user1
Active Subscriptions
2 Active
RURAL
Zone ID: #3
ACTIVE
MERCHANT / KIOSK
Adnan Alashram
MERCHANT / KIOSK
100000.00 SYP
PAYMENT METHOD
CASH
DURATION
7 Days
TRANSACTION ID
#2
Issued: 08/08/2026, 10:55:39 AM
Card: #1
DAMASCUS
Zone ID: #1
ACTIVE
HOME
ALERTS
1
REPORTS
PAYMENTS
ACCOUNT

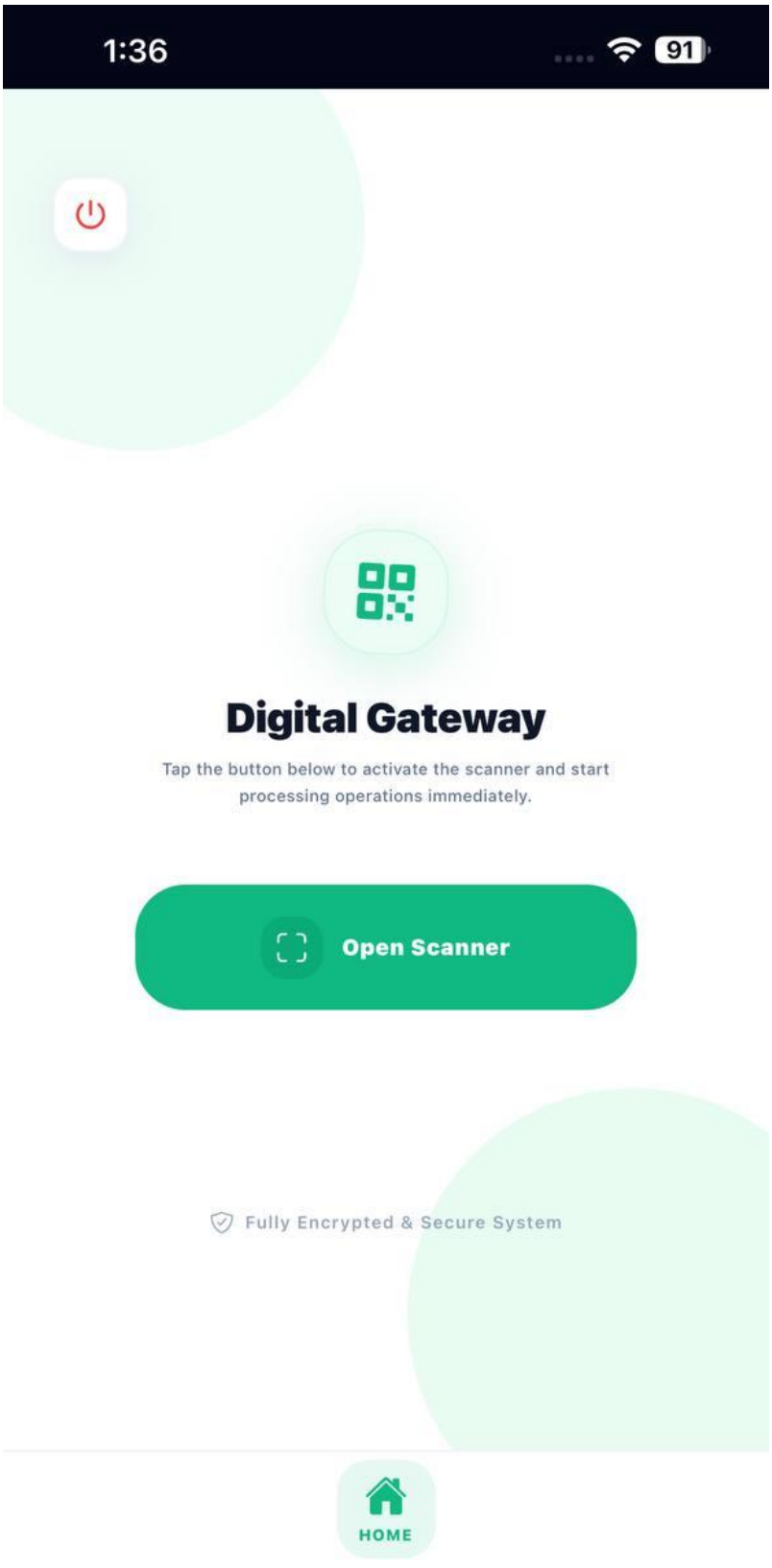

1:34
91
Edit Profile
user1
PASSENGER ACCOUNT
PERSONAL INFORMATION
Full Name
user1
Phone Number
0959602775
Save Changes
HOME
ALERTS
1
REPORTS
PAYMENTS
ACCOUNT


1:35
91
New Qassion Mall
نيو قاسيون مول
Mount Qasioun
جبل قاسيون
OFF DUTY
MASAKIN BARZA
مساكن برزة
RUKEN AL DIN
ركن الدين
Maamal
AL SALHIYEH
Abbasid Square
ساحة العباسيين
Damascus
دمشق
Al-Azm Palace
قصر العظم
DWEL'A
Andalus Hospital
ACTIVE ROUTE
مهاجرين باب توما
STOPS
3
DISTANCE
9.0 km
FARE
3000.00
START WORK NOW
Home
Notification
Profile


1:35
91
Read All
Route Deviation Warning
9:53 PM
You are 2328.27 meters away from your assigned route.
Account Approved
9:44 PM
Your driver account has been approved. You can now login.
Home
Notification
Profile


1:35
91
Update Your Profile
Driver1
DRIVER ID: #7721
PERSONAL INFORMATION
Full Name
Driver1
Phone Number
0959602775
Save Changes
Request to deactivate the account
Home
Notification
Profile


1:39
91
ZONE SUBSCRIPTION
CARD HOLDER
user1
No Email
CARD NUMBER
**** **** **** ****
ACTIVE
ACTIVE SUBSCRIPTIONS (2)
DAMASCUS
Level 3 • MONTHLY
ACTIVE
Start: 07/08/2026
Expires: 06/09/2026
RURAL
Level 3 • WEEKLY
ACTIVE
Start: 08/08/2026
Expires: 15/08/2026
+ ADD NEW SUBSCRIPTION
SELECT ZONE
suburbs
Level 2
damascus
Level 3
rural
Level 3
SUBSCRIPTION TYPE
DAILY
WEEKLY
MONTHLY
Activate Subscription


1:36
91
Digital Gateway
Tap the button below to activate the scanner and start processing operations immediately.
Open Scanner
Fully Encrypted & Secure System
HOME

2. Achievement of Project Objectives

The objectives defined in the first chapter were reviewed, and it was found that most of the main project objectives were successfully achieved through the development and implementation of the proposed system.

A mobile application was developed that allows users to register and select their account type as either a passenger or a driver. The system also provides a mechanism for reviewing driver registration requests, where administrators can approve or reject applications through the administrative dashboard.

In addition, an administrative dashboard was implemented to manage users, buses, routes, and other system-related data efficiently and effectively. The objective of bus tracking was also achieved by integrating GPS technology, allowing passengers to view the real-time locations of buses on a map and improving the overall transportation experience.

Furthermore, a notification system was implemented to inform drivers when their registration requests are approved or rejected. A complaint management feature was also developed, enabling users to submit complaints and feedback to the administration, which can then be reviewed through the dashboard.

However, the favorites feature was not implemented in the current version of the project due to the focus on the core functionalities and limited development time. This feature can be considered for future enhancements of the system.

Based on the achieved results, it can be concluded that the system successfully met its primary objectives and provided an effective solution for managing transportation services and monitoring buses in a more efficient and user-friendly manner.

3. Comparative Analysis with Existing Systems

A comparison was conducted between the proposed system and several local transportation and service applications used in Syria, such as YallaGo and similar platforms. The purpose of this comparison is to highlight the main differences in terms of performance and operational approach. The comparison focuses on three key criteria: speed, accuracy, and cost.

Criterion	Existing Systems (e.g., YallaGo)	Proposed System
Speed	These systems generally provide good service; however, data update speed may sometimes be affected by factors such as internet quality or system load, which can lead to slight delays in displaying information.	The system provides real-time updates of bus locations using GPS and WebSocket technologies, enabling near-instant data updates and improving the overall user experience.
Accuracy	Location accuracy in some applications may vary due to reliance on multiple data sources or slight delays in updates, which can affect real-time precision in certain cases.	The system relies directly on GPS data from the driver, which ensures higher accuracy in displaying real-time bus locations.
Cost	These applications typically operate under commercial models and larger infrastructure, which may result in higher operational costs.	The proposed system is designed to be lightweight and can initially operate on limited resources, making it more cost-effective and easier to scale in the future.

Overall, the proposed system offers a practical and suitable solution for the targeted domain, with noticeable improvements in data update speed and tracking accuracy, while maintaining low operational costs and providing better scalability for future enhancements.

4. Strengths and Weaknesses Analysis

- Strength

  The system has several strengths that contributed to achieving the project objectives and improving the user experience. The most important strengths include:

1.Providing real-time bus tracking using GPS technology, allowing passengers to easily and accurately monitor bus locations.

2.Having an integrated administrative dashboard that enables efficient management of users, buses, routes, and other system-related data.

3.Supporting two types of users (passengers and drivers), with the ability for administrators to review and approve or reject driver registration requests.

4.Providing a notification system that informs drivers about the approval or rejection of their registration requests.

5.Including a complaint management feature that allows users to submit complaints and feedback directly to the administration.

6.Offering a simple and user-friendly interface that enables users to interact with the system without requiring advanced technical knowledge.

7.Utilizing modern technologies in the development of the mobile application, administrative dashboard, and backend system, which facilitates future maintenance and enhancements.

- Weaknesses

  Despite the advantages offered by the system, there are several aspects that could be improved in future versions, including:

  1.The absence of a direct communication or chat system between users and the administration.

  2.The limited functionality of the notification system, which is currently restricted to driver registration approval and rejection notifications.

3.The lack of a favorites feature for saving frequently used routes or locations.

4.The absence of an advanced complaint tracking mechanism that allows users to follow the status of their submitted complaints.

5.The accuracy of real-time tracking depends on the availability and quality of the driver's internet connection and GPS signal.

In general, the strengths of the system outweigh its weaknesses, as the project successfully focused on implementing the core functionalities and achieving its primary objectives, while leaving room for future improvements and additional features.

5. Key Performance Indicators (KPIs)

The performance of the proposed system was evaluated through practical testing of its core functionalities during the development phase. The tests focused on system responsiveness, bus tracking functionality, user management operations, and complaint submission processes.

The results showed that the system was able to perform the required operations efficiently within the local development environment. Bus locations were displayed correctly on the map, user registration and driver approval processes were completed successfully, and complaints were submitted and managed without significant issues. In addition, the administrative dashboard provided smooth access to system data and management functions.

Regarding response time, the system demonstrated acceptable performance during testing, with data being retrieved and displayed within a reasonable timeframe. The GPS tracking functionality also operated as expected, depending on the accuracy of the available GPS data and network connectivity.

It should be noted that all testing was conducted in a local development environment, where the mobile application, backend server, database, and administrative dashboard were executed locally. Therefore, the obtained performance results reflect local testing conditions only.

Since the system was not deployed on a production hosting environment or tested under real-world operational loads, it was not possible to accurately measure advanced performance indicators such as large-scale user capacity, server scalability, long-term availability, or real-world response times. These evaluations can be conducted in future work after deploying the system to a production environment.

Overall, the conducted tests indicate that the system successfully fulfills its intended functions and provides satisfactory performance within the scope of the project requirements.

6. Project Summary and Key Achievements

This project focused on the design and development of an (Public transportation platform) aimed at improving user experience and facilitating the monitoring of transportation services. A mobile application was developed that allows users to register and select their account type as either a passenger or a driver, as well as view real-time bus locations on a map using GPS technology.

In addition, a comprehensive administrative dashboard was implemented to manage users, buses, routes, and driver registration requests. A complaint management feature was also provided, enabling users to submit their feedback and issues to the administration. The project contributed to delivering a practical solution that enhances the management of transportation services and provides more accurate and accessible information to users, while utilizing modern technologies and ensuring the system can be further extended in the future.

7. Challenges and Difficulties Faced by the Project

During the development of the project, several technical, organizational, and time-related challenges were encountered. One of the main technical challenges was the implementation of the notification system. An initial attempt was made to use WebSocket technology to implement real-time notifications similar to the bus tracking system. However, this approach was not suitable for push notifications in mobile applications, especially when the application runs in the background, as WebSocket connections may not remain active consistently in such cases.

Another challenge was the integration of Firebase Cloud Messaging (FCM). Although Firebase provides a powerful notification service, its implementation required additional configuration and testing on real physical devices, as well as more development time for proper setup and integration, which was not available within the project timeline.

As a result, Expo Notifications was adopted as an alternative solution, as it provided a simpler and more stable approach for handling push notifications within the Expo-based React Native application.

In addition, time constraints represented a significant challenge, limiting the ability to implement more advanced features and further optimizations. Despite these challenges, the core functionalities of the system were successfully implemented and tested.

8. Future Development Suggestions

The system can be further enhanced in the future by adding a set of improvements and features that increase performance and improve the overall user experience. One of the main suggestions is the implementation of an intelligent chatbot system that allows users to submit quick inquiries and receive instant responses regarding application usage, trips, and available services. This would reduce the need for direct communication with the administration and improve user support efficiency.

Another enhancement is the addition of a Favorites feature, allowing users to save frequently used routes, locations, or trips for faster and more convenient access during regular usage.

It is also recommended to improve the notification system by adding more event-based alerts, such as bus delays, route changes, or bus arrival notifications at stations, which would significantly enhance the user experience.

Furthermore, the complaint management system can be improved by introducing status tracking (e.g., pending, in progress, resolved, rejected), providing users with greater transparency and better monitoring of their submitted requests.

In addition, further improvements can be made to the user interface and user experience (UI/UX) to make the system more intuitive and responsive, including features such as Dark Mode support and performance optimization for low-end devices.

Finally, the system can be deployed to a real production environment instead of a local setup, enabling large-scale testing, supporting more users, and improving scalability and overall system stability.

9. Recommendations

This section includes a set of recommendations that may benefit future students who will work on similar projects, as well as institutions that may adopt this system in the future.

For future students, it is recommended to focus on the system analysis and requirement gathering phase before starting implementation, as this helps reduce changes during development and saves both time and effort. It is also advisable to divide the project into independent modules such as the mobile application, backend server, and administrative dashboard, and develop each module separately to improve maintainability and simplify debugging.

Continuous testing throughout the development process is also highly recommended, especially for critical features such as location tracking and notifications, to ensure system stability and reliability. In addition, it is better to prioritize core functionalities in the early stages and then gradually add advanced features such as UI/UX improvements or additional functionalities.

For institutions that may adopt this system, it is recommended to deploy the application in a real production environment instead of a local setup, in order to ensure better performance and scalability. Regular maintenance and continuous system updates are also important to guarantee long-term stability. Moreover, providing user training and clear documentation will help ensure smooth adoption and efficient use of the system.

Overall, applying these recommendations would enhance system quality, improve usability, and ensure long-term sustainability and scalability.

Chapter 6 Report OverView

1. Introduction

This chapter provides an overview of the report structure and explains the purpose of each chapter. It aims to guide the reader through the organization of the report and illustrate how each section contributes to the development and documentation of the Public Transportation Platform.

2. Report Structure and Purposes

Chapter 1: Introduction

This chapter introduces the project, presents the problem statement, defines the project objectives, and provides an overview of the proposed Public Transportation Platform. It also explains the organization of the report.

Chapter 2: Theoretical Study and Analysis

This chapter presents the theoretical background and fundamental concepts related to public transportation systems. It also includes the system analysis phase, covering requirements gathering, user stories, software requirements specification (SRS), use case modeling, and initial test cases.

Chapter 3: Design and Architecture

This chapter describes the system design and architecture. It includes the overall system structure, database design, UML diagrams, system components, and the relationships between different modules of the platform.

Chapter 4: Testing and Implementation

This chapter presents the practical implementation of the system, including the technologies and tools used during development. It also covers system testing activities, test execution, test results, and quality evaluation to verify that the system meets its requirements.

Chapter 5: Results and Recommendations

This chapter summarizes the outcomes of the project and evaluates the extent to which the project objectives have been achieved. It also discusses the system's strengths, limitations, future improvements, and recommendations for further development.

3. Summary

This chapter provides a general overview of the report structure and explains the purpose of each chapter. The organization of the report ensures a logical progression from project introduction and analysis to system design, implementation, testing, and final evaluation, providing a comprehensive presentation of the Public Transportation Platform.

References

- Sommerville, I. Software Engineering, 10th Edition, Pearson, 2015.
- Fowler, M. UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd Edition, Addison-Wesley, 2004.
- React Native Documentation
- Expo Documentation
- Fielding, R. Architectural Styles and the Design of Network-based Software Architectures, Doctoral Dissertation, University of California, Irvine, 2000.